Minnesota State University, Mankato

CIS4/544 REPORT

Sprint 3

Team 1:

David Onyango	davidomondi.onyango@mnsu.edu
Eunsoo Park	eunsoo.park@mnsu.edu
Kiara Boyer	kiara.boyer@mnsu.edu
Pramudya Wicaksono	pramudya.wicaksono@mnsu.edu
Rei Onishi	rei.onishi@mnsu.edu

TABLE OF CONTENTS

Introduction **4**
Objectives 4
Deliverables 5
1. Compare your SQL implementation in the Sprint2 with the proposed solution for Qlist1 5
1.1. Comparison report with Implement the Qlist1 against system catalog of Oracle System Catalog 5
1.1.3. For every schema, list Table_ID, Table_Name, attribute_id, attribute_Name, PK_ID, FK_ID, NN_ID, constraint_type, and constraint_Name 6
1.1.4. For every schema, analyze PK, FK,and IDX 6
1.1.4.1. list tables without PK 6
1.1.4.2. list tables with PK but without FK 6
1.1.4.3. list tables with PK but without IDX 6
1.1.5. For every database/schema 6
1.1.5.1. rank them according to number of NN (Not Null) have created 6
1.1.5.2. For every table list fields with NN 7
1.1.6. For every database, list tables without PK 7
1.1.7. For every database, list tables without IDX 7
1.1.8. For every database, list tables with PK but without FK 8
1.1.9. For every database/schema, list tables with IDX, index name (constraint name), fields used in IDX 8
1.1.10. For every database/schema, list tables with CC 9
1.2. Comparison report with Implement the Qlist1 against system catalog of MSSQL System Catalog 10
1.2.1. For every schema, list Schema, # tables, # records 10
1.2.2. For every schema, list table name, # attributes, # PK_ attr, # FK_ attr, # NN_attr, # IDX_attr, # CC_attr, # UQ_attrs, # PGM_attr" 10
1.2.3. For every schema, list Table_ID, Table_Name, attribute_id, attribute_Name, PK_ID, FK_ID, NN_ID, constraint_type, and constraint_Name 10
1.2.4. For every schema, analyze PK, FK,and IDX 10
1.2.4.1. list tables without PK 10
1.2.4.2. list tables with PK but without FK 10
1.2.4.3. list tables with PK but without IDX 11
1.2.5. For every database/schema 11
1.2.5.1. rank them according to number of NN (Not Null) have created 11
1.2.5.2. For every table list fields with NN 11
1.2.6. For every database, list tables without PK 12
1.2.7. For every database, list tables without IDX 12
1.2.8. For every database, list tables with PK but without FK 12
1.2.9. For every database/schema, list tables with IDX, index name (constraint name), fields used in IDX 13

1.2.10. For every database/schema, list tables with CC..... 14
1.3. Comparison report with Implement the Qlist1 against DGDB database in MSSQL..... 14
1.3.1. For every schema, list Schema, # tables, # records.....14
1.3.2. For every schema, list table name, # attributes, # PK_ attr, # FK_ attr, # NN_attr, # IDX_attr, # CC_attr, # UQ_attrs, # PGM_attr"..... 15
1.3.3. For every schema, list Table_ID, Table_Name, attribute_id, attribute_Name, PK_ID, FK_ID, NN_ID, constraint_type, and constraint_Name.....15
1.3.4. For every schema, analyze PK, FK,and IDX..... 15
1.3.4.1. list tables without PK..... 15
1.3.4.2. list tables with PK but without FK..... 15
1.3.4.3. list tables with PK but without IDX..... 15
1.3.5. For every database/schema..... 15
1.3.5.1. rank them according to number of NN (Not Null) have created.....15
1.3.5.2. For every table list fields with NN..... 16
1.3.6. For every database, list tables without PK..... 16
1.3.7. For every database, list tables without IDX..... 16
1.3.8. For every database, list tables with PK but without FK..... 17
1.3.9. For every database/schema, list tables with IDX, index name (constraint name), fields used in IDX..... 17
1.3.10. For every database/schema, list tables with CC..... 18
2. Repeat SQL implementation done in Sprint2 but replacing it with database programming..... 19
2.1. Implement the Qlist1 against the system catalog of Oracle System Catalog..... 19
2.1.1. For every schema, list Schema, # tables, # records.....19
2.1.2. For every schema, list table name, # attributes, # PK_ attr, # FK_ attr, # NN_attr, # IDX_attr, # CC_attr, # UQ_attrs, # PGM_attr"..... 20
2.1.3. For every schema, list Table_ID, Table_Name, attribute_id, attribute_Name, PK_ID, FK_ID, NN_ID, constraint_type, and constraint_Name..... 22
2.1.4. For every schema, analyze PK, FK,and IDX..... 23
2.1.4.1. list tables without PK..... 23
2.1.4.2. list tables with PK but without FK.....24
2.1.4.3. list tables with PK but without IDX..... 24
2.1.5. For every database/schema..... 25
2.1.5.1. rank them according to number of NN (Not Null) have created..... 25
2.1.5.2. For every table list fields with NN.....26
2.1.6. For every database, list tables without PK..... 27
2.1.7. For every database, list tables without IDX.....28
2.1.8. For every database, list tables with PK but without FK.....29
2.1.9. For every database/schema, list tables with IDX, index name (constraint name), fields used in IDX..... 30
2.1.10. For every database/schema, list tables with CC.....31
2.2. Implement the Qlist1 against the system catalog of MSSQL System Catalog..... 34
2.3. Implement the Qlist1 against the DGDB database in MSSQL..... 47
3. Expand Pivot implementation in Sprint2 by including Pivot interface..... 58
3.1. Develop a design for the Pivot User Interface (PUI).....58

3.2. Implement PUI design using Excel Pivot worksheet.....58
3.3. Modify the pivot worksheet developed in Sprint2 to match he proposed solution for Qlist1.....59
3.3.1. For every schema, list Schema, # tables, # records..... 59
3.3.1.1. Oracle.....59
3.3.1.2. MSSQL.....59
3.3.1.3. DGDB.....60
3.3.2. For every schema, list table name, # attributes, # PK_ attr, # FK_ attr, # NN_attr, # IDX_attr, # CC_attr, # UQ_attrs, # PGM_attr.....60
3.3.2.1. Oracle.....60
3.3.2.2. MSSQL.....61
3.3.2.3. DGDB.....62
3.3.3. For every schema, list Table_ID, Table_Name, attribute_id, attribute_Name, PK_ID, FK_ID, NN_ID, constraint_type, and constraint_Name..... 63
3.3.3.1. Oracle..... 63
3.3.3.2. MSSQL..... 63
3.3.3.3. DGDB..... 64
3.3.4. For every schema, analyze PK, FK,and IDX..... 64
3.3.4.1. list tables without PK.....64
3.3.4.1.1. Oracle.....64
3.3.4.1.2. MSSQL.....65
3.3.4.1.3. DGDB.....65
3.3.4.2. list tables with PK but without FK.....65
3.3.4.2.1. Oracle.....65
3.3.4.2.2. MSSQL.....65
3.3.4.2.3. DGDB.....65
3.3.4.3. list tables with PK but without IDX..... 66
3.3.4.3.1. Oracle.....66
3.3.4.3.2. MSSQL.....66
3.3.4.3.3. DGDB.....66
3.3.5. For every database/schema,..... 66
3.3.5.1. rank them according to number of NN (Not Null) have created..... 66
3.3.5.1.2. MSSQL..... 66
3.3.5.2. For every table list fields with NN.....67
3.3.5.2.2. MSSQL..... 67
3.3.6. For every database, list tables without PK.....68
3.3.6.2. MSSQL..... 68
3.3.7. For every database, list tables without IDX..... 69
3.3.7.1. Oracle.....69
3.3.7.2. MSSQL.....69
3.3.7.3. DGDB.....70
3.3.8. For every database, list tables with PK but without FK (D).....70
3.3.8.1. Oracle.....70

3.3.8.2. MSSQL....71
3.3.8.3. DGDB....72
3.3.9. For every database/schema, list tables with IDX, index name (constraint name), fields used in IDX....72
3.3.9.1. Oracle....72
3.3.9.2. MSSQL....73
3.3.9.3. DGDB....73
3.3.10. For every database/schema, list tables with CC....74
3.3.10.1. Oracle....74
3.3.10.2. MSSQL....74
3.3.10.3. DGDB....74
3.4. Link all sheets to the PUI templates....75
Scrum Report....76
Analysis....82
Appendix....82
Next Sprint....104
1. Sprint 4 Planning....104
2. Sprint Goal....104
3. Deliverables....104

Introduction

The goal of Sprint 3, which ran from February 17th to February 28th, 2025, was to improve SQL implementations from Sprint 2 by comparing Qlist1 queries across various database system catalogs, such as the DGDB database in MSSQL, Oracle System Catalog (OSC), and MSSQL System Catalog (MSC). We used database programming techniques to replace standard SQL queries, examined metadata inconsistencies, and extended our Pivot implementation by creating an Excel Pivot User Interface (PUI). Creating a structured PUI template, making sure that the results of SQL, PL/SQL, and pivot worksheets were consistent, and connecting all of the sheets to the PUI were among the main tasks. We created a sprint report and presentation, updated project management tools like Trello and Gantt charts, and thoroughly documented our findings in reports.

Objectives

The primary objective of this sprint is to conduct a comprehensive comparison and verification of metadata across three critical database systems: DGDB (MSSQL), Oracle System Catalog (OSC), and MSSQL System Catalog (MSC). By leveraging SQL programming and Pivot Table analysis, we aim to identify discrepancies and inconsistencies in the metadata structures of these databases. This will enable us to generate a detailed discrepancy report that highlights variations in schema, table attributes, constraints, and indexing patterns. The analysis will provide valuable insights into the alignment—or lack thereof—between the metadata representations in these systems, ensuring data integrity and consistency across the organization.

Another key objective is to enhance the efficiency and accuracy of our metadata analysis by implementing advanced database programming techniques. This includes refining SQL queries,

utilizing PL/SQL for Oracle-specific operations, and developing Pivot Table-based workflows to streamline data comparison and reporting. By automating repetitive tasks and creating reusable templates, we aim to reduce manual effort and minimize the risk of errors in our analysis. Additionally, we will focus on extending our Pivot implementation by designing an Excel Pivot User Interface (PUI) that integrates SQL and Pivot Table results into a unified, user-friendly dashboard. This will facilitate easier interpretation of findings and support decision-making processes.

Furthermore, this sprint emphasizes collaboration and task management within the Scrum framework. By breaking down tasks into smaller, manageable units and assigning them to specific team members, we aim to ensure timely completion and thorough verification of each deliverable. Each task will be reviewed by a peer to maintain quality and accuracy, and progress will be tracked using project management tools like Trello and Gantt charts. The final objective is to compile our findings into a comprehensive sprint report and presentation, documenting our methodologies, results, and recommendations for future improvements. This will serve as a valuable resource for stakeholders and provide a foundation for subsequent sprints.

Deliverables

1. Compare your SQL implementation in the Sprint2 with the proposed solution for Qlist1

1.1. Comparison report with Implement the Qlist1 against system catalog of Oracle System Catalog.

1.1.1. For every schema, list Schema, # tables, # records

Since my SQL query counts tables by schema and uses ALL_TABLES to sum all records, it offers a more comprehensive solution than the professor's and gives a clear overview of the database's contents. The professor's query, on the other hand, is incomplete because it only finds schema and table names without aggregation. While the professor's query would need thorough restructuring, my method would need to be modified to use sys.schemas, sys.tables, and sys.partitions when implemented in MSSQL. To modify my query for the DGDB database in MSSQL, I would substitute sys.dm_db_partition_stats for NUM_ROWS for record counts. Overall, my question provides a more comprehensive summary of the schema's contents than the professor's.

1.1.2. For every schema, list table name, # attributes, # PK_ attr, # FK_ attr, # NN_attr, # IDX_attr, # CC_attr, # UQ_attrs, # PGM_attr"

The professor's query and my SQL query take different approaches to summarizing schema data. Within a given schema, my query aggregates counts for different kinds of attributes per table, including primary keys, foreign keys, non-nullable attributes, indexes, check constraints, and unique constraints. It groups the results by schema and table for a brief overview and employs LEFT JOIN with multiple tables to find constraints and indexes. The professor's query, on the other hand, offers comprehensive details about every column in a table, including its schema, table name, column ID, and related constraints, all without aggregation. While my query provides a higher-level summary, it concentrates on individual column-level data and yields more detailed results.

1.1.3. For every schema, list Table_ID, Table_Name, attribute_id, attribute_Name, PK_ID, FK_ID, NN_ID, constraint_type, and constraint_Name

The purposed solution query and my SQL query take different approaches to summarizing schema data. My query extracts detailed attribute-level information, including column IDs, constraint names, and types. It uses multiple LEFT JOIN operations to bring in primary key (PK), foreign key (FK), and other constraints from all_constraints and all_cons_columns. By explicitly checking for NULL values, my query determines the presence or absence of constraints for each column.

The proposed solution, however, shifts focus to a higher-level aggregation, counting the number of each constraint type per table instead of listing individual columns. It sums the occurrences of PKs, FKs, non-null attributes (NN), indexed attributes, check constraints, and unique constraints for a given schema and groups by table name. This results in a compact summary of constraints per table rather than a column-by-column breakdown.

1.1.4. For every schema, analyze PK, FK,and IDX

1.1.4.1. list tables without PK

The proposed solution and my SQL query take different approaches to identifying tables that lack a primary key (PK). My query retrieves table names from all_tables and uses a LEFT JOIN on all_constraints to check if a table has a PK. If no PK is found (c.table_name IS NULL), the table is included in the results. The proposed solution improves performance by using EXISTS instead of LEFT JOIN. Instead of joining tables and checking for NULL values, it directly excludes tables that have a PK. Additionally, the proposed solutionsupports multiple schemas, whereas mine requires manual schema modification.

1.1.4.2. list tables with PK but without FK

My query identifies tables with a PK using user_constraints. It then performs a LEFT JOIN on user_constraints again to check for FK constraints, filtering out tables where no FK exists (fk.table_name IS NULL). The proposed solution is more optimized. Instead of LEFT JOIN, it uses NOT EXISTS, which avoids unnecessary table joins. It first selects tables with PKs and then excludes those that have an FK constraint.

1.1.4.3. list tables with PK but without IDX

My query identifies tables with PKs using user_constraints and user_tables, then performs a LEFT JOIN on user_indexes to check for missing indexes. The proposed solution is more efficient, using NOT EXISTS instead of LEFT JOIN. Instead of filtering for NULL values from a join, it directly excludes tables that have an index.

1.1.5. For every database/schema

1.1.5.1. rank them according to number of NN (Not Null) have created

Our query and the sample query both count the number of NOT NULL (NN) columns per schema, but they differ in schema selection, structure, and ordering. Our query focuses on a specific set of 10 schemas, whereas the sample query includes a broader range of 17 schemas, but since queries such as IT, FAOES, MLS and MLS2 are not available, thus the schemas won't be queried as it will lead to errors. Structurally, our query nests the results within a subquery before applying the final ORDER BY nn_count DESC, whereas the sample query performs sorting directly on the aggregated results. This nesting does not change the final outcome but may slightly affect performance. Additionally, the

naming conventions differ—our query renames OWNER as schema_name and counts as nn_count, while the sample query keeps OWNER as owner and names the count as num_nn_constraints. Due to the difference in schema selection, our query provides a more focused ranking, while the sample query covers a wider range of schemas, making direct ranking comparisons possible only for the common schemas between them.

Since the schemas that were queried are different for both, The results for both queries vary. The sample query output produced FAOES as the highest number of Not Nulls. On the other hand for our query, it produced ZeotaDB as the highest number of Not Nulls. This is consistent with the queries themselves, since ZeotaDB is not present on the sample query, and FAOES is not present on our query.

1.1.5.2. For every table list fields with NN

The sample query selects owner, table_name, and column_name, ordering the results by owner, table_name, and column_id, and filters a broader set of schemas (17 in total), such as BOOK, IT, and ZEOTA. In contrast, the our query renames the owner column as schema_name for clearer output, and orders the results by owner, table_name, and column_name instead of column_id, which leads to sorting by column names alphabetically. Additionally, our query filters a smaller set of schemas (10 in total), including BOOK, IT4, and ZEOTADB. The main difference in ordering is that sample query sorts based on the internal column structure (using column_id), while our query sorts by column_name. Furthermore, our query uses column aliases to improve readability, while sample query uses the default column names. These differences result in variations in schema selection, column naming, and sorting behavior between the two queries.

1.1.6. For every database, list tables without PK

Both our query and the sample query aim to retrieve tables without primary key constraints from specified schemas but differ in approach. Our query uses a LEFT JOIN between ALL_TAB_COLUMNS and ALL_CONSTRAINTS, checking for NULL in the constraint name to identify tables without primary keys, and explicitly groups and orders the results. The sample query, on the other hand, employs a NOT EXISTS subquery to efficiently filter tables without primary keys, which generally performs better, especially with larger datasets, as it stops processing once a match is found. While both queries return the same result, the sample query is typically more performant, especially for larger databases, due to the use of NOT EXISTS.

1.1.7. For every database, list tables without IDX

In sprint2 the SQL query implemented to identify tables without indexes created two Common Table Expressions (CTEs). The first CTE, named TableList, retrieved the schema name, table name, and object name from the all_tables and all_tab_columns views, ensuring that the table and column information are matched based on the table name and owner. It filtered out system schemas and schemas starting with 'SYS' to focus on user-defined schemas. The second CTE, named IndexList, retrieved the table name as object_name and the index name from the all_tables and all_indexes views, ensuring that the table and index information are matched based on the table name and owner. The main query then selected the schema name and table name from the TableList CTE and performed a LEFT JOIN with the IndexList CTE on the object_name to find tables without indexes.

The proposed query selects the owner and table_name from the all_tables view, which contains metadata about all tables in the database. The WHERE clause includes a subquery that checks for the existence of indexes on each table by matching the owner and table_name in the all_indexes view. If no matching index is found, the table is included in the result set.

Every record match between the two queries except for the three additional tables in FAOES in the proposed query.

1.1.8. For every database, list tables with PK but without FK

The sprint 2 SQL query designed to list tables with primary key (PK) but no foreign key (FK) used three Common Table Expressions (CTEs) to do the implementation. The TableList CTE retrieved the schema name, table name, and object name from the all_tables and all_tab_columns views, filtering out system schemas. The PKList CTE retrieved the table name and primary key constraint name from the all_constraints view, filtering for primary key constraints. The FKList CTE retrieved the table name and foreign key constraint name from the all_constraints view, filtering for foreign key constraints. The main query then joins these CTEs to find tables that have a primary key but do not have a foreign key.

The proposed query selects the owner and table name from the all_tables view, using subqueries to check for the existence of primary key constraints and the non-existence of foreign key constraints in the all_constraints view. It filters the results to include only those tables whose owners are in a specified list of schemas and limits the result set to the first 20 rows.

The sprint2 query therefore uses CTEs to break down the problem into smaller, more manageable parts, making it easier to understand and maintain. The proposed query uses subqueries within the main query to achieve the same result. This can be more efficient in terms of execution but may be harder to read and maintain. Both queries however achieve the same results.

1.1.9. For every database/schema, list tables with IDX, index name (constraint name), fields used in IDX

The two SQL queries provided are designed to retrieve index-related metadata from an Oracle database by querying the ALL_INDEXES and ALL_IND_COLUMNS views. Both queries aim to list indexes along with the tables they belong to, their names, and the indexed columns. However, they differ in approach, filtering criteria, and aggregation, leading to different result counts—191 rows for the Sprint 2 query (from Sprint 2) and 270 rows for the the proposed query (from Sprint 3).

The query from Sprint 2 uses the LISTAGG function to concatenate multiple indexed columns into a single field for each index. It groups results by schema (owner), table, index name, index type, and uniqueness, ensuring that each index appears only once with its indexed columns aggregated into a single row. Additionally, it explicitly filters by a predefined set of schemas, including BOOK, COMPANY, IT4, OES21, OOES, PROP2, PUBCO, SP, SPJ2, and ZEOTADB. This grouping mechanism, combined with the specific schema filtering, results in a more condensed dataset.

The query from Sprint 3, on the other hand, does not use aggregation. Instead, it retrieves distinct rows for each combination of schema, table, index, and indexed column, leading to multiple rows for indexes with multiple columns. The distinct operation ensures there are no duplicate rows, but since

it does not group or concatenate columns like the Sprint 2 query, indexes with multiple indexed columns appear as separate rows for each column. Moreover, it applies a broader schema filter, including additional schemas such as EDU, FAOES, MLS, MLS2, OES2, OES4, OLYMPIC, and SALES. This expanded schema set naturally increases the number of results.

The difference in result count—191 rows from the Sprint 2 query versus 270 rows from the the proposed query—can be attributed to these factors. The Sprint 2 query consolidates multiple column indexes into single rows and filters on a smaller set of schemas, reducing the overall count. The the proposed query expands the schema list and allows multiple rows for multi-column indexes, leading to a higher count. The Sprint 2 query provides a more compact, high-level view of indexes and their structure, whereas the the proposed query offers a more detailed, column-level breakdown, making it useful for granular analysis of indexed fields.

1.1.10. For every database/schema, list tables with CC

The two SQL queries aim to retrieve information about tables in specific schemas, but they focus on different aspects of indexing and constraints, leading to varying results. Sprint 2 query returns 191 results, whereas the proposed query returns only 113. This discrepancy arises from the distinct approaches each query takes in fetching data.

Sprint 2 query is primarily concerned with indexing information and indirectly captures constraints through joins with all_constraints. It retrieves schema names, table names, index names, constraint names (if applicable), and indexed columns. The query joins all_indexes with all_ind_columns to gather index details and attempts a LEFT JOIN with all_constraints to include any associated constraints. However, it does not filter explicitly for check constraints (constraint_type = 'C'). Instead, it returns all indexed tables, including those with and without check constraints, which likely contributes to the higher result count.

In contrast, the proposed query directly targets tables that contain check constraints (constraint_type = 'C'). It selects from all_tables, filtering only those tables that have an associated check constraint in all_constraints. The use of EXISTS ensures that only tables with at least one check constraint are included in the results. Additionally, the list of schemas in the proposed query is slightly different from Sprint 2 query, which further contributes to the variance in the result count. By focusing exclusively on tables with check constraints, the proposed query yields a more refined and specific result, explaining why it returns fewer rows (113) compared to Sprint 2 query's broader dataset.

Ultimately, Sprint 2 query is more index-centric and returns a broader set of indexed tables, some of which may not necessarily have check constraints. the proposed query, being more constraint-specific, provides a precise answer to the question by listing only those tables that explicitly contain check constraints. This distinction in scope and filtering criteria explains the difference in their outputs.

1.2. Comparison report with Implement the Qlist1 against system catalog of MSSQL System Catalog.

1.2.1. For every schema, list Schema, # tables, # records

The professor's query and my SQL query both seek to receive schema statistics, but they are not as complex. Using a straightforward JOIN between sys.tables and sys.partitions, my query retrieves the schema name, table count, and record count for the "Book" database. For tables where index_id <= 1, it aggregates the record count. However, the professor's query is more detailed; it uses a cursor to loop through all online databases, automatically runs SQL for each database, and stores the results in a temporary table. The professor's query handles several databases and offers a wider scope, whereas my query is more straightforward and restricted to a single database.

1.2.2. For every schema, list table name, # attributes, # PK_ attr, # FK_ attr, # NN_attr, # IDX_attr, # CC_attr, # UQ_attrs, # PGM_attr"

Based on metadata from INFORMATION_SCHEMA and sys views, my query aggregates data on schema attributes such as primary key (PK), foreign key (FK), not null (NN), index (IDX), check constraints (CC), unique constraints (UQ), and programmatic (PGM) attributes. It operates within a single database at a time. It counts the crucial attributes for every table by combining multiple joins and conditionals. In contrast, the professor's query counts them independently for each schema and runs several distinct queries across various schemas (ZEOTA, PROP, and SP) for every constraint type (PK, FK, attributes, and indexes). The professor's method requires several, distinct counts for every schema, offering more precise counts for various databases rather than a comprehensive count, whereas my query aggregates the data within a single query.

1.2.3. For every schema, list Table_ID, Table_Name, attribute_id, attribute_Name, PK_ID, FK_ID, NN_ID, constraint_type, and constraint_Name

In MSSQL, my query retrieves schema analysis details by directly checking constraints in sys.tables, sys.foreign_keys, and sys.indexes. I use LEFT JOIN operations to determine missing constraints and indexes, ensuring that results are sorted at the table level.

The proposed solution, however, divides its logic across different schemas (SP, ZEOTA, PROP) and executes multiple schema-specific queries. Instead of using sys.objects or sys.indexes directly, it leverages INFORMATION_SCHEMA.TABLE_CONSTRAINTS, which provides an alternative metadata source.

1.2.4. For every schema, analyze PK, FK,and IDX

1.2.4.1. list tables without PK

My query retrieves table names from sys.tables and performs a LEFT JOIN on sys.key_constraints to find PKs. It filters out tables where no PK is found (pk.parent_object_id IS NULL). The proposed solution takes a different approach by using INFORMATION_SCHEMA.TABLE_CONSTRAINTS, filtering out tables that do not have PRIMARY KEY constraints. Additionally, the proposed solution is executed separately for different schemas (Zeota, SP, PROP).

1.2.4.2. list tables with PK but without FK

My query selects tables with PKs from sys.tables and sys.key_constraints and performs a LEFT JOIN on sys.foreign_keys to check for missing FKs. The proposed solution takes a different approach by

using sys.foreign_key_columns instead of sys.foreign_keys. It is executed separately for SP, ZEOTA, and PROP schemas.

1.2.4.3. list tables with PK but without IDX

My query selects tables with PKs from sys.tables and sys.key_constraints and performs a LEFT JOIN on sys.indexes to check for missing indexes. The proposed solution refines this by using sys.indexes and sys.foreign_key_columns. It is executed separately for different schemas.

1.2.5. For every database/schema

1.2.5.1. rank them according to number of NN (Not Null) have created

Both the sample query and our query count NOT NULL (NN) columns per schema across multiple SQL Server databases but differ in approach and efficiency. Our query uses INFORMATION_SCHEMA.COLUMNS, iterates through databases with a cursor, and combines results using UNION ALL, making it straightforward but potentially less efficient. The sample query, on the other hand, retrieves data directly from sys.columns, sys.tables, and sys.schemas, which can improve performance by avoiding metadata abstraction. Additionally, the sample query stores results in a temporary table (#SchemaNNCounts) and applies RANK() OVER (PARTITION BY DatabaseName ORDER BY NotNullCount DESC), providing structured ranking within each database. While both rely on cursors, which can slow execution, the sample query is more efficient and offers better insights through ranking. If ranking and performance optimization are key, the sample query is preferable, while our query remains simpler and easier to modify.

In terms of results, the sample query produces all available schemas, however there are some issues regarding the ranking structure of the output. The sample query resulted in all ranks as 1 (one), which suggests there might be an issue on the Rank query. However, in our query, we use sorting instead of ranking, thus producing a descending order of the number of not-nulls for each schema.

1.2.5.2. For every table list fields with NN

Our script and the sample script both aim to retrieve non-nullable column information from multiple databases but differ in structure, approach, and flexibility. Our script uses a cursor to iterate through a predefined list of databases (BOOK, COMPANY, etc.) and constructs a dynamic SQL query that combines the results from all databases using UNION ALL. After the cursor completes, it removes the last UNION ALL and executes the dynamic SQL in a single batch with sp_executesql. This method is efficient for querying a fixed set of databases but lacks adaptability for dynamically changing environments. On the other hand, the sample script iterates through all online databases by checking state_desc = 'ONLINE' in the sys.databases view, and for each database, it runs a query to gather column information, inserting the results into a temporary table (#FieldDetails). After all databases are processed, it retrieves and displays the results. This approach is more flexible as it works with all online databases, making it suitable for environments where databases may be added or removed frequently. Additionally, the sample script uses system views like sys.columns, sys.tables, and sys.schemas to gather column data, whereas our script uses INFORMATION_SCHEMA.COLUMNS. The sample script's use of a temporary table allows for easier aggregation and retrieval of data, while our script constructs a single dynamic SQL statement. Overall, our script is ideal for a controlled set of databases, while the sample script offers more flexibility and scalability for handling all online databases.

1.2.6. For every database, list tables without PK

Both queries aim to identify tables without primary key constraints across multiple databases, but they differ in their approach and execution. The sample query creates a stored procedure that uses a cursor to iterate over all user databases (excluding system databases) and dynamically constructs SQL to identify tables without primary keys. Results are stored in a temporary table (#Results), which is later queried and dropped. This approach is more structured, with a focus on managing intermediate results in a temporary table, providing better control over data processing. It is ideal for handling large or complex sets of data, though it introduces additional setup with temporary tables.

In contrast, our query uses a cursor to iterate over a predefined list of specific databases and dynamically builds a single SQL statement that checks for tables without primary keys by using LEFT JOIN. The results are combined using UNION ALL, and the trailing union is removed before execution. This method is more compact and direct, with no intermediate table handling, making it simpler and faster for smaller datasets. However, it might be less scalable or flexible compared to the sample query when dealing with larger or more dynamic databases, as it doesn't offer the same level of result management.

1.2.7. For every database, list tables without IDX

The sprint 2 query designed to list tables without indexes for every database in SQL Server environment starts by declaring variables and creating a temporary table to store the results. A cursor is then used to iterate through each database, excluding system databases and 'DGDB'. For each database, a dynamic SQL statement which uses CTEs to create a list of tables and their associated indexes is constructed and executed. The main query then inserts into the temporary table the names of tables that do not have any associated indexes. The results are then selected from the temporary table, and the temporary table is dropped.

The proposed query on the other hand declares variables and uses a cursor to iterate through a predefined list of databases. For each database, a dynamic SQL statement is constructed and executed. The statement selects the database name, schema name, and table name from the sys.tables and sys.schemas views, performing a left join with the sys.indexes view to find tables without indexes.

Sprint 2 query uses CTEs and a temporary table to store and manage the results, which can be useful for complex queries and intermediate result sets. It also iterates through all databases, excluding system databases and DGDB. On the other hand, the proposed query directly selects from the system views and limits the scope to a predefined list of databases. This makes it more efficient, simple and straightforward.

1.2.8. For every database, list tables with PK but without FK

The Sprint 2 query was designed to list tables with primary keys (PK) but without foreign keys (FK) across all databases in an SQL Server environment. It declares variables and creating a temporary table to store the results. A cursor is used to iterate through each database, excluding system databases and 'DGDB'. For each database, a dynamic SQL statement is constructed and executed. This statement uses Common CTEs to create a list of tables, primary key constraints, and foreign key constraints. The main query then inserts into the temporary table the names of tables that have a primary key but do not have a foreign key. The results are selected from the temporary table, and the temporary table is dropped.

The proposed query begins by checking if a stored procedure named ListTablesWithPKWithoutFK exists and drops it if it does. The procedure is then created, which includes the logic to list tables with primary keys but without foreign keys. Inside the procedure, a temporary table is created to store the results. A cursor is used to iterate through a predefined list of databases, excluding system databases and 'DGDB'. For each database, a dynamic SQL statement is constructed and executed. This statement selects the database name, schema name, and table name from the INFORMATION_SCHEMA.TABLES view, checking for the existence of primary key constraints and the non-existence of foreign key constraints. The results are inserted into the temporary table, which is then selected and dropped at the end of the procedure.

Sprint 2 query uses CTEs and a temporary table to manage the results. This can be useful for complex queries and intermediate result sets. It also iterates through all databases, excluding system databases and 'DGDB'. The proposed query uses a stored procedure to encapsulate the logic, making it reusable and easier to manage. It limits the scope to a predefined list of databases, which can be more efficient and straightforward. Both queries effectively identify tables with primary keys but without foreign keys, but they differ in their approach and complexity.

1.2.9. For every database/schema, list tables with IDX, index name (constraint name), fields used in IDX

In Sprint 2, the Sprint 2 query aimed to gather indexing information from several specified databases: 'Book', 'Company', 'DGDB', 'OES2', 'Prop2', 'SP', 'Sales', and 'ZeotaDB'. It used a dynamic SQL approach with a cursor that iterated over these databases to gather details about each database's indexes. Specifically, the query provided insights into table names, index names, index types, uniqueness of indexes, and the columns involved in those indexes. The results were aggregated using the STRING_AGG function, which combined column names from the index columns into a comma-separated string. This query resulted in 168 records, focusing primarily on indexed tables and the indexes themselves, with details on how each index was structured for each database.

On the other hand, the proposed query served a similar purpose but was broader in scope and incorporated constraints. It created a stored procedure named GetConstraintColumnUsageAllDatabases, which not only looked for indexing information but also identified columns involved in constraints across a larger set of databases. This procedure used a cursor to iterate through all databases, excluding system databases like 'master', 'tempdb', 'model', and 'msdb'. Instead of just indexing data, it also collected information about constraint usage, such as primary keys, unique constraints, and other foreign keys in each database’s tables. This resulted in a total of 568 records, significantly more than the Sprint 2 query. The inclusion of constraint data likely contributed to the larger result set, as each database could have multiple constraints per table, leading to more records being returned. Furthermore, this query also utilized a temporary table #Results to store intermediate data before selecting and displaying the final output.

In conclusion, while both queries aimed to analyze the databases for indexing and constraint information, the the proposed query from Sprint 3 was more comprehensive, covering both indexes and constraints, leading to a larger result set. The Sprint 2 query from Sprint 2 focused solely on index data, which explains its smaller result count. The evolution of the the proposed query represents a broader and more detailed exploration of database schemas and their usage of indexes and constraints.

1.2.10. For every database/schema, list tables with CC

The Sprint 2 query, from Sprint 2, uses a dynamic SQL approach with a cursor to loop through a list of specified databases and retrieve check constraints from the sys.check_constraints and sys.tables system views. The query begins by declaring a cursor that fetches a list of database names (Book, Company, DGDB, OES2, Prop2, SP, Sales, ZeotaDB). The query builds dynamic SQL by concatenating the USE statement for each database with a SELECT query that fetches the check constraints, including schema name, table name, constraint name, and check condition. The dynamic SQL is then executed using sp_executesql. This query is useful when a specific set of databases is targeted, and the goal is to retrieve check constraints for those particular databases only.

On the other hand, the the proposed query, from Sprint 3, implements a stored procedure, ListTablesWithCheckConstraint, that serves a broader purpose. It first checks if the procedure already exists and drops it if necessary. It then creates a temporary table #Results to store the output, which includes the database name, schema name, table name, and constraint name. The cursor in this query loops through all databases except the system databases (master, tempdb, model, msdb, and DGDB) and builds a dynamic SQL statement to fetch tables with check constraints from the INFORMATION_SCHEMA.TABLE_CONSTRAINTS and INFORMATION_SCHEMA.TABLES views. The INSERT INTO statement ensures that the results are collected into the temporary table. Once the cursor finishes iterating over the databases, the procedure selects all the results from the temporary table and drops it at the end. This query is more flexible as it targets all user databases, providing a broader scope for listing tables with check constraints.

Comparing both queries, the main difference lies in the scope and structure. The Sprint 2 query is designed for a fixed set of databases, making it less flexible but suitable for a specific set of projects. The the proposed query, however, is designed to handle all databases (except system databases), making it more scalable and adaptable for broader environments. Additionally, the the proposed query uses a stored procedure, which is more structured and reusable in a production environment, compared to the ad-hoc dynamic SQL approach in the Sprint 2 query. The use of a temporary table in Sprint 3 helps aggregate results in one place, while the Sprint 2 query directly executes multiple dynamic queries across different databases. Finally, Sprint 3's query also incorporates the INFORMATION_SCHEMA views, which are more standardized and often preferred for portability across different SQL Server environments, while Sprint 2 directly accesses system catalog views (sys.check_constraints and sys.tables), which are more specific to SQL Server's internal structure.

1.3. Comparison report with Implement the Qlist1 against DGDB database in MSSQL.

1.3.1. For every schema, list Schema, # tables, # records

Although they adopt different strategies, both queries seek to list schema-level table and record counts. With an emphasis on base tables, my query retrieves the schema name, table name, and record count by joining system views (sys.schemas, sys.tables, and sys.partitions). Using a different database structure, the professor's query counts attributes and records per table using custom tables (DG_Attributes, DG_Tables, and DG_Schema). Whereas the professor's question examines attributes within tables, mine focuses on records in tables.

1.3.2. For every schema, list table name, # attributes, # PK_ attr, # FK_ attr, # NN_attr, # IDX_attr, # CC_attr, # UQ_attrs, # PGM_attr"

The structure and methodology of the two queries vary, but they both seek to list schema-level table and attribute statistics. My query joins other tables for constraints and counts different attributes and constraints (PK, FK, NN, etc.) straight from the DG_Tables and DG_Attributes tables. Similar work is done by the professor's query, which groups by schema and table name and uses outer joins for every constraint table. Even through using different join strategies and table structures, both queries concentrate on the same attributes. The professor's query aggregates using various identifiers (pk_id, fk_id, etc.), whereas my query uses Attributes_ID counting.

1.3.3. For every schema, list Table_ID, Table_Name, attribute_id, attribute_Name, PK_ID, FK_ID, NN_ID, constraint_type, and constraint_Name

In DGDB, my query retrieves tables without PKs, tables with PKs but without FKs, and tables with PKs but without indexes using LEFT JOIN operations on sys.tables, sys.key_constraints, and sys.indexes.

Theproposed solution, on the other hand, uses DG_Schema, DG_Tables, and specialized attribute constraint tables (DG_attributes_PK, DG_attributes_FK, DG_attributes_Index). It directly extracts tables that meet specific constraint conditions, making it more tailored to DGDB's structure.

1.3.4. For every schema, analyze PK, FK,and IDX

1.3.4.1. list tables without PK

My query uses sys.tables and sys.key_constraints to check for missing PKs in DGDB. It follows a standard relational approach, similar to MSSQL. The proposed solution, however, leverages DGDB-specific metadata tables (DG_Schema, DG_Tables, and DG_Attributes_PK). It directly filters out tables that lack a PK using DG_attributes_PK, making it more efficient.

1.3.4.2. list tables with PK but without FK

My query uses sys.tables and sys.foreign_keys to filter out tables with missing FK constraints. The proposed solution is optimized for DGDB's structured metadata. Instead of relying on system views, it uses DG_attributes_PK and DG_attributes_FK to directly check if a table with a PK is missing an FK.

1.3.4.3. list tables with PK but without IDX

My query uses sys.tables and sys.indexes to identify tables missing indexes. The proposed solution however, utilizes DG_attributes_Index, which is a dedicated DGDB metadata table. This allows for faster, more structured indexing checks than my approach.

1.3.5. For every database/schema

1.3.5.1. rank them according to number of NN (Not Null) have created

The sample query and our query both count NOT NULL (NN) constraints per schema but differ in methodology and output structure. The sample query ranks schemas based on their NN count using RANK() OVER (ORDER BY T.NN_COUNT DESC), ensuring a structured ranking system. It relies on INNER JOINs, which means schemas without NN constraints are excluded. In contrast, our query uses LEFT JOINs, ensuring all schemas are included even if they have no NN constraints, making it more comprehensive. Additionally, the sample query counts distinct NN constraints using a subquery, while our query directly counts

dan.Constraint_Type = 'NN' and orders the results by NN count. If ranking is the priority, the sample query is better, but for a broader view that includes all schemas, our query is more flexible and informative.

1.3.5.2. For every table list fields with NN

Our query and the sample query both aim to retrieve non-nullable column information but differ in their join types, filtering, and output formatting. Our query uses LEFT JOIN to include all records from the DG_Attributes table, ensuring that even if there are no matches in the DG_Attributes_NotNull, DG_Tables, or DG_Schema tables, those records will still appear in the result with NULL values. It also filters the results based on the 'NN' constraint and limits the output to specific schemas (SPJ2, ZEOTA, BOOK). In contrast, the sample query uses INNER JOIN to return only records with matching entries across all tables, providing a more restrictive result set. The sample query does not include any schema filtering and sorts the results by schema_name, Table_Name, and Attributes_Name, while also renaming the output columns to standardize their names. Therefore, our query is more specific, focusing on particular schemas and constraints, while the sample query offers a broader result set with consistent output and sorting.

1.3.6. For every database, list tables without PK

The sample query creates a stored procedure that iterates over multiple databases (excluding system databases) using a cursor. It dynamically constructs SQL for each database to find tables without primary keys by checking INFORMATION_SCHEMA.TABLE_CONSTRAINTS. The results are inserted into a temporary table (#Results), and after processing all databases, the procedure selects and drops the temporary table. This method is flexible, allowing the query to run across several databases, but it involves more complexity due to dynamic SQL and the management of intermediate results in the temporary table. It is useful for environments where querying across multiple databases is required.

In contrast, our query operates within a single database (DGDB) and is more straightforward. It retrieves tables without primary keys using LEFT JOIN between specific tables in the DGDB database, filtering those without primary keys by checking for the absence of Attributes_ID in the dbo.DG_Attributes_PK table. This approach is simpler and more efficient for single-database use cases, as it avoids the overhead of dynamic SQL and cursors, focusing on direct joins and filtering. However, it lacks the flexibility of the sample query when it comes to handling multiple databases.

1.3.7. For every database, list tables without IDX

The sprint2 query is designed to list all tables without indexes (IDX) for every database in the system. The query uses CTEs to create a list of tables and a list of indexes. It then performs a left join between these lists to find tables without indexes and inserts the results into a

temporary table. It then selects and orders the results from the temporary table before dropping it.

The proposed query is implemented in a more straightforward manner. It directly queries the DG_Tables, DG_Attributes, and DG_Attributes_Index tables to find tables without indexes. The query joins these tables and filters the results where IDX_ID is empty. This approach is simpler and more efficient as it avoids the use of cursors and temporary tables.

The Sprint2 query was more complex and involved multiple steps, including the use of cursors, dynamic SQL, and temporary tables. It is more flexible and can handle multiple databases but not effective for this scenario with only one database being queried. The proposed solution is simpler and more efficient for this single database scenario since it does not need to iterate through multiple databases.

1.3.8. For every database, list tables with PK but without FK

The sprint2 query lists all tables with primary keys but without foreign keys for every database in the system. It uses a cursor to iterate through each database. The query uses Common Table Expressions (CTEs) to create a list of tables, a list of tables with primary keys (PKList), and a list of tables with foreign keys (FKList). It then performs an inner join between TableList and PKList and a left join with FKList to find tables with primary keys but without foreign keys, and inserts the results into a temporary table.

The proposed query directly queries the DG_Schema, dg_tables, DG_Attributes, DG_Attributes_PK, and DG_Attributes_FK tables to find tables with primary keys but without foreign keys. The query joins these tables and filters the results where PK_ID is not null and fk_id is null. This approach is simpler and more efficient as it avoids the use of cursors and temporary tables.

Sprint2 query is more complex and involves multiple steps, including the use of cursors, dynamic SQL, and temporary tables. It is more flexible and can handle multiple databases but not effective for this scenario with only one database being queried.The proposed solution is simpler and more efficient for this single database scenario since it does not need to iterate through multiple databases.

1.3.9. For every database/schema, list tables with IDX, index name (constraint name), fields used in IDX

In Sprint 2, the Sprint 2 query focuses on retrieving information about indexes and their associated columns for specific tables in the DGDB database. The query joins system tables like sys.tables, sys.indexes, sys.index_columns, and sys.columns to extract the relevant data. Specifically, it selects the table name, index name, index type, column name, and whether the index is unique. The tables targeted by this query are hardcoded within the WHERE clause and are primarily related to database schema, tables, attributes, roles, and auditing. The query orders the results by the table name, index name, and index column ID to provide a detailed and organized view of index usage within a

specified set of tables. This approach is ideal when you need to inspect indexes and their column mappings for specific, predefined tables in the database, particularly when working with system tables or custom schemas related to database structure and management.

On the other hand, the query from Sprint 3 takes a more schema-based and relational approach by referencing user-defined tables like DG_Schema, dg_tables, DG_Attributes, and DG_Attributes_Index. This query seeks to retrieve a broader perspective by extracting schema and table information alongside index constraint details, including the constraint name (which corresponds to the index name) and an identifier for the index (IDX_ID). By joining these tables, it provides a list of tables across schemas with their associated indexes and the respective constraint names, rather than focusing on individual columns that make up the indexes. The results are ordered by the table name, making it easier to analyze index usage across schemas and tables. This query is more flexible in terms of its ability to capture a wide array of table and index information based on schema-level relationships, rather than being limited to a predefined list of tables.

Comparing these two queries, we can see that the Sprint 2 query is more granular and column-specific, focusing on the columns involved in indexes and distinguishing between unique and non-unique indexes. The the proposed query, however, provides a broader view by focusing on constraints as a whole and does not break down the specific columns used within indexes. While the Sprint 2 query is better suited for detailed analysis of indexes and their components across a specific set of tables, Sprint 3's query excels in providing a higher-level overview of index constraints within the context of schema and table relationships, making it more scalable for broader database or schema-level analysis.

1.3.10. For every database/schema, list tables with CC

The Sprint 2 query from Sprint 2 focuses on retrieving distinct combinations of schema and table information, but it only checks for the existence of check constraints indirectly. It begins by selecting the schema_id, schema_name, database_id, and table_name from the DG_Schema, dg_tables, and DG_Attributes tables, joined based on relationships such as Database_ID matching schema_ID and Table_ID linking DG_Attributes and dg_tables. A LEFT JOIN is then performed with the DG_Attributes_ConstCheck table, which is essential for determining if a check constraint exists. The query filters results to only include rows where CC_ID (the identifier of the check constraint) is not null, suggesting that a check constraint exists for that table.

This query structure is useful for capturing basic schema-to-table relationships and filtering for tables that have associated check constraints. However, it does not fully emphasize performance optimization or clarity in the relationship between the schema, table, and constraint information, which could be enhanced in later iterations.

The the proposed query from Sprint 3 builds upon the first by refining its structure to better capture the relationship between schemas, tables, and check constraints. While the overall goal of listing tables with check constraints remains the same, Sprint 3's query adopts a more focused and precise approach. It continues to use the DG_Schema, dg_tables, and DG_Attributes tables, but its approach to joining the DG_Attributes_ConstCheck table is improved. The use of LEFT JOIN ensures that all tables are included, even if they do not have check constraints, but only the tables with valid CC_ID values will be returned by the query due to the WHERE clause filtering on c.CC_ID is not null.

This adjustment enhances the clarity and accuracy of the results, ensuring that the query effectively lists only those tables with check constraints while maintaining a straightforward structure. Moreover, Sprint 3's query refines the use of distinctness (distinct) to ensure that the output contains unique combinations of schema and table information, further improving the result's quality.

Both queries aim to meet the same requirement of listing tables with check constraints across schemas and databases. However, the query from Sprint 3 shows a more refined approach in its structure and clarity. While the Sprint 2 query is functional, it is slightly less optimized in terms of performance and clarity of relationships. The refinement in Sprint 3 indicates a deeper understanding of the problem, where the query is more streamlined and focused on efficiently retrieving only the necessary results, without redundant data or complexity. Therefore, the the proposed query is the more optimal solution for the task at hand, offering improved readability and performance while still meeting the original requirements.

2. Repeat SQL implementation done in Sprint2 but replacing it with database programming

2.1. Implement the Qlist1 against the system catalog of Oracle System Catalog.

2.1.1. For every schema, list Schema, # tables, # records

Query:

```
SET SERVEROUTPUT ON;
CREATE OR REPLACE PROCEDURE GetSchemaTableDetails AS
BEGIN
	FOR schema_record IN (
	SELECT owner
	FROM all_tables
	WHERE owner IN ('BOOK', 'COMPANY', 'EDU', 'FAOES', 'IT', 'MLS', 'MLS2', 'OES2',
		'OES4', 'OLYMPIC', 'OOES', 'PROP2', 'PUBCO', 'SALES', 'SP', 'SPJ', 'ZEOTA')
	GROUP BY owner
	) LOOP
	DECLARE
	v_num_tables NUMBER;
	v_num_records NUMBER;
	BEGIN
	SELECT COUNT(table_name)
	INTO v_num_tables
	FROM all_tables
	WHERE owner = schema_record.owner;

	SELECT SUM(num_rows)
	INTO v_num_records
	FROM all_tables
	WHERE owner = schema_record.owner;


	DBMS_OUTPUT.PUT_LINE('Schema: ' || schema_record.owner ||
		' | Number of Tables: ' || v_num_tables ||
		' | Number of Records: ' || NVL(v_num_records, 0));
	END;
	END LOOP;
END;
/
```

```
EXEC GETSCHEMATABLEDETAILS;
```

Result:

Schema: OES4 | Number of Tables: 25 | Number of Records: 5070
Schema: PROP2 | Number of Tables: 8 | Number of Records: 82
Schema: BOOK | Number of Tables: 6 | Number of Records: 173
Schema: SP | Number of Tables: 3 | Number of Records: 34
Schema: PUBCO | Number of Tables: 25 | Number of Records: 467
Schema: FAOES | Number of Tables: 40 | Number of Records: 6825
Schema: MLS2 | Number of Tables: 10 | Number of Records: 22758
Schema: OOES | Number of Tables: 15 | Number of Records: 154
Schema: COMPANY | Number of Tables: 6 | Number of Records: 45
Schema: MLS | Number of Tables: 17 | Number of Records: 4110

PL/SQL procedure successfully completed.

2.1.2. For every schema, list table name, # attributes, # PK_ attr, # FK_ attr, # NN_attr, # IDX_attr, # CC_attr, # UQ_attrs, # PGM_attr"

Query:

```
CREATE OR REPLACE PROCEDURE GetTableConstraintsDetails AS
BEGIN
        FOR schema_rec IN (
        SELECT DISTINCT owner
        FROM all_tab_columns
        WHERE owner IN ('BOOK', 'COMPANY', 'EDU', 'FAOES', 'IT', 'MLS', 'MLS2', 'OES2',
                'OES4', 'OLYMPIC', 'OOES', 'PROP2', 'PUBCO', 'SALES', 'SP', 'SPJ', 'ZEOTA')
        ) LOOP
        DECLARE
        v_table_name   VARCHAR2(30);
        v_attr_id   NUMBER;
        v_attr_name             VARCHAR2(30);
        v_pk_id     VARCHAR2(30);
        v_fk_id     VARCHAR2(30);
        v_nn_id     VARCHAR2(30);
        v_constr_type  CHAR(1);
        v_constr_name  VARCHAR2(30);
        BEGIN
        FOR table_rec IN (
        SELECT t.table_name,
        t.column_id AS attribute_id,
        t.column_name AS attribute_name,
        (CASE
                WHEN pk.constraint_type IS NOT NULL THEN pk.constraint_name
                ELSE NULL
        END) AS pk_id,
        (CASE
                WHEN fk.constraint_type IS NOT NULL THEN fk.constraint_name
                ELSE NULL
        END) AS fk_id,
        (CASE
                WHEN nn.constraint_type IS NOT NULL THEN nn.constraint_name
                ELSE NULL
        END) AS nn_id,
        (CASE
                WHEN pk.constraint_type IS NOT NULL THEN 'P'
```

```
                WHEN fk.constraint_type IS NOT NULL THEN 'R'
                WHEN nn.constraint_type IS NOT NULL THEN 'C'
                ELSE NULL
        END) AS constraint_type,
        (CASE
                WHEN pk.constraint_type IS NOT NULL THEN pk.constraint_name
                WHEN fk.constraint_type IS NOT NULL THEN fk.constraint_name
                WHEN nn.constraint_type IS NOT NULL THEN nn.constraint_name
                ELSE NULL
        END) AS constraint_name
        FROM all_tab_columns t
        LEFT JOIN all_cons_columns cc_pk
        ON t.owner = cc_pk.owner
        AND t.table_name = cc_pk.table_name
        AND t.column_name = cc_pk.column_name
        LEFT JOIN all_constraints pk
        ON cc_pk.owner = pk.owner
        AND cc_pk.constraint_name = pk.constraint_name
        AND pk.constraint_type = 'P'
        LEFT JOIN all_cons_columns cc_fk
        ON t.owner = cc_fk.owner
        AND t.table_name = cc_fk.table_name
        AND t.column_name = cc_fk.column_name
        LEFT JOIN all_constraints fk
        ON cc_fk.owner = fk.owner
        AND cc_fk.constraint_name = fk.constraint_name
        AND fk.constraint_type = 'R'
        LEFT JOIN all_cons_columns cc_nn
        ON t.owner = cc_nn.owner
        AND t.table_name = cc_nn.table_name
        AND t.column_name = cc_nn.column_name
        LEFT JOIN all_constraints nn
        ON cc_nn.owner = nn.owner
        AND cc_nn.constraint_name = nn.constraint_name
        AND nn.constraint_type = 'C'
        WHERE t.owner = schema_rec.owner
        ORDER BY t.table_name, t.column_id
        ) LOOP
        DBMS_OUTPUT.PUT_LINE('Schema: ' || schema_rec.owner ||
                ' | Table: ' || table_rec.table_name ||
                ' | Attribute: ' || table_rec.attribute_name ||
                ' | PK: ' || NVL(table_rec.pk_id, 'None') ||
                ' | FK: ' || NVL(table_rec.fk_id, 'None') ||
                ' | NN: ' || NVL(table_rec.nn_id, 'None') ||
                ' | Constraint Type: ' || NVL(table_rec.constraint_type, 'None') ||
                ' | Constraint Name: ' || NVL(table_rec.constraint_name, 'None'));
        END LOOP;
        END;
        END LOOP;
END;
/
```

Result:

Procedure GETTABLECONSTRAINTSDETAILS compiled

But then had trouble actually showing the table (kept resetting my connection)

2.1.3. For every schema, list Table_ID, Table_Name, attribute_id, attribute_Name, PK_ID, FK_ID, NN_ID, constraint_type, and constraint_Name

Query:

```
CREATE OR REPLACE PROCEDURE GetColumnConstraints (
  p_owner IN VARCHAR2,
  p_table_name IN VARCHAR2
) AS
  CURSOR c_columns IS
    SELECT
      c.table_name AS Table_Name,
      c.column_id AS Attribute_ID,
      c.column_name AS Attribute_Name,
      CASE
        WHEN pk.column_name IS NOT NULL THEN 'Yes' ELSE 'No'
      END AS PK_ID,
      CASE
        WHEN fk.column_name IS NOT NULL THEN 'Yes' ELSE 'No'
      END AS FK_ID,
      CASE
        WHEN c.nullable = 'N' THEN 'Yes' ELSE 'No'
      END AS NN_ID,
      co.constraint_type,
      co.constraint_name
    FROM all_tab_columns c
    LEFT JOIN (
      SELECT acc.table_name, acc.column_name
      FROM all_constraints ac
      JOIN all_cons_columns acc ON ac.constraint_name = acc.constraint_name AND ac.owner = acc.owner
      WHERE ac.constraint_type = 'P' AND ac.owner = p_owner
    ) pk ON c.table_name = pk.table_name AND c.column_name = pk.column_name
    LEFT JOIN (
      SELECT acc.table_name, acc.column_name
      FROM all_constraints ac
      JOIN all_cons_columns acc ON ac.constraint_name = acc.constraint_name AND ac.owner = acc.owner
      WHERE ac.constraint_type = 'R' AND ac.owner = p_owner
    ) fk ON c.table_name = fk.table_name AND c.column_name = fk.column_name
    LEFT JOIN (
      SELECT acc.table_name, acc.column_name, ac.constraint_type, ac.constraint_name
      FROM all_constraints ac
      JOIN all_cons_columns acc ON ac.constraint_name = acc.constraint_name AND ac.owner = acc.owner
      WHERE ac.owner = p_owner
    ) co ON c.table_name = co.table_name AND c.column_name = co.column_name
    WHERE c.owner = p_owner AND c.table_name = p_table_name
    ORDER BY c.table_name, c.column_id;

  v_rec c_columns%ROWTYPE;
BEGIN
  OPEN c_columns;
  LOOP
    FETCH c_columns INTO v_rec;
    EXIT WHEN c_columns%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE(
      'Table: ' || v_rec.Table_Name ||
      '| Attribute ID: ' || v_rec.Attribute_ID ||
      '| Attribute Name: ' || v_rec.Attribute_Name ||
      '| PK: ' || v_rec.PK_ID ||
      '| FK: ' || v_rec.FK_ID ||
      '| NN: ' || v_rec.NN_ID ||
      '| Constraint Type: ' || v_rec.constraint_type ||
```

```
        '| Constraint Name: ' || v_rec.constraint_name
      );
   END LOOP;
   CLOSE c_columns;
END;
/
```

Result:

Table: AUTHOR| Attribute ID: 1| Attribute Name: AUTHOR_NUM| PK: Yes| FK: No| NN: Yes| Constraint Type: P| Constraint Name: SYS_C006987
Table: AUTHOR| Attribute ID: 2| Attribute Name: AUTHOR_LAST| PK: No| FK: No| NN: No| Constraint Type: | Constraint Name:
Table: AUTHOR| Attribute ID: 3| Attribute Name: AUTHOR_FIRST| PK: No| FK: No| NN: No| Constraint Type: | Constraint Name:
Table: BOOK| Attribute ID: 1| Attribute Name: BOOK_CODE| PK: Yes| FK: No| NN: Yes| Constraint Type: P| Constraint Name: SYS_C006990
Table: BOOK| Attribute ID: 2| Attribute Name: TITLE| PK: No| FK: No| NN: No| Constraint Type: | Constraint Name:
Table: BOOK| Attribute ID: 3| Attribute Name: PUBLISHER_CODE| PK: No| FK: Yes| NN: No| Constraint Type: R| Constraint Name: BOOK_PUBLISHER_CODE_FK
Table: BOOK| Attribute ID: 4| Attribute Name: TYPE| PK: No| FK: No| NN: No| Constraint Type: | Constraint Name:
Table: BOOK| Attribute ID: 5| Attribute Name: PRICE| PK: No| FK: No| NN: No| Constraint Type: | Constraint Name:
Table: BOOK| Attribute ID: 6| Attribute Name: PAPERBACK| PK: No| FK: No| NN: No| Constraint Type: | Constraint Name:
Table: BRANCH| Attribute ID: 1| Attribute Name: BRANCH_NUM| PK: Yes| FK: No| NN: Yes| Constraint Type: P| Constraint Name: SYS_C006989
Table: BRANCH| Attribute ID: 2| Attribute Name: BRANCH_NAME| PK: No| FK: No| NN: No| Constraint Type: | Constraint Name:
Table: BRANCH| Attribute ID: 3| Attribute Name: BRANCH_LOCATION| PK: No| FK: No| NN: No| Constraint Type: | Constraint Name:
Table: BRANCH| Attribute ID: 4| Attribute Name: NUM_EMPLOYEES| PK: No| FK: No| NN: No| Constraint Type: | Constraint Name:
Table: INVENTORY| Attribute ID: 1| Attribute Name: BOOK_CODE| PK: Yes| FK: Yes| NN: Yes| Constraint Type: C| Constraint Name: SYS_C006991
Table: INVENTORY| Attribute ID: 1| Attribute Name: BOOK_CODE| PK: Yes| FK: Yes| NN: Yes| Constraint Type: R| Constraint Name: INVENTORY_BOOK_CODE_FK
Table: INVENTORY| Attribute ID: 1| Attribute Name: BOOK_CODE| PK: Yes| FK: Yes| NN: Yes| Constraint Type: P| Constraint Name: SYS_C006993
Table: INVENTORY| Attribute ID: 2| Attribute Name: BRANCH_NUM| PK: Yes| FK: Yes| NN: Yes| Constraint Type: R| Constraint Name: INVENTORY_BRANCH_NUM_FK
Table: INVENTORY| Attribute ID: 2| Attribute Name: BRANCH_NUM| PK: Yes| FK: Yes| NN: Yes| Constraint Type: C| Constraint Name: SYS_C006992
Table: INVENTORY| Attribute ID: 2| Attribute Name: BRANCH_NUM| PK: Yes| FK: Yes| NN: Yes| Constraint Type: P| Constraint Name: SYS_C006993
Table: INVENTORY| Attribute ID: 3| Attribute Name: ON_HAND| PK: No| FK: No| NN: No| Constraint Type: | Constraint Name:
Table: WROTE| Attribute ID: 1| Attribute Name: BOOK_CODE| PK: Yes| FK: Yes| NN: Yes| Constraint Type: P| Constraint Name: SYS_C006995
Table: WROTE| Attribute ID: 1| Attribute Name: BOOK_CODE| PK: Yes| FK: Yes| NN: Yes| Constraint Type: R| Constraint Name: WROTE_BOOK_CODE_FK
Table: WROTE| Attribute ID: 2| Attribute Name: AUTHOR_NUM| PK: Yes| FK: Yes| NN: Yes| Constraint Type: R| Constraint Name: WROTE_AUTHOR_NUM_FK
Table: WROTE| Attribute ID: 2| Attribute Name: AUTHOR_NUM| PK: Yes| FK: Yes| NN: Yes| Constraint Type: C| Constraint Name: SYS_C006994
Table: WROTE| Attribute ID: 2| Attribute Name: AUTHOR_NUM| PK: Yes| FK: Yes| NN: Yes| Constraint Type: P| Constraint Name: SYS_C006995
Table: WROTE| Attribute ID: 3| Attribute Name: SEQUENCE| PK: No| FK: No| NN: No| Constraint Type: | Constraint Name:

2.1.4. For every schema, analyze PK, FK,and IDX

2.1.4.1. list tables without PK

Query:

```
CREATE OR REPLACE PROCEDURE ListTablesWithoutPK (
   p_owner IN VARCHAR2
) AS
   CURSOR c_tables_no_pk IS
      SELECT t.table_name
      FROM all_tables t
      LEFT JOIN all_constraints c
         ON t.table_name = c.table_name
```

```
        AND c.constraint_type = 'P'
        AND t.owner = c.owner
      WHERE t.owner = p_owner
        AND c.table_name IS NULL
      ORDER BY t.table_name;

  v_table_name VARCHAR2(128);
BEGIN
  OPEN c_tables_no_pk;
  LOOP
    FETCH c_tables_no_pk INTO v_table_name;
    EXIT WHEN c_tables_no_pk%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('Table without PK: ' || v_table_name);
  END LOOP;
  CLOSE c_tables_no_pk;
END;
/
```

Result:

```
Procedure LISTTABLESWITHOUTPK compiled
PL/SQL procedure successfully completed.
```

2.1.4.2. list tables with PK but without FK

Query:

```
CREATE OR REPLACE PROCEDURE ListTablesWithPKNoFK (
  p_owner IN VARCHAR2
) AS
  CURSOR c_tables_pk_no_fk IS
    SELECT t.table_name
    FROM user_tables t
    JOIN user_constraints pk ON t.table_name = pk.table_name AND pk.constraint_type = 'P'
    LEFT JOIN user_constraints fk ON t.table_name = fk.table_name AND fk.constraint_type = 'R'
    WHERE fk.table_name IS NULL
    ORDER BY t.table_name;

  v_table_name VARCHAR2(128);
BEGIN
  OPEN c_tables_pk_no_fk;
  LOOP
    FETCH c_tables_pk_no_fk INTO v_table_name;
    EXIT WHEN c_tables_pk_no_fk%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE('Table with PK, without FK: ' || v_table_name);
  END LOOP;
  CLOSE c_tables_pk_no_fk;
END;
/
```

Result:

```
Table with PK, without FK: DEPARTMENT

PL/SQL procedure successfully completed.
```

2.1.4.3. list tables with PK but without IDX

Query:

```
CREATE OR REPLACE PROCEDURE ListTablesWithPKNoIndex (
  p_owner IN VARCHAR2
) AS
  CURSOR c_tables_pk_no_index IS
```

```
      SELECT t.table_name
      FROM user_tables t
      JOIN user_constraints pk ON t.table_name = pk.table_name AND pk.constraint_type = 'P'
      LEFT JOIN user_indexes i ON t.table_name = i.table_name
      WHERE i.table_name IS NULL
      ORDER BY t.table_name;

   v_table_name VARCHAR2(128);
BEGIN
   OPEN c_tables_pk_no_index;
   LOOP
      FETCH c_tables_pk_no_index INTO v_table_name;
      EXIT WHEN c_tables_pk_no_index%NOTFOUND;

      DBMS_OUTPUT.PUT_LINE('Table with PK, without Index: ' || v_table_name);
   END LOOP;
   CLOSE c_tables_pk_no_index;
END;
/
```

Result:

Procedure LISTTABLESWITHPKNOINDEX compiled

PL/SQL procedure successfully completed.

2.1.5. For every database/schema

2.1.5.1. rank them according to number of NN (Not Null) have created

Query:

```
CREATE OR REPLACE PROCEDURE Get_Non_Nullable_Column_Count (p_cursor OUT SYS_REFCURSOR) AS
BEGIN
        -- Insert results into the temporary table
        INSERT INTO temp_nn_count (schema_name, nn_count)
        SELECT OWNER AS schema_name,
        COUNT(*) AS nn_count
        FROM ALL_TAB_COLUMNS
        WHERE NULLABLE = 'N'
        AND OWNER IN ('BOOK', 'IT4', 'ZEOTADB', 'COMPANY', 'OES21', 'OOES', 'PROP2', 'PUBCO', 'SP', 'SPJ2')
        GROUP BY OWNER
        ORDER BY nn_count DESC;

        -- Check if data was inserted into temp_nn_count
        DBMS_OUTPUT.PUT_LINE('Data inserted into temp_nn_count.');

        -- Open the ref cursor to output the contents of the temporary table
        OPEN p_cursor FOR
        SELECT * FROM temp_nn_count ORDER BY nn_count DESC;

        -- Optional: Clear the temporary table (to prepare for the next run)
        DELETE FROM temp_nn_count;
END Get_Non_Nullable_Column_Count;
/

SET SERVEROUTPUT ON;

DECLARE
        v_cursor SYS_REFCURSOR;
        v_schema_name VARCHAR2(50);
        v_nn_count NUMBER;
BEGIN
        -- Call the procedure and pass the cursor to fetch data
```

```
        Get_Non_Nullable_Column_Count(v_cursor);

        -- Fetch and display the results
        LOOP
        FETCH v_cursor INTO v_schema_name, v_nn_count;
        EXIT WHEN v_cursor%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE('Schema: ' || v_schema_name || ' | Non-nullable count: ' || v_nn_count);
        END LOOP;

        -- Close the cursor
        CLOSE v_cursor;
END;
/
```

Result:

```
Procedure GET_NON_NULLABLE_COLUMN_COUNT compiled
PL/SQL procedure successfully completed.
Data inserted into temp_nn_count.
Schema: ZEOTADB | Non-nullable count: 102
Schema: OES21 | Non-nullable count: 39
Schema: PUBCO | Non-nullable count: 38
Schema: IT4 | Non-nullable count: 25
Schema: OOES | Non-nullable count: 25
Schema: COMPANY | Non-nullable count: 12
Schema: PROP2 | Non-nullable count: 10
Schema: BOOK | Non-nullable count: 8
Schema: SPJ2 | Non-nullable count: 6
Schema: SP | Non-nullable count: 5
```

2.1.5.2. For every table list fields with NN

Query:

```
CREATE OR REPLACE PROCEDURE Get_Non_Nullable_Columns(p_cursor OUT SYS_REFCURSOR) AS
BEGIN
        -- Open the ref cursor and fetch the results of the query
        OPEN p_cursor FOR
        SELECT
        OWNER AS schema_name,
        TABLE_NAME AS table_name,
        COLUMN_NAME AS column_name
        FROM
        ALL_TAB_COLUMNS
        WHERE
        NULLABLE = 'N'
        AND OWNER IN ('BOOK', 'IT4', 'ZEOTADB', 'COMPANY', 'OES21', 'OOES', 'PROP2', 'PUBCO', 'SP', 'SPJ2')  -- Filter for the specific
schemas
        ORDER BY
        OWNER, TABLE_NAME, COLUMN_NAME;
END Get_Non_Nullable_Columns;
/

SET SERVEROUTPUT ON;

DECLARE
        v_cursor SYS_REFCURSOR;
        v_schema_name VARCHAR2(50);
        v_table_name VARCHAR2(50);
        v_column_name VARCHAR2(50);
BEGIN
        -- Call the procedure and pass the cursor to fetch data
        Get_Non_Nullable_Columns(v_cursor);
```

```
        -- Fetch and display the results
        LOOP
        FETCH v_cursor INTO v_schema_name, v_table_name, v_column_name;
        EXIT WHEN v_cursor%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE('Schema: ' || v_schema_name || ' | Table: ' || v_table_name || ' | Column: ' || v_column_name);
        END LOOP;

        -- Close the cursor
        CLOSE v_cursor;
END;
/
```

Result:

```
Procedure GET_NON_NULLABLE_COLUMNS compiled
PL/SQL procedure successfully completed.
Schema: BOOK | Table: AUTHOR | Column: AUTHOR_NUM
Schema: BOOK | Table: BOOK | Column: BOOK_CODE
Schema: BOOK | Table: BRANCH | Column: BRANCH_NUM
Schema: BOOK | Table: INVENTORY | Column: BOOK_CODE
Schema: BOOK | Table: INVENTORY | Column: BRANCH_NUM
Schema: BOOK | Table: PUBLISHER | Column: PUBLISHER_CODE
Schema: BOOK | Table: WROTE | Column: AUTHOR_NUM
Schema: BOOK | Table: WROTE | Column: BOOK_CODE
Schema: COMPANY | Table: DEPARTMENT | Column: DNAME
Schema: COMPANY | Table: DEPARTMENT | Column: DNUMBER
…
Schema: ZEOTADB | Table: VENDORPRICE | Column: PRODUCT_NO
Schema: ZEOTADB | Table: VENDORPRICE | Column: VENDOR_NO
Schema: ZEOTADB | Table: VENDORPRICE | Column: VPRICE
Schema: ZEOTADB | Table: VENDORPRICE | Column: VPRICE_NO
Schema: ZEOTADB | Table: VENDOR_QTYDISCOUNT | Column: QTYDISCOUNT_NO
Schema: ZEOTADB | Table: VNDOR_PROMOTION | Column: VPID
Schema: ZEOTADB | Table: WAREHOUSE | Column: WAREHOUSE_NO
```

2.1.6. For every database, list tables without PK

Query:

```
CREATE OR REPLACE PROCEDURE Get_Tables_Without_Primary_Key(p_cursor OUT SYS_REFCURSOR) AS
BEGIN
        -- Open the ref cursor and execute the query
        OPEN p_cursor FOR
        SELECT
        atc.OWNER AS schema_name,
        atc.TABLE_NAME
        FROM
        ALL_TAB_COLUMNS atc
        LEFT JOIN ALL_CONSTRAINTS ac
        ON atc.OWNER = ac.OWNER
        AND atc.TABLE_NAME = ac.TABLE_NAME
        AND ac.CONSTRAINT_TYPE = 'P'
        WHERE
        atc.OWNER IN ('BOOK', 'IT4', 'ZEOTADB', 'COMPANY', 'OES21', 'OOES', 'PROP2', 'PUBCO', 'SP', 'SPJ2')  -- Filter for specific schemas
        AND ac.CONSTRAINT_NAME IS NULL
        GROUP BY
        atc.OWNER, atc.TABLE_NAME
        ORDER BY
        atc.OWNER, atc.TABLE_NAME;
END Get_Tables_Without_Primary_Key;
/

SET SERVEROUTPUT ON;
```

```
DECLARE
        v_cursor SYS_REFCURSOR;
        v_schema_name VARCHAR2(50);
        v_table_name VARCHAR2(50);
BEGIN
        -- Call the procedure and pass the cursor to fetch data
        Get_Tables_Without_Primary_Key(v_cursor);

        -- Fetch and display the results
        LOOP
        FETCH v_cursor INTO v_schema_name, v_table_name;
        EXIT WHEN v_cursor%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE('Schema: ' || v_schema_name || ' | Table: ' || v_table_name);
        END LOOP;

        -- Close the cursor
        CLOSE v_cursor;
END;
/
```

Result:

```
Procedure GET_TABLES_WITHOUT_PRIMARY_KEY compiled
PL/SQL procedure successfully completed.
Schema: BOOK | Table: TEMP_NN_COUNT
Schema: IT4 | Table: MISSING_GEN_ED
Schema: IT4 | Table: MISSING_PREREQS
Schema: IT4 | Table: RETAKE_GEN_ED
Schema: IT4 | Table: RETAKE_PREREQS
Schema: OES21 | Table: REGION_STATE
Schema: OES21 | Table: RETURN_CATEGORY
Schema: OES21 | Table: SALARY
Schema: OES21 | Table: STATE_CAPITAL
Schema: OES21 | Table: TIMEDIMENSION
Schema: PROP2 | Table: LEASE
```

2.1.7. For every database, list tables without IDX

Query:

```
SET SERVEROUTPUT ON;
CREATE OR REPLACE PROCEDURE GetTablesWithoutIndexes AS
BEGIN
  FOR schema_record IN (
        SELECT owner
        FROM all_tables
        WHERE owner IN ('BOOK', 'COMPANY', 'EDU', 'FAOES', 'IT4', 'MLS', 'MLS2', 'OES2',
                'OES4', 'OLYMPIC', 'OOES', 'PROP2', 'PUBCO', 'SALES', 'SP', 'SPJ', 'ZEOTA')
        GROUP BY owner
  ) LOOP
        DECLARE
        v_table_name VARCHAR2(256);
        BEGIN
        FOR table_record IN (
        SELECT table_name
        FROM all_tables a
        WHERE NOT EXISTS (
        SELECT 1
        FROM all_indexes c
        WHERE a.owner = c.owner
        AND a.table_name = c.table_name
        )
```

```
            AND a.owner = schema_record.owner
            ) LOOP
            v_table_name := table_record.table_name;
            DBMS_OUTPUT.PUT_LINE('Schema: ' || schema_record.owner ||
                        ' | Table without Index: ' || v_table_name);
            END LOOP;
            END;
   END LOOP;
END;
/
```

Results:

```
Schema: OES4 | Table without Index: EMPLOYEE_COMMISSION_EVERY_YEAR
Schema: OES4 | Table without Index: ORDERS_AUDIT
Schema: OES4 | Table without Index: EMPLOYEE_AUDIT
Schema: PROP2 | Table without Index: LEASE
Schema: BOOK | Table without Index: TEMP_NN_COUNT
Schema: FAOES | Table without Index: REGION_STATE
Schema: FAOES | Table without Index: TIMEDIMENSION
Schema: FAOES | Table without Index: SALARY
Schema: FAOES | Table without Index: RETURN_CATEGORY
Schema: FAOES | Table without Index: STATE_CAPITAL
Schema: FAOES | Table without Index: USLOG$_EMP_FUZZY_SALES_MV
Schema: FAOES | Table without Index: USLOG$_EMP_FUZZY_ORDERS_MV
Schema: FAOES | Table without Index: USLOG$_EMP_FUZZY_PRODUCTS_
Schema: FAOES | Table without Index: USLOG$_EMP_FUZZY_PERFORMAN
Schema: IT4 | Table without Index: MISSING_PREREQS
Schema: IT4 | Table without Index: RETAKE_PREREQS
Schema: IT4 | Table without Index: MISSING_GEN_ED
Schema: IT4 | Table without Index: RETAKE_GEN_ED
```

2.1.8. For every database, list tables with PK but without FK

Query:

```
SET SERVEROUTPUT ON;
CREATE OR REPLACE PROCEDURE GetTablesWithPKWithoutFK AS
BEGIN
   FOR schema_record IN (
            SELECT owner
            FROM all_tables
            WHERE owner IN ('BOOK', 'COMPANY', 'EDU', 'FAOES', 'IT4', 'MLS', 'MLS2', 'OES2',
                        'OES4', 'OLYMPIC', 'OOES', 'PROP2', 'PUBCO', 'SALES', 'SP', 'SPJ', 'ZEOTA')
            GROUP BY owner
   ) LOOP
            DECLARE
            v_table_name VARCHAR2(256);
            BEGIN
            FOR table_record IN (
            SELECT table_name
            FROM all_tables a
            WHERE EXISTS (
            SELECT 1
            FROM all_constraints c
            WHERE a.owner = c.owner
            AND a.table_name = c.table_name
            AND c.constraint_type = 'P'
            )
            AND NOT EXISTS (
            SELECT 1
            FROM all_constraints c
            WHERE a.owner = c.owner
```

```
            AND a.table_name = c.table_name
            AND c.constraint_type = 'R'
            )
            AND a.owner = schema_record.owner
            ) LOOP
            v_table_name := table_record.table_name;
            DBMS_OUTPUT.PUT_LINE('Schema: ' || schema_record.owner ||
                        ' | Table with PK but without FK: ' || v_table_name);
            END LOOP;
            END;
  END LOOP;
END;
/
```

Results:

```
Schema: OES4 | Table with PK but without FK: EMPLOYEE_POSITION
Schema: OES4 | Table with PK but without FK: RETURN_CATEGORY
Schema: OES4 | Table with PK but without FK: EMPLOYEE_STATUS
Schema: OES4 | Table with PK but without FK: REGIONS
Schema: OES4 | Table with PK but without FK: PRODUCT
Schema: OES4 | Table with PK but without FK: VENDOR
Schema: OES4 | Table with PK but without FK: TAX
Schema: PROP2 | Table with PK but without FK: PRIVATE_OWNER
Schema: PROP2 | Table with PK but without FK: REGISTRATION
Schema: PROP2 | Table with PK but without FK: VIEWING
Schema: PROP2 | Table with PK but without FK: CLIENT
Schema: PROP2 | Table with PK but without FK: BRANCH
Schema: BOOK | Table with PK but without FK: PUBLISHER
Schema: BOOK | Table with PK but without FK: BRANCH
Schema: BOOK | Table with PK but without FK: AUTHOR
Schema: SP | Table with PK but without FK: S
Schema: SP | Table with PK but without FK: P
……
Schema: COMPANY | Table with PK but without FK: DEPARTMENT
Schema: MLS | Table with PK but without FK: MLS_EMPPOSITION
Schema: MLS | Table with PK but without FK: MLS_PLSTATUS
Schema: MLS | Table with PK but without FK: MLS_GAMETYPE
Schema: MLS | Table with PK but without FK: MLS_SPONSOR
Schema: MLS | Table with PK but without FK: MLS_TEAM
```

2.1.9. For every database/schema, list tables with IDX, index name (constraint name), fields used in IDX

Query:

```
CREATE OR REPLACE TYPE indexed_column_rec AS OBJECT (
  owner VARCHAR2(50),
  table_name VARCHAR2(50),
  index_name VARCHAR2(50),
  column_name VARCHAR2(50)
);
/

CREATE OR REPLACE TYPE indexed_column_table AS TABLE OF indexed_column_rec;
/

CREATE OR REPLACE FUNCTION get_indexed_columns RETURN indexed_column_table PIPELINED AS
BEGIN
  FOR rec IN (
    SELECT DISTINCT i.owner, i.table_name, i.index_name, c.column_name
    FROM all_indexes i
    JOIN all_ind_columns c ON c.index_name = i.index_name
```

```
    WHERE i.owner IN ('BOOK', 'COMPANY', 'EDU', 'FAOES',
                'IT4', 'MLS', 'MLS2', 'OES2', 'OES4',
                'OLYMPIC', 'OOES', 'PROP2', 'PUBCO',
                'SALES', 'SP', 'SPJ', 'ZEOTA')
    ORDER BY i.owner
  ) LOOP
    PIPE ROW(indexed_column_rec(rec.owner, rec.table_name, rec.index_name, rec.column_name));
  END LOOP;
  RETURN;
END get_indexed_columns;
/

CREATE OR REPLACE PROCEDURE fetch_top_indexes(p_limit IN NUMBER, p_result OUT SYS_REFCURSOR) AS
BEGIN
  OPEN p_result FOR
    SELECT * FROM (
      SELECT * FROM TABLE(get_indexed_columns())
    ) WHERE ROWNUM < p_limit;
END fetch_top_indexes;
/
```

Result:

```
PL/SQL procedure successfully completed.
PL/SQL procedure successfully completed.
PL/SQL procedure successfully completed.
PL/SQL procedure successfully completed.

Type INDEXED_COLUMN_REC compiled
Type INDEXED_COLUMN_TABLE compiled
Function GET_INDEXED_COLUMNS compiled
Procedure FETCH_TOP_INDEXES compiled
PL/SQL procedure successfully completed.
OWNER | TABLE_NAME | INDEX_NAME | COLUMN_NAME
--------------------------------------------------
BOOK | AUTHOR | SYS_C006987 | AUTHOR_NUM
BOOK | BOOK | SYS_C006990 | BOOK_CODE
BOOK | BRANCH | SYS_C006989 | BRANCH_NUM
BOOK | INVENTORY | SYS_C006993 | BOOK_CODE
BOOK | INVENTORY | SYS_C006993 | BRANCH_NUM
BOOK | PUBLISHER | SYS_C006988 | PUBLISHER_CODE
BOOK | WROTE | SYS_C006995 | AUTHOR_NUM
BOOK | WROTE | SYS_C006995 | BOOK_CODE
COMPANY | DEPARTMENT | DEPARTMENT_PK | DNUMBER
COMPANY | DEPENDENT | DEPENDENT_PK | DEPENDENT_NAME
COMPANY | DEPENDENT | DEPENDENT_PK | ESSN
COMPANY | DEPT_LOCATION | DEPARTMENT_LOCATION_PK | DLOCATION
COMPANY | DEPT_LOCATION | DEPARTMENT_LOCATION_PK | DNUMBER
COMPANY | EMPLOYEE | EMPLOYEE_PK | SSN
COMPANY | PROJECT | PROJECT_PK | PNUMBER
COMPANY | WORKS_ON | WORKS_ON_PK | ESSN
COMPANY | WORKS_ON | WORKS_ON_PK | PNO
FAOES | BACKORDER | BACKORDER_BACKORDERNO_PK | BACKORDER_NO
FAOES | BRANCH | BRANCH_BRANCHNO_PK | BRANCH_NO
```

2.1.10. For every database/schema, list tables with CC

Query:

```
BEGIN
  EXECUTE IMMEDIATE 'DROP PROCEDURE fetch_top_constrained_tables';
EXCEPTION WHEN OTHERS THEN NULL;
END;
```

```
/

BEGIN
   EXECUTE IMMEDIATE 'DROP FUNCTION get_constrained_tables';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
   EXECUTE IMMEDIATE 'DROP TYPE constrained_table_list';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
   EXECUTE IMMEDIATE 'DROP TYPE constrained_table_rec';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

CREATE OR REPLACE TYPE constrained_table_rec AS OBJECT (
   owner      VARCHAR2(50),
   table_name  VARCHAR2(50)
);
/

CREATE OR REPLACE TYPE constrained_table_list AS TABLE OF constrained_table_rec;
/

CREATE OR REPLACE FUNCTION get_constrained_tables RETURN constrained_table_list PIPELINED AS
BEGIN
   FOR rec IN (
      SELECT DISTINCT a.owner, a.table_name
      FROM all_tables a
      WHERE EXISTS (
         SELECT 1 FROM all_constraints c
         WHERE a.owner = c.owner
          AND a.table_name = c.table_name
          AND c.constraint_type = 'C'
      )
      AND a.owner IN ('BOOK', 'COMPANY', 'EDU', 'FAOES',
                  'IT4', 'MLS', 'MLS2', 'OES2', 'OES4',
                  'OLYMPIC', 'OOES', 'PROP2', 'PUBCO',
                  'SALES', 'SP', 'SPJ', 'ZEOTA')
      ORDER BY a.owner
   ) LOOP
      PIPE ROW(constrained_table_rec(rec.owner, rec.table_name));
   END LOOP;
   RETURN;
END get_constrained_tables;
/

CREATE OR REPLACE PROCEDURE fetch_top_constrained_tables(
   p_limit  IN NUMBER,
   p_result OUT SYS_REFCURSOR
) AS
BEGIN
   OPEN p_result FOR
      SELECT * FROM (
         SELECT * FROM TABLE(get_constrained_tables())
      ) WHERE ROWNUM < p_limit;
```

```
END fetch_top_constrained_tables;
/

SET SERVEROUTPUT ON;
DECLARE
   v_cursor SYS_REFCURSOR;
   v_owner VARCHAR2(50);
   v_table_name VARCHAR2(50);
BEGIN
   fetch_top_constrained_tables(10, v_cursor);

   DBMS_OUTPUT.PUT_LINE('OWNER | TABLE_NAME');
   DBMS_OUTPUT.PUT_LINE('---------------------');

   LOOP
      FETCH v_cursor INTO v_owner, v_table_name;
      EXIT WHEN v_cursor%NOTFOUND;

      DBMS_OUTPUT.PUT_LINE(v_owner || ' | ' || v_table_name);
   END LOOP;

   CLOSE v_cursor;
END;
/
```

Result:

```
PL/SQL procedure successfully completed.
PL/SQL procedure successfully completed.
PL/SQL procedure successfully completed.
PL/SQL procedure successfully completed.

Type CONSTRAINED_TABLE_REC compiled
Type CONSTRAINED_TABLE_LIST compiled
Function GET_CONSTRAINED_TABLES compiled
Procedure FETCH_TOP_CONSTRAINED_TABLES compiled
PL/SQL procedure successfully completed.
OWNER | TABLE_NAME
---------------------
BOOK | INVENTORY
BOOK | WROTE
COMPANY | DEPARTMENT
COMPANY | DEPENDENT
COMPANY | DEPT_LOCATION
COMPANY | EMPLOYEE
COMPANY | PROJECT
COMPANY | WORKS_ON
FAOES | BRANCH

PL/SQL procedure successfully completed.
PL/SQL procedure successfully completed.
PL/SQL procedure successfully completed.
PL/SQL procedure successfully completed.
Type CONSTRAINED_TABLE_REC compiled
Type CONSTRAINED_TABLE_LIST compiled
Function GET_CONSTRAINED_TABLES compiled
Procedure FETCH_TOP_CONSTRAINED_TABLES compiled
PL/SQL procedure successfully completed.
OWNER | TABLE_NAME
---------------------
BOOK | INVENTORY
```

BOOK | WROTE
COMPANY | DEPARTMENT
COMPANY | DEPENDENT
COMPANY | DEPT_LOCATION
COMPANY | EMPLOYEE
COMPANY | PROJECT
COMPANY | WORKS_ON
FAOES | BRANCH

2.2. Implement the Qlist1 against the system catalog of MSSQL System Catalog.

2.2.1. For every schema, list Schema, # tables, # records

Query:

```
CREATE PROCEDURE GetSchemaStats
AS
BEGIN
  DECLARE @DatabaseName NVARCHAR(128);
  DECLARE @SQL NVARCHAR(MAX);

  CREATE TABLE #SchemaStats (
    DatabaseName NVARCHAR(128),
    SchemaName NVARCHAR(128),
    TableCount INT,
    RecordCount BIGINT
  );

  DECLARE db_cursor CURSOR FOR
    SELECT name
    FROM sys.databases
    WHERE state_desc = 'ONLINE';

  OPEN db_cursor;
  FETCH NEXT FROM db_cursor INTO @DatabaseName;

  WHILE @@FETCH_STATUS = 0
  BEGIN
    SET @SQL = '
      USE ' + QUOTENAME(@DatabaseName) + ';
      INSERT INTO #SchemaStats (DatabaseName, SchemaName, TableCount, RecordCount)
      SELECT
        ''' + @DatabaseName + ''' AS DatabaseName,
        s.name AS SchemaName,
        COUNT(t.object_id) AS TableCount,
        SUM(p.rows) AS RecordCount
      FROM
        sys.schemas s
      LEFT JOIN
        sys.tables t ON s.schema_id = t.schema_id
      LEFT JOIN
        sys.partitions p ON t.object_id = p.object_id AND p.index_id IN (0,1)
      GROUP BY
        s.name;';

    EXEC sp_executesql @SQL;

    FETCH NEXT FROM db_cursor INTO @DatabaseName;
  END

  CLOSE db_cursor;
```

```
    DEALLOCATE db_cursor;

    SELECT
        DatabaseName,
        SchemaName,
        TableCount,
        RecordCount
    FROM #SchemaStats
    ORDER BY DatabaseName, SchemaName;

    DROP TABLE #SchemaStats;
END;

EXEC GetSchemaStats;
```

Result:

DatabaseName	SchemaName	TableCount	RecordCount
book	db_accessadmin	0	NULL
book	db_backupoperator	0	NULL
book	db_datareader	0	NULL
book	db_datawriter	0	NULL
book	db_ddladmin	0	NULL
…			
ZeotaDB	db_owner	0	NULL
ZeotaDB	db_securityadmin	0	NULL
ZeotaDB	dbo	74	14345
ZeotaDB	guest	0	NULL
ZeotaDB	INFORMATION_SCHEMA	0	NULL
ZeotaDB	sys	0	NULL

2.2.2. For every schema, list table name, # attributes, # PK_ attr, # FK_ attr, # NN_attr, # IDX_attr, # CC_attr, # UQ_attrs, # PGM_attr"

Query:

```
CREATE PROCEDURE GetSchemaTableConstraintsDetails
AS
BEGIN
    DECLARE @DatabaseName NVARCHAR(128);
    DECLARE @SQL NVARCHAR(MAX);

    CREATE TABLE #TableConstraints (
        DatabaseName NVARCHAR(128),
        SchemaName NVARCHAR(128),
        TableName NVARCHAR(128),
        AttributeCount INT,
        PrimaryKeyCount INT,
        ForeignKeyCount INT,
        NotNullCount INT,
        IndexCount INT,
        CheckConstraintCount INT,
        UniqueConstraintCount INT
    );

    DECLARE db_cursor CURSOR FOR
        SELECT name
        FROM sys.databases
        WHERE state_desc = 'ONLINE';

    OPEN db_cursor;
    FETCH NEXT FROM db_cursor INTO @DatabaseName;
```

```
  WHILE @@FETCH_STATUS = 0
  BEGIN
    SET @SQL = '
      USE ' + QUOTENAME(@DatabaseName) + ';
      INSERT INTO #TableConstraints (DatabaseName, SchemaName, TableName, AttributeCount, PrimaryKeyCount, ForeignKeyCount,
NotNullCount, IndexCount, CheckConstraintCount, UniqueConstraintCount)
      SELECT
        ''' + @DatabaseName + ''' AS DatabaseName,
        s.name AS SchemaName,
        t.name AS TableName,

        (SELECT COUNT(*)
         FROM INFORMATION_SCHEMA.COLUMNS c
         WHERE c.TABLE_SCHEMA = s.name
         AND c.TABLE_NAME = t.name) AS AttributeCount,

        (SELECT COUNT(*)
         FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS tc
         WHERE tc.TABLE_SCHEMA = s.name
         AND tc.TABLE_NAME = t.name
         AND tc.CONSTRAINT_TYPE = ''PRIMARY KEY'') AS PrimaryKeyCount,

        (SELECT COUNT(*)
         FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS tc
         WHERE tc.TABLE_SCHEMA = s.name
         AND tc.TABLE_NAME = t.name
         AND tc.CONSTRAINT_TYPE = ''FOREIGN KEY'') AS ForeignKeyCount,

        (SELECT COUNT(*)
         FROM INFORMATION_SCHEMA.COLUMNS c
         WHERE c.TABLE_SCHEMA = s.name
         AND c.TABLE_NAME = t.name
         AND c.IS_NULLABLE = ''NO'') AS NotNullCount,

        (SELECT COUNT(*)
         FROM sys.indexes i
         WHERE i.object_id = t.object_id) AS IndexCount,

        (SELECT COUNT(*)
         FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS tc
         WHERE tc.TABLE_SCHEMA = s.name
         AND tc.TABLE_NAME = t.name
         AND tc.CONSTRAINT_TYPE = ''CHECK'') AS CheckConstraintCount,

        (SELECT COUNT(*)
         FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS tc
         WHERE tc.TABLE_SCHEMA = s.name
         AND tc.TABLE_NAME = t.name
         AND tc.CONSTRAINT_TYPE = ''UNIQUE'') AS UniqueConstraintCount

      FROM sys.schemas s
      JOIN sys.tables t ON s.schema_id = t.schema_id;';

    EXEC sp_executesql @SQL;

    FETCH NEXT FROM db_cursor INTO @DatabaseName;
  END

  CLOSE db_cursor;
  DEALLOCATE db_cursor;

  SELECT * FROM #TableConstraints ORDER BY DatabaseName, SchemaName, TableName;
```

```
  DROP TABLE #TableConstraints;
END;

EXEC GetSchemaTableConstraintsDetails;
```

Results:

DatabaseName	SchemaName	TableName	AttributeCount	PrimaryKeyCount	ForeignKeyCount	NotNullCount	IndexCount	CheckConstraintCount	UniqueConstraintCount
book	dbo	AUTHOR	3	1	0	1	1	0	0
book	dbo	BOOK	6	1	0	1	1	0	0
book	dbo	BRANCH	4	1	0	1	1	0	0
book	dbo	DatabaseDictionary	10	0	0	0	1	0	0
…									
ZeotaDB	dbo	Vendor_QTYDISCOUNT	5	1	1	1	1	0	0
ZeotaDB	dbo	VENDORPRICE	5	1	2	4	1	0	0
ZeotaDB	dbo	Vndor_PROMOTION	6	1	2	1	1	0	0
ZeotaDB	dbo	Warehouse	7	1	0	1	1	0	0

2.2.3. For every schema, list Table_ID, Table_Name, attribute_id, attribute_Name, PK_ID, FK_ID, NN_ID, constraint_type, and constraint_Name

Query:

```
CREATE PROCEDURE GetColumnConstraints
  @TableName NVARCHAR(128)
AS
BEGIN
  SELECT
    t.name AS Table_Name,
    c.column_id AS Attribute_ID,
    c.name AS Attribute_Name,
    CASE WHEN pk.column_name IS NOT NULL THEN 'Yes' ELSE 'No' END AS PK_ID,
    CASE WHEN fk.column_name IS NOT NULL THEN 'Yes' ELSE 'No' END AS FK_ID,
    CASE WHEN c.is_nullable = 0 THEN 'Yes' ELSE 'No' END AS NN_ID,
    ISNULL(cons.type_desc, 'No Constraint') AS Constraint_Type,
    ISNULL(cons.name, 'None') AS Constraint_Name
  FROM sys.tables t
  JOIN sys.columns c ON t.object_id = c.object_id
  LEFT JOIN (
    SELECT ic.object_id, c.name AS column_name
    FROM sys.indexes i
    JOIN sys.index_columns ic ON i.object_id = ic.object_id AND i.index_id = ic.index_id
    JOIN sys.columns c ON ic.object_id = c.object_id AND ic.column_id = c.column_id
    WHERE i.is_primary_key = 1
  ) pk ON c.object_id = pk.object_id AND c.name = pk.column_name
  LEFT JOIN (
    SELECT fkc.parent_object_id, c.name AS column_name
    FROM sys.foreign_key_columns fkc
    JOIN sys.columns c ON fkc.parent_object_id = c.object_id AND fkc.parent_column_id = c.column_id
  ) fk ON c.object_id = fk.parent_object_id AND c.name = fk.column_name
  LEFT JOIN sys.objects cons ON c.object_id = cons.object_id AND cons.type IN ('PK', 'FK', 'UQ', 'CK')
  WHERE t.type = 'U' AND t.name = @TableName
  ORDER BY t.name, c.column_id;
END;
GO
```

Result:

DatabaseDictionary	1	SchemaName	No	No	No	No Constraint	None
DatabaseDictionary	2	TableName	No	No	No	No Constraint	None
DatabaseDictionary	3	ColumnName	No	No	No	No Constraint	None
DatabaseDictionary	4	DataType	No	No	No	No Constraint	None

DatabaseDictionary	5	Precision	No	No	No	No Constraint	None	
DatabaseDictionary	6	Scale	No	No	No	No Constraint	None	
DatabaseDictionary	7	IsNullable	No	No	No	No Constraint	None	
DatabaseDictionary	8	DataDefault		No	No	No	No Constraint	None
DatabaseDictionary	9	IsPrimaryKey		No	No	No	No Constraint	None
DatabaseDictionary	10	IsForeignKey		No	No	No	No Constraint	None

2.2.4. For every schema, analyze PK, FK,and IDX

2.2.4.1. list tables without PK

Query:

```
CREATE PROCEDURE ListTablesWithoutPK
AS
BEGIN
   SELECT t.name AS Table_Name
   FROM sys.tables t
   LEFT JOIN sys.key_constraints pk
      ON t.object_id = pk.parent_object_id AND pk.type = 'PK'
   WHERE pk.parent_object_id IS NULL
   ORDER BY t.name;
END;
GO
```

Result:

DatabaseDictionary

2.2.4.2. list tables with PK but without FK

Query:

```
CREATE PROCEDURE ListTablesWithPKNoFK
AS
BEGIN
   SELECT t.name AS Table_Name
   FROM sys.tables t
   JOIN sys.key_constraints pk ON t.object_id = pk.parent_object_id AND pk.type = 'PK'
   LEFT JOIN sys.foreign_keys fk ON t.object_id = fk.parent_object_id
   WHERE fk.parent_object_id IS NULL
   ORDER BY t.name;
END;
GO
```

Result:

AUTHOR
BOOK
BRANCH
INVENTORY
PUBLISHER
sysdiagrams
WROTE
DEPARTMENT
sysdiagrams
…

2.2.4.3. list tables with PK but without IDX

Query:

```
CREATE PROCEDURE ListTablesWithPKNoIndex
AS
BEGIN
   SELECT t.name AS Table_Name
   FROM sys.tables t
```

```
  JOIN sys.key_constraints pk ON t.object_id = pk.parent_object_id AND pk.type = 'PK'
  LEFT JOIN sys.indexes i ON t.object_id = i.object_id
  WHERE i.object_id IS NULL
  ORDER BY t.name;
END;
GO
```

Result:

No results

2.2.5. For every database/schema

2.2.5.1. rank them according to number of NN (Not Null) have created

Query:

```
CREATE PROCEDURE Get_Non_Nullable_Column_Count
AS
BEGIN
        DECLARE @sql NVARCHAR(MAX) = '';
        DECLARE @databaseName NVARCHAR(128);

        -- Cursor to iterate over the specified databases
        DECLARE db_cursor CURSOR FOR
        SELECT name
        FROM sys.databases
        WHERE name IN ('BOOK', 'COMPANY', 'EDU', 'OES21', 'OOES', 'PROP2', 'PUBCO', 'SP', 'SPJ2', 'ZEOTADB');

        OPEN db_cursor;
        FETCH NEXT FROM db_cursor INTO @databaseName;

        -- Loop through the databases and build the dynamic SQL query
        WHILE @@FETCH_STATUS = 0
        BEGIN
        SET @sql = @sql + '
        SELECT ''' + @databaseName + ''' AS database_name,
        TABLE_SCHEMA AS schema_name,
        COUNT(*) AS nn_count
        FROM [' + @databaseName + '].INFORMATION_SCHEMA.COLUMNS
        WHERE IS_NULLABLE = ''NO''
        GROUP BY TABLE_SCHEMA
        UNION ALL';

        FETCH NEXT FROM db_cursor INTO @databaseName;
        END;

        CLOSE db_cursor;
        DEALLOCATE db_cursor;

        -- Remove the last 'UNION ALL'
        SET @sql = LEFT(@sql, LEN(@sql) - LEN('UNION ALL')) + '
        ORDER BY nn_count DESC;';

        -- Execute the dynamic SQL
        EXEC sp_executesql @sql;
END;
GO

EXEC Get_Non_Nullable_Column_Count;
```

Result:

database_name	schema_name	nn_count

ZeotaDB	dbo	105
EDU	dbo	28
Company	dbo	26
Prop2	dbo	13
book	dbo	11
SP	dbo	7

2.2.5.2. For every table list fields with NN

Query:

```
CREATE PROCEDURE Get_Non_Nullable_Columns
AS
BEGIN
	DECLARE @sql NVARCHAR(MAX) = '';
	DECLARE @databaseName NVARCHAR(128);

	-- Cursor to iterate over the specified databases
	DECLARE db_cursor CURSOR FOR
	SELECT name
	FROM sys.databases
	WHERE name IN ('BOOK', 'COMPANY', 'EDU', 'OES21', 'OOES', 'PROP2', 'PUBCO', 'SP', 'SPJ2', 'ZEOTADB');

	OPEN db_cursor;
	FETCH NEXT FROM db_cursor INTO @databaseName;

	-- Loop through the databases and build the dynamic SQL query
	WHILE @@FETCH_STATUS = 0
	BEGIN
	SET @sql = @sql + '
	SELECT
	''' + @databaseName + ''' AS database_name,
	TABLE_SCHEMA AS schema_name,
	TABLE_NAME AS table_name,
	COLUMN_NAME AS column_name
	FROM [' + @databaseName + '].INFORMATION_SCHEMA.COLUMNS
	WHERE IS_NULLABLE = ''NO''
	UNION ALL ';

	FETCH NEXT FROM db_cursor INTO @databaseName;
	END;

	CLOSE db_cursor;
	DEALLOCATE db_cursor;

	-- Remove the last 'UNION ALL'
	SET @sql = LEFT(@sql, LEN(@sql) - LEN('UNION ALL'));

	-- Execute the dynamic SQL
	EXEC sp_executesql @sql;
END;
GO

EXEC Get_Non_Nullable_Columns;
```

Result:

database_name	schema_name	table_name	column_name
Company	dbo	DEPARTMENT	DNUMBER

Company	dbo	DEPARTMENT	DNAME
Company	dbo	DEPARTMENT	MGRSSN
Company	dbo	DEPARTMENT	MGRSTARTDATE
Company	dbo	EMPLOYEE	SSN
Company	dbo	EMPLOYEE	FNAME
Company	dbo	EMPLOYEE	LNAME
Company	dbo	EMPLOYEE	BDATE
Company	dbo	EMPLOYEE	ADDRESS
…	…	…	…
book	dbo	INVENTORY	BOOK_CODE
book	dbo	INVENTORY	BRANCH_NUM
book	dbo	PUBLISHER	PUBLISHER_CODE
book	dbo	WROTE	BOOK_CODE
book	dbo	WROTE	AUTHOR_NUM
book	dbo	sysdiagrams	name
book	dbo	sysdiagrams	principal_id
book	dbo	sysdiagrams	diagram_id

2.2.6. For every database, list tables without PK

Query:

```
CREATE PROCEDURE Get_Tables_Without_Primary_Key
AS
BEGIN
        DECLARE @sql NVARCHAR(MAX) = '';
        DECLARE @databaseName NVARCHAR(128);

        -- Cursor to iterate over the specified databases
        DECLARE db_cursor CURSOR FOR
        SELECT name
        FROM sys.databases
        WHERE name IN ('BOOK', 'COMPANY', 'EDU', 'OES21', 'OOES', 'PROP2', 'PUBCO', 'SP', 'SPJ2', 'ZEOTADB')
        AND state_desc = 'ONLINE'; -- Ensures the database is online

        OPEN db_cursor;
        FETCH NEXT FROM db_cursor INTO @databaseName;

        -- Loop through each database to construct the dynamic SQL
        WHILE @@FETCH_STATUS = 0
        BEGIN
        SET @sql = @sql + '
        SELECT
        ''' + @databaseName + ''' AS database_name,
        t.TABLE_SCHEMA AS schema_name,
        t.TABLE_NAME AS table_name
        FROM [' + @databaseName + '].INFORMATION_SCHEMA.TABLES t
        LEFT JOIN [' + @databaseName + '].INFORMATION_SCHEMA.TABLE_CONSTRAINTS tc
        ON t.TABLE_NAME = tc.TABLE_NAME
        AND t.TABLE_SCHEMA = tc.TABLE_SCHEMA
        AND tc.CONSTRAINT_TYPE = ''PRIMARY KEY''
        WHERE t.TABLE_TYPE = ''BASE TABLE''
        AND tc.TABLE_NAME IS NULL
        UNION ALL ';

        FETCH NEXT FROM db_cursor INTO @databaseName;
        END;
```

```
        -- Clean up cursor
        CLOSE db_cursor;
        DEALLOCATE db_cursor;

        -- Remove the last 'UNION ALL' from the dynamic SQL
        SET @sql = LEFT(@sql, LEN(@sql) - LEN('UNION ALL'));

        -- Execute the final dynamic SQL
        EXEC sp_executesql @sql;
END;
GO

EXEC Get_Tables_Without_Primary_Key;
```

Result:

database_name	schema_name	table_name
Company	dbo	DatabaseDictionary
Prop2	dbo	lease
Prop2	dbo	DatabaseDictionary
SP	dbo	DatabaseDictionary
ZeotaDB	dbo	DatabaseDictionary
book	dbo	DatabaseDictionary

2.2.7. For every database, list tables without IDX

Query:

```
CREATE PROCEDURE GetTablesWithoutIndexes
AS
BEGIN
  DECLARE @DatabaseName VARCHAR(100);
  DECLARE @SQL VARCHAR(MAX);

  DECLARE db_cursor CURSOR FOR
  SELECT name
  FROM sys.databases
  WHERE name IN ('company', 'mls2', 'book', 'oes2', 'prop2', 'sp', 'zeota');

  OPEN db_cursor;
  FETCH NEXT FROM db_cursor INTO @DatabaseName;

  WHILE @@FETCH_STATUS = 0
  BEGIN
        SET @SQL = 'USE ' + QUOTENAME(@DatabaseName) + '; ' +
        'SELECT
                '''' + @DatabaseName + '''' AS DatabaseName,
                sch.name AS SchemaName,
                tbl.name AS TableName
        FROM
                sys.tables tbl
        INNER JOIN
                sys.schemas sch ON tbl.schema_id = sch.schema_id
        LEFT JOIN
                sys.indexes idx ON tbl.object_id = idx.object_id
        WHERE
                tbl.type = ''U''
                AND idx.object_id IS NULL
        ORDER BY
                DatabaseName, sch.name, tbl.name;';
```

```
            EXEC(@SQL);
            FETCH NEXT FROM db_cursor INTO @DatabaseName;
    END;

    CLOSE db_cursor;
    DEALLOCATE db_cursor;
END;
```

Results:

```
/*
No records Retrieved
*/
```

2.2.8. For every database, list tables with PK but without FK

Query:

```
IF OBJECT_ID('ListTablesWithPKWithoutFK', 'P') IS NOT NULL
    DROP PROCEDURE ListTablesWithPKWithoutFK;
GO
CREATE PROCEDURE ListTablesWithPKWithoutFK
AS
BEGIN
    SET NOCOUNT ON;
    CREATE TABLE #Results (
            DatabaseName NVARCHAR(255),
            SchemaName NVARCHAR(255),
            TableName NVARCHAR(255)
    );

    DECLARE db_cursor CURSOR FOR
    SELECT name
    FROM sys.databases
    WHERE name IN ('company', 'mls2', 'book', 'oes2', 'prop2', 'sp', 'zeota');

    DECLARE @DatabaseName NVARCHAR(255);
    DECLARE @SQL NVARCHAR(MAX);
    OPEN db_cursor;
    FETCH NEXT FROM db_cursor INTO @DatabaseName;

    WHILE @@FETCH_STATUS = 0
    BEGIN
            SET @SQL = '
            USE ' + QUOTENAME(@DatabaseName) + ';
            INSERT INTO #Results (DatabaseName, SchemaName, TableName)
            SELECT
            ''' + @DatabaseName + ''',
            t.TABLE_SCHEMA,
            t.TABLE_NAME
            FROM
            INFORMATION_SCHEMA.TABLES t
            WHERE
            t.TABLE_TYPE = ''BASE TABLE''
            AND EXISTS (
            SELECT 1
            FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS pk
            WHERE
            pk.TABLE_SCHEMA = t.TABLE_SCHEMA
            AND pk.TABLE_NAME = t.TABLE_NAME
            AND pk.CONSTRAINT_TYPE = ''PRIMARY KEY''
            )
            AND NOT EXISTS (
            SELECT 1
```

```
            FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS fk
            WHERE
            fk.TABLE_SCHEMA = t.TABLE_SCHEMA
            AND fk.TABLE_NAME = t.TABLE_NAME
            AND fk.CONSTRAINT_TYPE = ''FOREIGN KEY''
            );';
            EXEC sp_executesql @SQL;
            FETCH NEXT FROM db_cursor INTO @DatabaseName;
  END;

  CLOSE db_cursor;
  DEALLOCATE db_cursor;

  SELECT * FROM #Results;
  DROP TABLE #Results;
  SET NOCOUNT OFF;
END;
GO
```

Results:

```
DatabaseName          SchemaName  TableName
Company   dbo   DEPARTMENT
Company   dbo   sysdiagrams
Prop2 dbo   branch
Prop2 dbo   client
Prop2 dbo   private_owner
Prop2 dbo   registration
……................................................
oes2  dbo   sysdiagrams
oes2  dbo   TAX
oes2  dbo   VENDOR
oes2  dbo   VENDORPRICE
```

2.2.9. For every database/schema, list tables with IDX, index name (constraint name), fields used in IDX

Query:

```
IF OBJECT_ID('dbo.GetConstraintColumnUsageAllDatabases', 'P') IS NOT NULL
  DROP PROCEDURE dbo.GetConstraintColumnUsageAllDatabases;
GO

CREATE PROCEDURE dbo.GetConstraintColumnUsageAllDatabases
AS
BEGIN
  SET NOCOUNT ON;

  DECLARE @DatabaseName NVARCHAR(255);
  DECLARE @SQL NVARCHAR(MAX);

  CREATE TABLE #Results (
    DatabaseName NVARCHAR(255),
    TableName NVARCHAR(255),
    ColumnName NVARCHAR(255),
    ConstraintName NVARCHAR(255)
  );

  DECLARE db_cursor CURSOR FOR
  SELECT name FROM sys.databases
  WHERE database_id > 4
  AND state_desc = 'ONLINE';
```

```
  OPEN db_cursor;
  FETCH NEXT FROM db_cursor INTO @DatabaseName;

  WHILE @@FETCH_STATUS = 0
  BEGIN
    SET @SQL = '
    USE [' + @DatabaseName + '];
    INSERT INTO #Results (DatabaseName, TableName, ColumnName, ConstraintName)
    SELECT
      ''' + @DatabaseName + ''' AS DatabaseName,
      cu.TABLE_NAME,
      cu.COLUMN_NAME,
      cu.CONSTRAINT_NAME
    FROM INFORMATION_SCHEMA.CONSTRAINT_COLUMN_USAGE cu;';

    BEGIN TRY
      EXEC sp_executesql @SQL;
    END TRY
    BEGIN CATCH
      PRINT 'Error accessing database: ' + @DatabaseName;
    END CATCH;

    FETCH NEXT FROM db_cursor INTO @DatabaseName;
  END;

  CLOSE db_cursor;
  DEALLOCATE db_cursor;

  SELECT * FROM #Results ORDER BY DatabaseName;

  DROP TABLE #Results;

  SET NOCOUNT OFF;
END;
GO

EXEC dbo.GetConstraintColumnUsageAllDatabases;
```

Result:

DatabaseName	TableName	ColumnName	ConstraintName
book	AUTHOR	AUTHOR_NUM	PK__AUTHOR__5C4B4B5EB2F4C670
book	BOOK	BOOK_CODE	PK__BOOK__D8BE9AD379326CE8
book	BRANCH	BRANCH_NUM	PK__BRANCH__14C8CD2C6A85D990
book	INVENTORY	BOOK_CODE	PK__INVENTOR__09F2160112EFCFF3
book	INVENTORY	BRANCH_NUM	PK__INVENTOR__09F2160112EFCFF3
ZeotaDB	Vndor_PROMOTION	Vendor_no	Vndor_PROMOTION_Vendor_no_FK
ZeotaDB	Vndor_PROMOTION	VPID	Vndor_PROMOTION_VPID_PK
ZeotaDB	Vendor_QTYDISCOUNT	VENDOR_NO	VQ_VENDOR_NO_FK
ZeotaDB	Warehouse	warehouse_no	warehouse_warehouse_no_PK
ZeotaDB	BRANCH	ZIP	BRANCH_ZIP_CHK

568 Rows

2.2.10. For every database/schema, list tables with CC

Query:

```
IF OBJECT_ID('dbo.ListTablesWithCheckConstraint', 'P') IS NOT NULL
  DROP PROCEDURE dbo.ListTablesWithCheckConstraint;
GO

CREATE PROCEDURE dbo.ListTablesWithCheckConstraint
AS
```

```
BEGIN
  SET NOCOUNT ON;

  DECLARE @DatabaseName NVARCHAR(255);
  DECLARE @SQL NVARCHAR(MAX);

  CREATE TABLE #Results (
    DatabaseName NVARCHAR(255),
    SchemaName NVARCHAR(255),
    TableName NVARCHAR(255),
    ConstraintName NVARCHAR(255)
  );

  DECLARE db_cursor CURSOR FOR
  SELECT name FROM sys.databases
  WHERE database_id > 4
  AND state_desc = 'ONLINE'
  AND name NOT IN ('DGDB');

  OPEN db_cursor;
  FETCH NEXT FROM db_cursor INTO @DatabaseName;

  WHILE @@FETCH_STATUS = 0
  BEGIN
    SET @SQL = '
    USE [' + @DatabaseName + '];
    INSERT INTO #Results (DatabaseName, SchemaName, TableName, ConstraintName)
    SELECT
      ''' + @DatabaseName + ''' AS DatabaseName,
      t.TABLE_SCHEMA,
      t.TABLE_NAME,
      c.CONSTRAINT_NAME
    FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS c
    JOIN INFORMATION_SCHEMA.TABLES t
      ON c.TABLE_SCHEMA = t.TABLE_SCHEMA
      AND c.TABLE_NAME = t.TABLE_NAME
    WHERE c.CONSTRAINT_TYPE = ''CHECK'';';

    BEGIN TRY
      EXEC sp_executesql @SQL;
    END TRY
    BEGIN CATCH
      PRINT 'Error accessing database: ' + @DatabaseName;
    END CATCH;

    FETCH NEXT FROM db_cursor INTO @DatabaseName;
  END;

  CLOSE db_cursor;
  DEALLOCATE db_cursor;

  SELECT * FROM #Results ORDER BY DatabaseName, SchemaName, TableName;

  DROP TABLE #Results;

  SET NOCOUNT OFF;
END;
GO

EXEC dbo.ListTablesWithCheckConstraint;
```

Result:

DatabaseName	SchemaName	TableName	ConstraintName
oes2	dbo	BRANCH	BRANCH_ZIP_CHK
ZeotaDB	dbo	BRANCH	BRANCH_ZIP_CHK

2.3. Implement the Qlist1 against the DGDB database in MSSQL.

2.3.1. For every schema, list Schema, # tables, # records

Query:

```
IF OBJECT_ID('tempdb..#Schema_Table_Stats') IS NOT NULL
   DROP TABLE #Schema_Table_Stats;

CREATE TABLE #Schema_Table_Stats (
   Schema_Name NVARCHAR(128),
   Num_Tables INT,
   Num_Records BIGINT
);

IF OBJECT_ID('dbo.GetSchemaTableStats') IS NOT NULL
   DROP PROCEDURE dbo.GetSchemaTableStats;
GO

CREATE PROCEDURE dbo.GetSchemaTableStats
AS
BEGIN
   SET NOCOUNT ON;

   TRUNCATE TABLE #Schema_Table_Stats;

   INSERT INTO #Schema_Table_Stats (Schema_Name, Num_Tables, Num_Records)
   SELECT s.name AS Schema_Name,
        COUNT(DISTINCT t.object_id) AS Num_Tables,
        SUM(p.rows) AS Num_Records
   FROM sys.schemas s
   LEFT JOIN sys.tables t ON s.schema_id = t.schema_id
   LEFT JOIN sys.partitions p ON t.object_id = p.object_id AND p.index_id IN (0,1)
   GROUP BY s.name;

END;
GO

EXEC dbo.GetSchemaTableStats;

SELECT * FROM #Schema_Table_Stats;


EXEC dbo.GetSchemaTableStats;
SELECT * FROM #Schema_Table_Stats;
```

Result:

```
Schema_Name        Num_Tables          Num_Records
db_accessadmin     0          NULL
db_backupoperator  0          NULL
db_datareader      0          NULL
db_datawriter      0          NULL
db_ddladmin        0          NULL
db_denydatareader  0          NULL
db_denydatawriter  0          NULL
db_owner 0         NULL
db_securityadmin   0          NULL
```

```
dbo         6           4
guest       0           NULL
INFORMATION_SCHEMA          0           NULL
sys         0           NULL
```

2.3.2. For every schema, list table name, # attributes, # PK_ attr, # FK_ attr, # NN_attr, # IDX_attr, # CC_attr, # UQ_attrs, # PGM_attr"

Query:

```
IF OBJECT_ID('dbo.GetSchemaTableDetails', 'P') IS NOT NULL
  DROP PROCEDURE dbo.GetSchemaTableDetails;
GO

CREATE PROCEDURE dbo.GetSchemaTableDetails
AS
BEGIN
  SET NOCOUNT ON;

  IF OBJECT_ID('tempdb..#Schema_Table_Details') IS NOT NULL
    DROP TABLE #Schema_Table_Details;

  CREATE TABLE #Schema_Table_Details (
    Schema_Name NVARCHAR(128),
    Table_Name NVARCHAR(128),
    NumberOfAttributes INT,
    NoOfPKs INT,
    NoOfFKs INT,
    NoOfCCs INT,
    NoOfIDXS INT,
    NoOfNNs INT,
    NoOfPGMs INT,
    NoOfUQIS INT
  );

  INSERT INTO #Schema_Table_Details (Schema_Name, Table_Name, NumberOfAttributes, NoOfPKs, NoOfFKs, NoOfCCs, NoOfIDXS,
NoOfNNs, NoOfPGMs, NoOfUQIS)
  SELECT
    s.name AS Schema_Name,
    t.name AS Table_Name,
    COUNT(c.column_id) AS NumberOfAttributes,
    COUNT(DISTINCT pk.index_id) AS NoOfPKs,
    COUNT(DISTINCT fk.referenced_object_id) AS NoOfFKs,
    COUNT(DISTINCT cc.object_id) AS NoOfCCs,
    COUNT(DISTINCT i.index_id) AS NoOfIDXS,
    COUNT(DISTINCT nn.column_id) AS NoOfNNs,
    COUNT(DISTINCT pg.object_id) AS NoOfPGMs,
    COUNT(DISTINCT uq.index_id) AS NoOfUQIS
  FROM
    sys.schemas s
  JOIN
    sys.tables t ON t.schema_id = s.schema_id
  LEFT JOIN
    sys.columns c ON c.object_id = t.object_id
  LEFT JOIN
    sys.indexes pk ON pk.object_id = t.object_id AND pk.is_primary_key = 1
  LEFT JOIN
    sys.foreign_keys fk ON fk.parent_object_id = t.object_id
  LEFT JOIN
    sys.check_constraints cc ON cc.parent_object_id = t.object_id
  LEFT JOIN
    sys.indexes i ON i.object_id = t.object_id AND i.is_unique = 0 AND i.is_primary_key = 0
```

```
  LEFT JOIN
    sys.columns nn ON nn.object_id = t.object_id AND nn.is_nullable = 0
  LEFT JOIN
    sys.objects pg ON pg.parent_object_id = t.object_id AND pg.type IN ('P', 'FN')  -- Procedures & Functions
  LEFT JOIN
    sys.indexes uq ON uq.object_id = t.object_id AND uq.is_unique = 1
  GROUP BY
    s.name, t.name;

  SELECT * FROM #Schema_Table_Details;
END;
GO

EXEC dbo.GetSchemaTableDetails;
```

Result:

Schema_Name	Table_Name	NumberOfAttributes	NoOfPKs	NoOfFKs	NoOfCCs	NoOfIDXS	NoOfNNs	NoOfPGMs	NoOfUQIS
dbo	MSreplication_options	36	0	0	0	1	6	0	0
dbo	spt_fallback_db	56	0	0	0	1	7	0	0
dbo	spt_fallback_dev	80	0	0	0	1	8	0	0
dbo	spt_fallback_usg	72	0	0	0	1	8	0	0
dbo	spt_monitor	121	0	0	0	1	11	0	0
dbo	sysdiagrams	15	1	0	0	0	3	0	1

2.3.3. For every schema, list Table_ID, Table_Name, attribute_id, attribute_Name, PK_ID, FK_ID, NN_ID, constraint_type, and constraint_Name

Query:

```
CREATE PROCEDURE GetColumnConstraints
  @TableName NVARCHAR(128)
AS
BEGIN
  SELECT
    t.name AS Table_Name,
    c.column_id AS Attribute_ID,
    c.name AS Attribute_Name,
    CASE WHEN pk.column_name IS NOT NULL THEN 'Yes' ELSE 'No' END AS PK_ID,
    CASE WHEN fk.column_name IS NOT NULL THEN 'Yes' ELSE 'No' END AS FK_ID,
    CASE WHEN c.is_nullable = 0 THEN 'Yes' ELSE 'No' END AS NN_ID,
    ISNULL(cons.type_desc, 'No Constraint') AS Constraint_Type,
    ISNULL(cons.name, 'None') AS Constraint_Name
  FROM sys.tables t
  JOIN sys.columns c ON t.object_id = c.object_id
  LEFT JOIN (
    SELECT ic.object_id, c.name AS column_name
    FROM sys.indexes i
    JOIN sys.index_columns ic ON i.object_id = ic.object_id AND i.index_id = ic.index_id
    JOIN sys.columns c ON ic.object_id = c.object_id AND ic.column_id = c.column_id
    WHERE i.is_primary_key = 1
  ) pk ON c.object_id = pk.object_id AND c.name = pk.column_name
  LEFT JOIN (
    SELECT fkc.parent_object_id, c.name AS column_name
    FROM sys.foreign_key_columns fkc
    JOIN sys.columns c ON fkc.parent_object_id = c.object_id AND fkc.parent_column_id = c.column_id
  ) fk ON c.object_id = fk.parent_object_id AND c.name = fk.column_name
  LEFT JOIN sys.objects cons ON c.object_id = cons.object_id AND cons.type IN ('PK', 'FK', 'UQ', 'CK')
  WHERE t.type = 'U' AND t.name = @TableName
  ORDER BY t.name, c.column_id;
END;
GO
```

Result:

DatabaseDictionary	1	SchemaName	No	No	No	No Constraint	None
DatabaseDictionary	2	TableName	No	No	No	No Constraint	None
DatabaseDictionary	3	ColumnName	No	No	No	No Constraint	None
DatabaseDictionary	4	DataType	No	No	No	No Constraint	None
DatabaseDictionary	5	Precision	No	No	No	No Constraint	None
DatabaseDictionary	6	Scale	No	No	No	No Constraint	None
DatabaseDictionary	7	IsNullable	No	No	No	No Constraint	None
DatabaseDictionary	8	DataDefault	No	No	No	No Constraint	None
DatabaseDictionary	9	IsPrimaryKey	No	No	No	No Constraint	None
DatabaseDictionary	10	IsForeignKey	No	No	No	No Constraint	None

2.3.4. For every schema, analyze PK, FK,and IDX

2.3.4.1. list tables without PK

Query:

```
CREATE PROCEDURE ListTablesWithoutPK
AS
BEGIN
  SELECT t.name AS Table_Name
  FROM sys.tables t
  LEFT JOIN sys.key_constraints pk
    ON t.object_id = pk.parent_object_id AND pk.type = 'PK'
  WHERE pk.parent_object_id IS NULL
  ORDER BY t.name;
END;
GO
```

Result:

DatabaseDictionary

2.3.4.2. list tables with PK but without FK

Query:

```
CREATE PROCEDURE ListTablesWithPKNoFK
AS
BEGIN
  SELECT t.name AS Table_Name
  FROM sys.tables t
  JOIN sys.key_constraints pk ON t.object_id = pk.parent_object_id AND pk.type = 'PK'
  LEFT JOIN sys.foreign_keys fk ON t.object_id = fk.parent_object_id
  WHERE fk.parent_object_id IS NULL
  ORDER BY t.name;
END;
GO
```

Result:

DG_DBMS
DG_Module
DG_Role
DG_Schema
DG_Users
DG_Vendor
DG_Views
sysdiagrams

2.3.4.3. list tables with PK but without IDX

Query:

```
CREATE PROCEDURE ListTablesWithPKNoIndex
AS
BEGIN
   SELECT t.name AS Table_Name
   FROM sys.tables t
   JOIN sys.key_constraints pk ON t.object_id = pk.parent_object_id AND pk.type = 'PK'
   LEFT JOIN sys.indexes i ON t.object_id = i.object_id
   WHERE i.object_id IS NULL
   ORDER BY t.name;
END;
GO
```

Result:

No results

2.3.5. For every database/schema

2.3.5.1. rank them according to number of NN (Not Null) have created

Query:

```
CREATE PROCEDURE Get_Non_Null_Attributes_Count
AS
BEGIN
        SET NOCOUNT ON;

        SELECT
        ds.schema_name,
        COUNT(dan.Constraint_Type) AS [# of NN]
        FROM DGDB.dbo.DG_Attributes da
        LEFT JOIN DGDB.dbo.DG_Attributes_NotNull dan
        ON da.Attributes_ID = dan.Attributes_ID
        LEFT JOIN DGDB.dbo.DG_Tables dt
        ON da.Table_id = dt.Table_ID
        LEFT JOIN DGDB.dbo.DG_Schema ds
        ON dt.Database_id = ds.schema_id
        WHERE dan.Constraint_Type = 'NN'
        GROUP BY ds.schema_name
        ORDER BY COUNT(dan.Constraint_Type) DESC;
END;
GO
```

Result:

schema_name	# of NN
Zeota	166
FURN	24
Camp	23
SPJ2	20
Book	15
GAMES	10

2.3.5.2. For every table list fields with NN

Query:

```
USE DGDB;
GO

CREATE PROCEDURE Get_Non_Null_Attributes
AS
```

```
BEGIN
        SET NOCOUNT ON;

        SELECT
        ds.schema_name,
        da.Table_Name,
        da.Attributes_Name
        FROM DGDB.dbo.DG_Attributes da
        LEFT JOIN DGDB.dbo.DG_Attributes_NotNull dan
        ON da.Attributes_ID = dan.Attributes_ID
        LEFT JOIN DGDB.dbo.DG_Tables dt
        ON da.Table_id = dt.Table_ID
        LEFT JOIN DGDB.dbo.DG_Schema ds
        ON dt.Database_id = ds.schema_id
        WHERE
        dan.Constraint_Type = 'NN'
        AND ds.schema_name IN ('SPJ2', 'ZEOTA', 'BOOK');
END;
GO
EXEC Get_Non_Null_Attributes;
```

Result:

schema_name	Table_Name	Attributes_Name
SPJ2	W	W#
SPJ2	W	CITY
SPJ2	S	S#
SPJ2	S	SNAME
SPJ2	P	P#
…		
Zeota	Emp_Status	Emp_NO
Zeota	Emp_Status	Status_NO
Zeota	EMPLOYEE	EMPLOYEE_NO
Zeota	EMPLOYEE	BRANCH_NO
Zeota	Employee_Position	Emp_POS_NO

2.3.6. For every database, list tables without PK

Query:

```
CREATE PROCEDURE Get_Non_Null_Attributes
AS
BEGIN
        -- Select the non-nullable attributes from the joined tables
        SELECT
        ds.schema_name,
        da.Table_Name,
        da.Attributes_Name
        FROM DGDB.dbo.DG_Attributes da
        LEFT JOIN DGDB.dbo.DG_Attributes_NotNull dan
        ON da.Attributes_ID = dan.Attributes_ID
        LEFT JOIN DGDB.dbo.DG_Tables dt
        ON da.Table_id = dt.Table_ID
        LEFT JOIN DGDB.dbo.DG_Schema ds
        ON dt.Database_id = ds.schema_id
        WHERE
        dan.Constraint_Type = 'NN'
        AND ds.schema_name IN ('SPJ2', 'ZEOTA', 'BOOK');
END;
GO

EXEC Get_Non_Null_Attributes;
```

Result:

schema_name	Table_Name	Attributes_Name
SPJ2	W	W#
SPJ2	W	CITY
SPJ2	S	S#
SPJ2	S	SNAME
SPJ2	P	P#
SPJ2	P	PNAME
SPJ2	P	WEIGHT
SPJ2	P	UNIT_COST
SPJ2	J	J#
SPJ2	J	JNAME
…	…	…
Zeota	EMP_payroll	amount
Zeota	EMP_payroll	payroll_NO
Zeota	EMP_payroll	Emp_NO
Zeota	Emp_salary	Emp_POS_NO
Zeota	EMP_Skill	Emp_NO
Zeota	EMP_Skill	Skill_NO
Zeota	Emp_Status	Emp_NO
Zeota	Emp_Status	Status_NO
Zeota	EMPLOYEE	EMPLOYEE_NO
Zeota	EMPLOYEE	BRANCH_NO
Zeota	Employee_Position	Emp_POS_NO

2.3.7. For every database, list tables without IDX

Query:

```
IF OBJECT_ID('ListTablesWithPKWithoutFK_DGDB', 'P') IS NOT NULL
   DROP PROCEDURE ListTablesWithPKWithoutFK_DGDB;
GO
CREATE PROCEDURE ListTablesWithPKWithoutFK_DGDB
AS
BEGIN
   SET NOCOUNT ON;
   CREATE TABLE #Results (
            DatabaseName NVARCHAR(255),
            SchemaName NVARCHAR(255),
            TableName NVARCHAR(255)
   );

   DECLARE db_cursor CURSOR FOR
   SELECT name
   FROM sys.databases
   WHERE name IN ('DGDB');

   DECLARE @DatabaseName NVARCHAR(255);
   DECLARE @SQL NVARCHAR(MAX);
   OPEN db_cursor;
   FETCH NEXT FROM db_cursor INTO @DatabaseName;
```

```
  WHILE @@FETCH_STATUS = 0
  BEGIN
            SET @SQL = '
            USE ' + QUOTENAME(@DatabaseName) + ';
            INSERT INTO #Results (DatabaseName, SchemaName, TableName)
            SELECT
            ''' + @DatabaseName + ''',
            t.TABLE_SCHEMA,
            t.TABLE_NAME
            FROM
        INFORMATION_SCHEMA.TABLES t
            WHERE
            t.TABLE_TYPE = ''BASE TABLE''
            AND EXISTS (
            SELECT 1
            FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS pk
            WHERE
            pk.TABLE_SCHEMA = t.TABLE_SCHEMA
            AND pk.TABLE_NAME = t.TABLE_NAME
            AND pk.CONSTRAINT_TYPE = ''PRIMARY KEY''
            )
            AND NOT EXISTS (
            SELECT 1
            FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS fk
            WHERE
            fk.TABLE_SCHEMA = t.TABLE_SCHEMA
            AND fk.TABLE_NAME = t.TABLE_NAME
            AND fk.CONSTRAINT_TYPE = ''FOREIGN KEY''
            );';
            EXEC sp_executesql @SQL;
            FETCH NEXT FROM db_cursor INTO @DatabaseName;
  END;

  CLOSE db_cursor;
  DEALLOCATE db_cursor;

  SELECT * FROM #Results;
  DROP TABLE #Results;
  SET NOCOUNT OFF;
END;
GO
```

Results:

DatabaseName	SchemaName	TableName
DGDB	dbo	DG_DBMS
DGDB	dbo	DG_Module
DGDB	dbo	DG_Role
DGDB	dbo	DG_Schema
DGDB	dbo	DG_Users
DGDB	dbo	DG_Vendor
DGDB	dbo	DG_Views
DGDB	dbo	sysdiagrams

2.3.8. For every database, list tables with PK but without FK

Query:

```
IF OBJECT_ID('ListTablesWithPKWithoutFK_DGDB', 'P') IS NOT NULL
  DROP PROCEDURE ListTablesWithPKWithoutFK_DGDB;
GO
CREATE PROCEDURE ListTablesWithPKWithoutFK_DGDB
```

```
AS
BEGIN
   SET NOCOUNT ON;
   CREATE TABLE #Results (
            DatabaseName NVARCHAR(255),
            SchemaName NVARCHAR(255),
            TableName NVARCHAR(255)
   );

   DECLARE db_cursor CURSOR FOR
   SELECT name
   FROM sys.databases
   WHERE name IN ('DGDB');

   DECLARE @DatabaseName NVARCHAR(255);
   DECLARE @SQL NVARCHAR(MAX);
   OPEN db_cursor;
   FETCH NEXT FROM db_cursor INTO @DatabaseName;

   WHILE @@FETCH_STATUS = 0
   BEGIN
            SET @SQL = '
            USE ' + QUOTENAME(@DatabaseName) + ';
            INSERT INTO #Results (DatabaseName, SchemaName, TableName)
            SELECT
            ''' + @DatabaseName + ''',
            t.TABLE_SCHEMA,
            t.TABLE_NAME
            FROM
            INFORMATION_SCHEMA.TABLES t
            WHERE
            t.TABLE_TYPE = ''BASE TABLE''
            AND EXISTS (
            SELECT 1
            FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS pk
            WHERE
            pk.TABLE_SCHEMA = t.TABLE_SCHEMA
            AND pk.TABLE_NAME = t.TABLE_NAME
            AND pk.CONSTRAINT_TYPE = ''PRIMARY KEY''
            )
            AND NOT EXISTS (
            SELECT 1
            FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS fk
            WHERE
            fk.TABLE_SCHEMA = t.TABLE_SCHEMA
            AND fk.TABLE_NAME = t.TABLE_NAME
            AND fk.CONSTRAINT_TYPE = ''FOREIGN KEY''
            );';
            EXEC sp_executesql @SQL;
            FETCH NEXT FROM db_cursor INTO @DatabaseName;
   END;

   CLOSE db_cursor;
   DEALLOCATE db_cursor;

   SELECT * FROM #Results;
   DROP TABLE #Results;
   SET NOCOUNT OFF;
END;
GO

EXEC ListTablesWithPKWithoutFK_DGDB;
```

Results:

DatabaseName	SchemaName	TableName
DGDB	dbo	DG_DBMS
DGDB	dbo	DG_Module
DGDB	dbo	DG_Role
DGDB	dbo	DG_Schema
DGDB	dbo	DG_Users
DGDB	dbo	DG_Vendor
DGDB	dbo	DG_Views
DGDB	dbo	sysdiagrams

2.3.9. For every database/schema, list tables with IDX, index name (constraint name), fields used in IDX

Query:

```
USE DGDB;
GO

IF OBJECT_ID('dbo.GetSchemaTableInfo', 'IF') IS NOT NULL
   DROP FUNCTION dbo.GetSchemaTableInfo;
GO

CREATE FUNCTION dbo.GetSchemaTableInfo()
RETURNS TABLE
AS
RETURN
(
   SELECT
      s.schema_ID,
      s.schema_name,
      t.Table_ID,
      t.table_name
   FROM dbo.DG_Schema s
   JOIN dbo.dg_tables t ON t.Database_ID = s.schema_ID
);
GO

IF OBJECT_ID('dbo.GetIndexedAttributes', 'P') IS NOT NULL
   DROP PROCEDURE dbo.GetIndexedAttributes;
GO

CREATE PROCEDURE dbo.GetIndexedAttributes
AS
BEGIN
   SET NOCOUNT ON;

   SELECT
      schemaInfo.schema_name,
      schemaInfo.table_name,
      i.Constraint_name,
      i.IDX_ID
   FROM dbo.GetSchemaTableInfo() AS schemaInfo
   JOIN dbo.DG_Attributes a ON a.Table_id = schemaInfo.Table_ID
   JOIN dbo.DG_Attributes_Index i ON i.Attributes_ID = a.Attributes_ID
   ORDER BY schemaInfo.table_name;
END;
GO

EXEC dbo.GetIndexedAttributes;
```

```
GO
```

Result:

schema_name	table_name	Constraint_name	IDX_ID
Camp	ADDRESS	ADDRESS_ADDID_IDX	DGC_IDX_ID_246
Camp	ADDRESS	ADDRESS_CITY_IDX	DGC_IDX_ID_247
Camp	ADDRESS	ADDRESS_STATE_IDX	DGC_IDX_ID_248
Camp	ADDRESS	ADDRESS_ZIP_IDX	DGC_IDX_ID_249
Camp	CUSTOMER	CUSTOMER_CFNAME_IDX	DGC_IDX_ID_262
Prop	viewing	Viewing_Property_NO_IDX	DGC_IDX_ID_145
Zeota	Vndor_PROMOTION	Vndor_PROMOTION_VPID_IDX	DGC_IDX_ID_85
Zeota	Vndor_PROMOTION	Vndor_PROMOTION_Vendor_no_IDX	DGC_IDX_ID_132
Zeota	Vndor_PROMOTION	Vndor_PROMOTION_promo_no_IDX	DGC_IDX_ID_131
Zeota	Warehouse	Warehouse_warehouse_no_IDX	DGC_IDX_ID_86

213 rows

2.3.10. For every database/schema, list tables with CC

Query:

```
USE DGDB;
GO

IF OBJECT_ID('dbo.GetTablesWithConstraints', 'P') IS NOT NULL
   DROP PROCEDURE dbo.GetTablesWithConstraints;
GO

CREATE PROCEDURE dbo.GetTablesWithConstraints
AS
BEGIN
   SET NOCOUNT ON;

   SELECT DISTINCT
      s.schema_ID,
      s.schema_name,
      t.database_ID,
      t.table_name
   FROM dbo.DG_Schema s
   JOIN dbo.dg_tables t ON t.Database_ID = s.schema_ID
   JOIN dbo.DG_Attributes a ON a.Table_id = t.Table_ID
   LEFT JOIN dbo.DG_Attributes_ConstCheck c ON c.Attributes_ID = a.Attributes_ID;
END;
GO

EXEC dbo.GetTablesWithConstraints;
GO
```

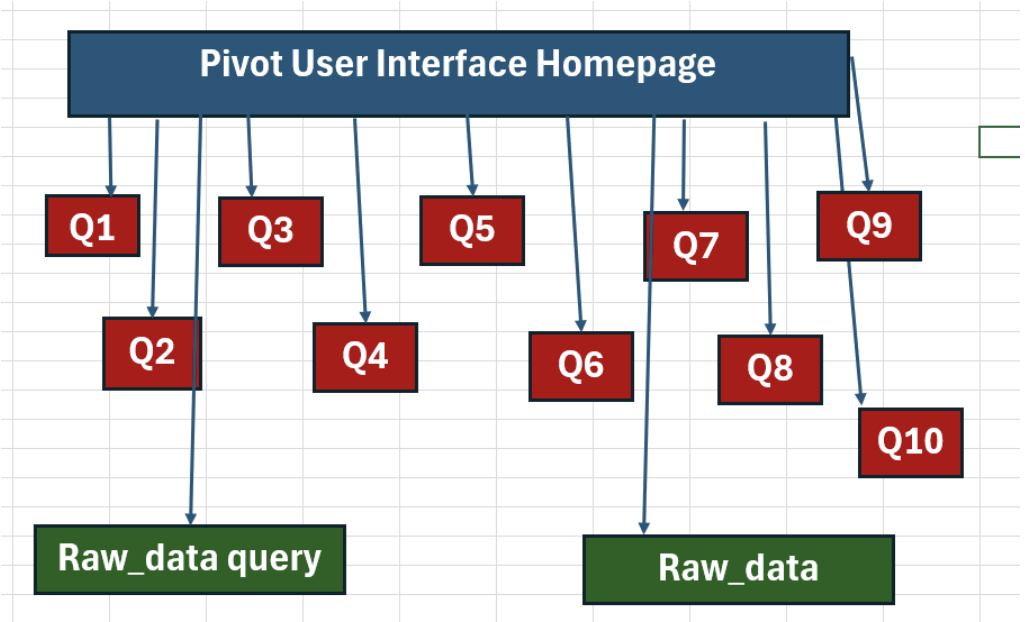
Result:

schema_ID	schema_name	database_ID	table_name
1	Zeota	1	backorder
1	Zeota	1	BRANCH
1	Zeota	1	Branch_Size
1	Zeota	1	Certificate
1	Zeota	1	Company
10	SPJ2	10	P
10	SPJ2	10	S
10	SPJ2	10	SPJ
10	SPJ2	10	SPW
10	SPJ2	10	W

135 Rows

3. Expand Pivot implementation in Sprint2 by including Pivot interface

3.1. Develop a design for the Pivot User Interface (PUI)



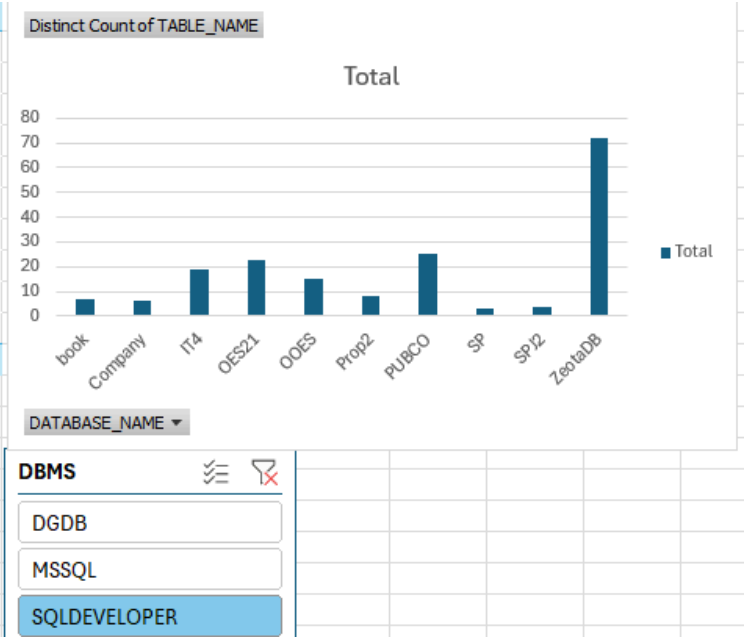
3.2. Implement PUI design using Excel Pivot worksheet

3.3. Modify the pivot worksheet developed in Sprint2 to match he proposed solution for Qlist1

3.3.1. For every schema, list Schema, # tables, # records

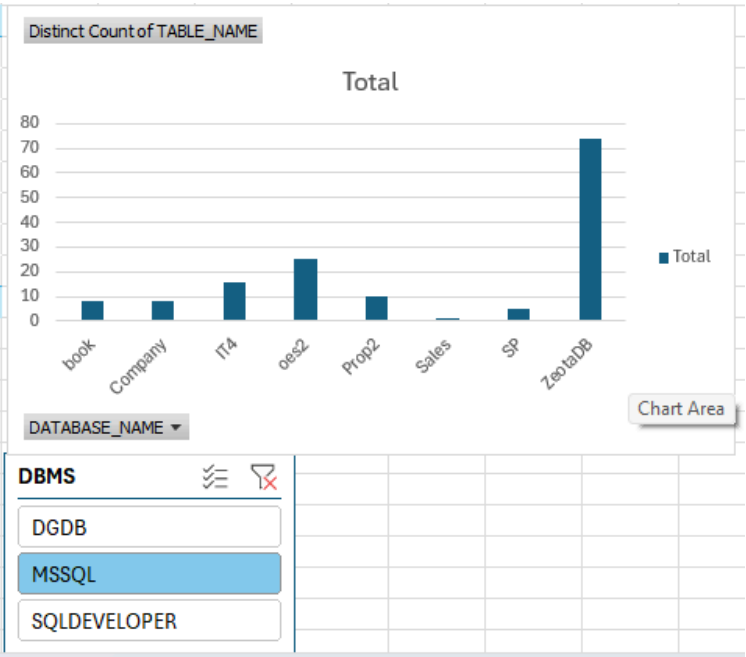
3.3.1.1. Oracle

Row Labels	Distinct Count of TABLE_NAME
book	7
Company	6
IT4	19
OES21	23
OOES	15
Prop2	8
PUBCO	25
SP	3
SPJ2	4
ZeotaDB	72
Grand Total	**138**



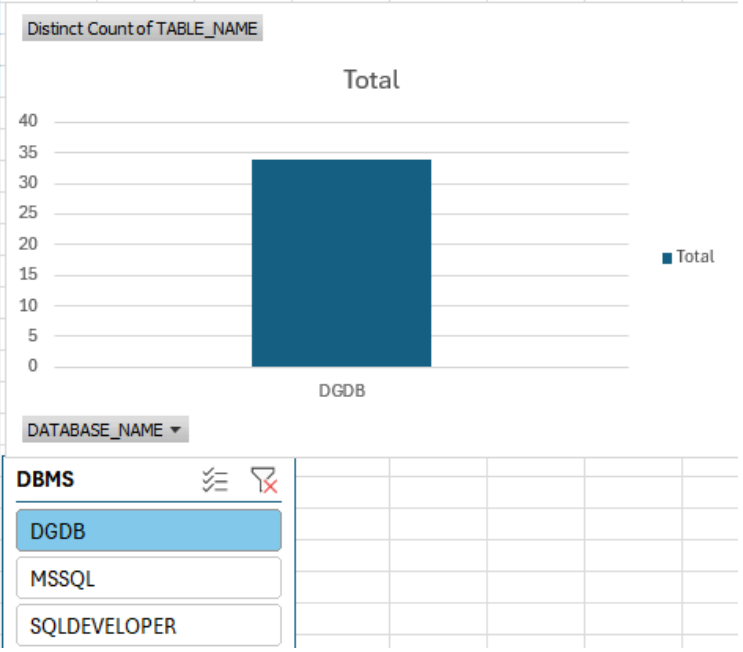
3.3.1.2. MSSQL

Row Labels	Distinct Count of TABLE_NAME
book	8
Company	8
IT4	16
oes2	25
Prop2	10
Sales	1
SP	5
ZeotaDB	74
Grand Total	**109**



3.3.1.3. DGDB

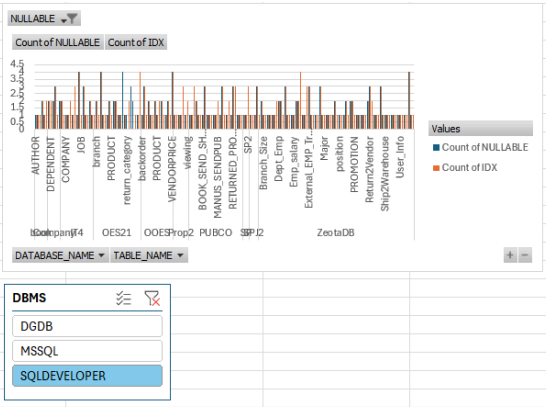
Row Labels	Distinct Count of TABLE_NAME
DGDB	34
Grand Total	**34**



3.3.2. For every schema, list table name, # attributes, # PK_ attr, # FK_ attr, # NN_attr, # IDX_attr, # CC_attr, # UQ_attrs, # PGM_attr

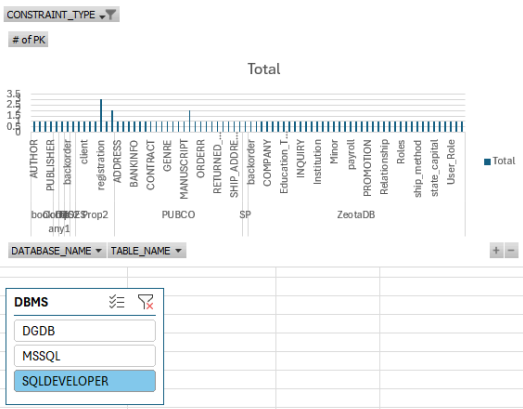
3.3.2.1. Oracle

Row Labels	Count of NULLABLE	Count of IDX
book	**8**	**8**
AUTHOR	1	1
BOOK	1	1
branch	1	1
INVENTORY	2	2
PUBLISHER	1	1
WROTE	2	2
Company	**12**	**12**
DEPARTMENT	2	2
DEPENDENT	2	2
DEPT_LOCATION	2	2
EMPLOYEE	3	3
PROJECT	1	1
WORKS_ON	2	2
IT4	**25**	**25**
CERT	2	2
CERTIFICATION	1	1
COMPANY	1	1
COURSE	1	1
COURSE_PROGRAM	2	2
DEPARTMENT	1	1
EMPLOYED	3	3
ENROLLMENT	1	1
INTERNSHIP	4	4
JOB	1	1

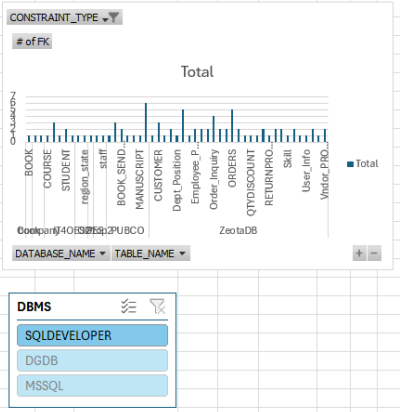


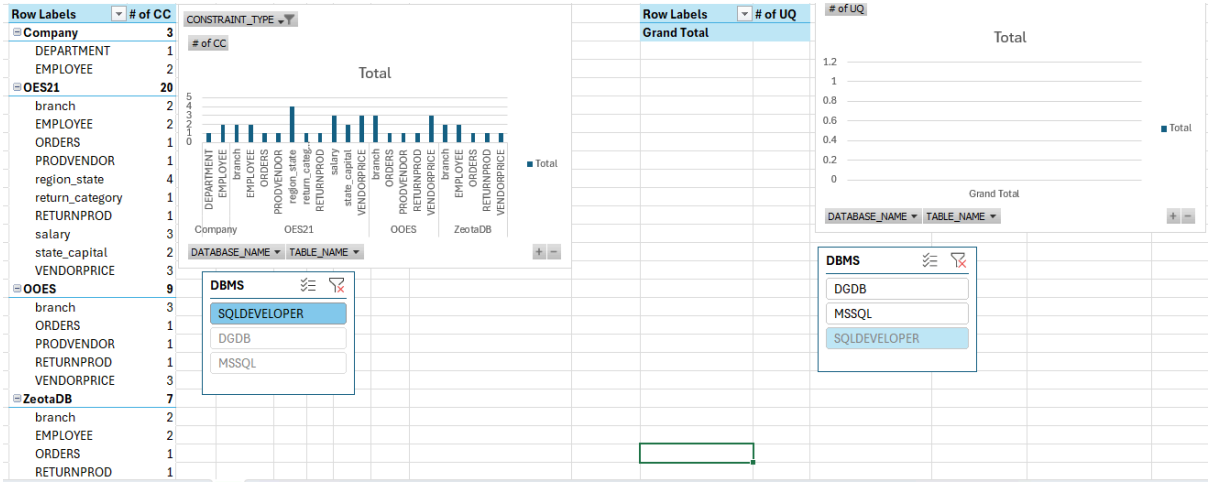
Row Labels	# of PK
book	**4**
AUTHOR	1
BOOK	1
branch	1
PUBLISHER	1
Company	**1**
DEPENDENT	1
IT4	**1**
prerequisite	1
OES21	**1**
backorder	1
OOES	**1**
backorder	1
Prop2	**10**
branch	1
client	1
private_owner	1
property_for_rent	1
registration	3
staff	1
viewing	2
PUBCO	**24**
ADDRESS	1
AUTHOR	1
AUTHOR_ADDRESS	1

# of PK
4
1
1
1
1
1
1
1
1
1
1
1
10
1
1
1
1
3
1
2
24
1
1
1

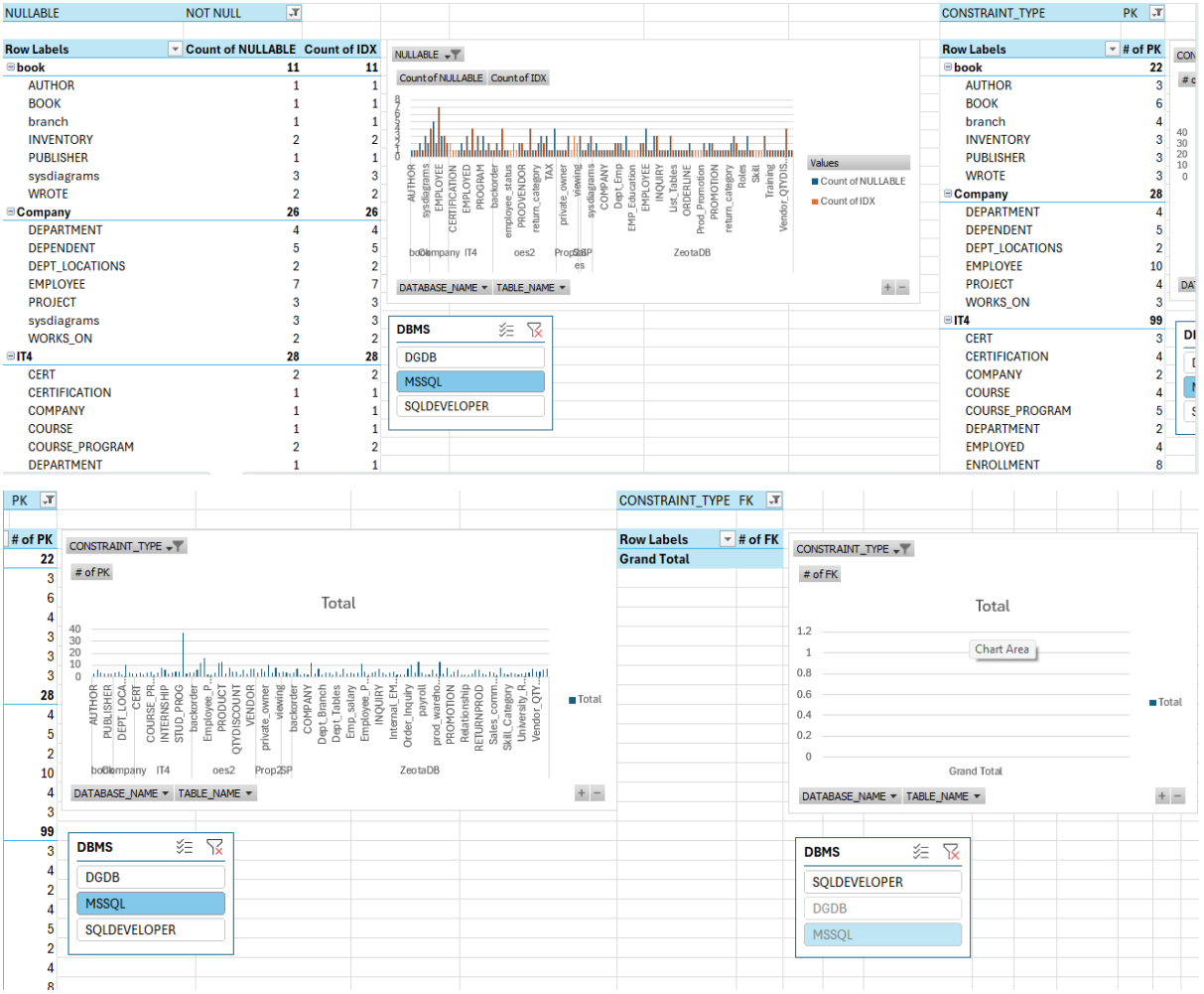


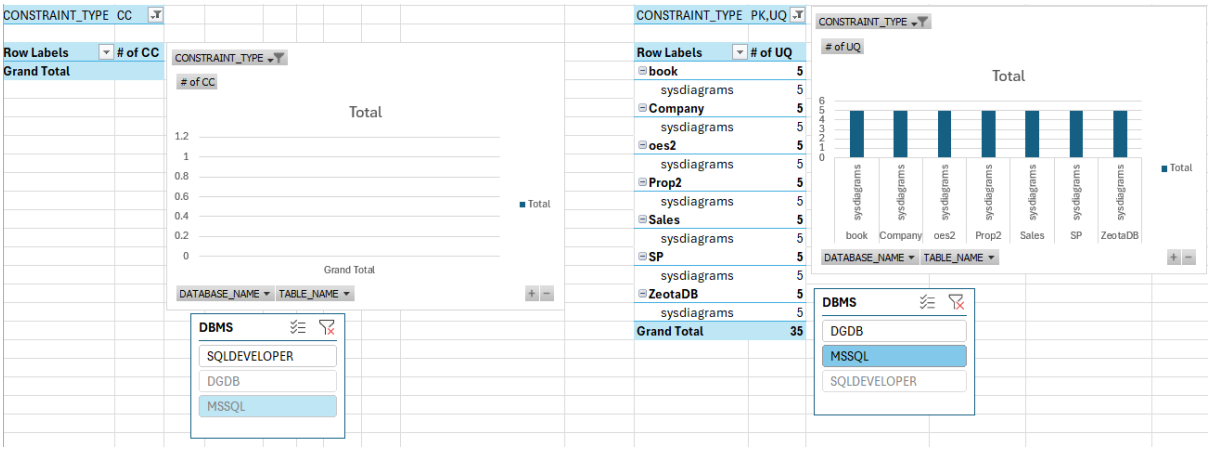
Row Labels	# of FK
book	**1**
BOOK	1
Company	**2**
EMPLOYEE	1
PROJECT	1
IT4	**8**
COURSE	1
ENROLLMENT	3
PROGRAM	1
STUDENT	2
TEACHER	1
OES21	**2**
CUSTOMER	1
region_state	1
OOES	**1**
O_CUSTOMER	1
Prop2	**2**
property_for_rent	1
staff	1
PUBCO	**15**
AUTHOR	1
BOOK	3
BOOK_SEND_SHOP	2
CONTRACT	1
MANUS_SENDPUB	1



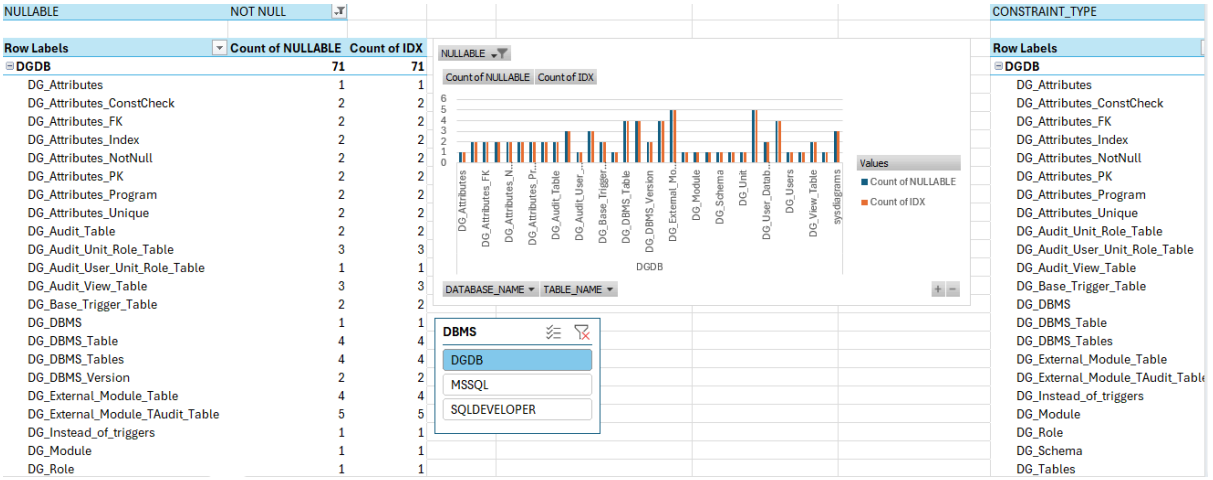

Row Labels # of CC
Company 3
DEPARTMENT 1
EMPLOYEE 2
OES21 20
branch 2
EMPLOYEE 2
ORDERS 1
PRODVENDOR 1
region_state 4
return_category 1
RETURNPROD 1
salary 3
state_capital 2
VENDORPRICE 3
OOES 9
branch 3
ORDERS 1
PRODVENDOR 1
RETURNPROD 1
VENDORPRICE 3
ZeotaDB 7
branch 2
EMPLOYEE 2
ORDERS 1
RETURNPROD 1
CONSTRAINT_TYPE
# of CC
Total
Company
OES21
OOES
ZeotaDB
Total
DATABASE_NAME
TABLE_NAME
DBMS
SQLDEVELOPER
DGDB
MSSQL
Row Labels # of UQ
Grand Total
# of UQ
Total
Grand Total
DATABASE_NAME
TABLE_NAME
DBMS
DGDB
MSSQL
SQLDEVELOPER

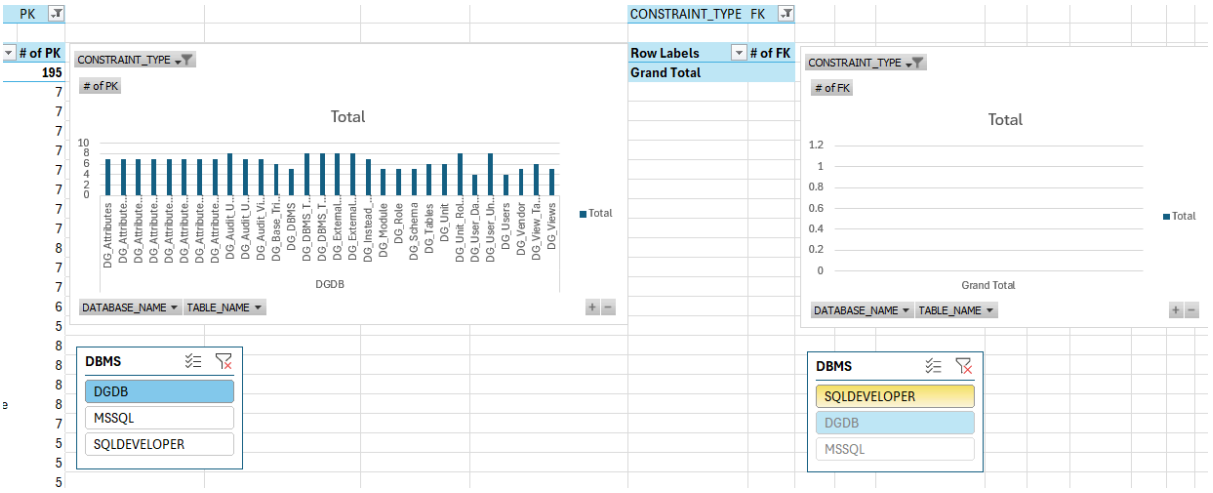
3.3.2.2. MSSQL

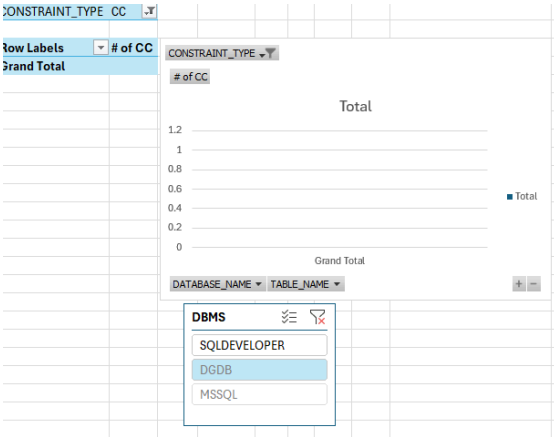

NULLABLE NOT NULL
Row Labels Count of NULLABLE Count of IDX
book 11 11
AUTHOR 1 1
BOOK 1 1
branch 1 1
INVENTORY 2 2
PUBLISHER 1 1
sysdiagrams 3 3
WROTE 2 2
Company 26 26
DEPARTMENT 4 4
DEPENDENT 5 5
DEPT_LOCATIONS 2 2
EMPLOYEE 7 7
PROJECT 3 3
sysdiagrams 3 3
WORKS_ON 2 2
IT4 28 28
CERT 2 2
CERTIFICATION 1 1
COMPANY 1 1
COURSE 1 1
COURSE_PROGRAM 2 2
DEPARTMENT 1 1
NULLABLE
Count of NULLABLE Count of IDX
Values
Count of NULLABLE
Count of IDX
ZeotaDB
DATABASE_NAME
TABLE_NAME
DBMS
DGDB
MSSQL
SQLDEVELOPER
CONSTRAINT_TYPE PK
Row Labels # of PK
book 22
AUTHOR 3
BOOK 6
branch 4
INVENTORY 3
PUBLISHER 3
WROTE 3
Company 28
DEPARTMENT 4
DEPENDENT 5
DEPT_LOCATIONS 2
EMPLOYEE 10
PROJECT 4
WORKS_ON 3
IT4 99
CERT 3
CERTIFICATION 4
COMPANY 2
COURSE 4
COURSE_PROGRAM 5
DEPARTMENT 2
EMPLOYED 4
ENROLLMENT 8
PK
# of PK
CONSTRAINT_TYPE
# of PK
Total
ZeotaDB
Total
DATABASE_NAME
TABLE_NAME
DBMS
DGDB
MSSQL
SQLDEVELOPER
CONSTRAINT_TYPE FK
Row Labels # of FK
Grand Total
CONSTRAINT_TYPE
# of FK
Total
Chart Area
Grand Total
Total
DATABASE_NAME
TABLE_NAME
DBMS
SQLDEVELOPER
DGDB
MSSQL

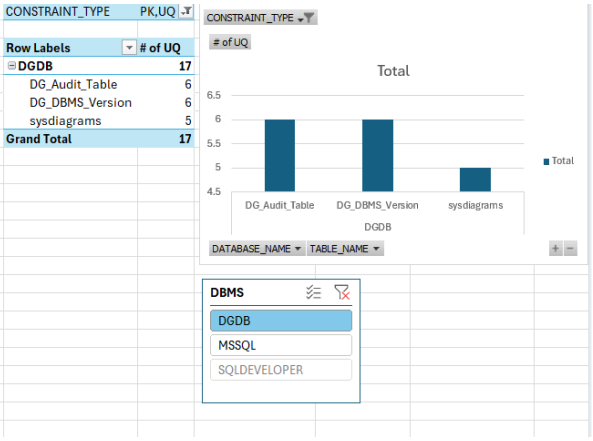

CONSTRAINT_TYPE CC
Row Labels # of CC
Grand Total
CONSTRAINT_TYPE
# of CC
Total
Grand Total
DATABASE_NAME TABLE_NAME
DBMS
SQLDEVELOPER
DGDB
MSSQL
CONSTRAINT_TYPE PK,UQ
Row Labels # of UQ
book 5
sysdiagrams 5
Company 5
sysdiagrams 5
oes2 5
sysdiagrams 5
Prop2 5
sysdiagrams 5
Sales 5
sysdiagrams 5
SP 5
sysdiagrams 5
ZeotaDB 5
sysdiagrams 5
Grand Total 35
CONSTRAINT_TYPE
# of UQ
Total
book
Company
oes2
Prop2
Sales
SP
ZeotaDB
DATABASE_NAME TABLE_NAME
DBMS
DGDB
MSSQL
SQLDEVELOPER

3.3.2.3. DGDB


NULLABLE NOT NULL
Row Labels Count of NULLABLE Count of IDX
DGDB 71 71
DG_Attributes 1 1
DG_Attributes_ConstCheck 2 2
DG_Attributes_FK 2 2
DG_Attributes_Index 2 2
DG_Attributes_NotNull 2 2
DG_Attributes_PK 2 2
DG_Attributes_Program 2 2
DG_Attributes_Unique 2 2
DG_Audit_Table 2 2
DG_Audit_Unit_Role_Table 3 3
DG_Audit_User_Unit_Role_Table 1 1
DG_Audit_View_Table 3 3
DG_Base_Trigger_Table 2 2
DG_DBMS 1 1
DG_DBMS_Table 4 4
DG_DBMS_Tables 4 4
DG_DBMS_Version 2 2
DG_External_Module_Table 4 4
DG_External_Module_TAudit_Table 5 5
DG_Instead_of_triggers 1 1
DG_Module 1 1
DG_Role 1 1
NULLABLE
Count of NULLABLE Count of IDX
DGDB
Values
Count of NULLABLE
Count of IDX
DATABASE_NAME TABLE_NAME
DBMS
DGDB
MSSQL
SQLDEVELOPER
CONSTRAINT_TYPE
Row Labels
DGDB
DG_Attributes
DG_Attributes_ConstCheck
DG_Attributes_FK
DG_Attributes_Index
DG_Attributes_NotNull
DG_Attributes_PK
DG_Attributes_Program
DG_Attributes_Unique
DG_Audit_Unit_Role_Table
DG_Audit_User_Unit_Role_Table
DG_Audit_View_Table
DG_Base_Trigger_Table
DG_DBMS
DG_DBMS_Table
DG_DBMS_Tables
DG_External_Module_Table
DG_External_Module_TAudit_Table
DG_Instead_of_triggers
DG_Module
DG_Role
DG_Schema
DG_Tables


PK
# of PK
195
CONSTRAINT_TYPE
# of PK
Total
DGDB
DATABASE_NAME TABLE_NAME
DBMS
DGDB
MSSQL
SQLDEVELOPER
CONSTRAINT_TYPE FK
Row Labels # of FK
Grand Total
CONSTRAINT_TYPE
# of FK
Total
Grand Total
DATABASE_NAME TABLE_NAME
DBMS
SQLDEVELOPER
DGDB
MSSQL





3.3.3. For every schema, list Table_ID, Table_Name, attribute_id, attribute_Name, PK_ID, FK_ID, NN_ID, constraint_type, and constraint_Name

3.3.3.1. Oracle

			CONSTRAINT_TYPE Values						
			CC			CC, FK			CC, PK
DATABASE_NAME	TABLE_NAME	COLUMN_NAME	Count of TABLE_ID	Count of CONSTRAINT_NAME	Sum of attribute_id	Count of TABLE_ID	Count of CONSTRAINT_NAME	Sum of attribute_id	Count of TABLE_ID
book	AUTHOR	AUTHOR_FIRST							
		AUTHOR_LAST							
		AUTHOR_NUM							
	AUTHOR Total								
	BOOK	BOOK_CODE							
		PAPERBACK							
		PRICE							
		PUBLISHER_CODE							
		TITLE							
		TYPE							
	BOOK Total								
	branch	BRANCH_LOCATION							
		BRANCH_NAME							
		BRANCH_NUM							
		NUM_EMPLOYEES							
	branch Total								
	INVENTORY	BOOK_CODE							
		BRANCH_NUM							
		ON_HAND							
	INVENTORY Total								
	PUBLISHER	CITY							
		PUBLISHER_CODE							
		PUBLISHER_NAME							
	PUBLISHER Total								
	TEMP_NN_COUNT	NN_COUNT							
		schema_name							
	TEMP_NN_COUNT Total								
	WROTE	AUTHOR_NUM							
		BOOK_CODE							
		SEQUENCE							
	WROTE Total								
book Total									

3.3.3.2. MSSQL

			CONSTRAINT_TYPE Values											
			NULL			PK			PK,UQ					
DATABASE_NAME	TABLE_NAME	COLUMN_NAME	Count of TABLE_ID	Count of CONSTRAINT_NAME	Sum of attribute_id	Count of TABLE_ID	Count of CONSTRAINT_NAME	Sum of attribute_id	Count of TABLE_ID	Count of CONSTRAINT_NAME	Sum of attribute_id	Total Count of TABLE_ID	Total Count of CONSTRAINT_NAME	Total Sum of attribute_id
book	AUTHOR	AUTHOR_FIRST				1	1	3				1	1	3
		AUTHOR_LAST				1	1	2				1	1	2
		AUTHOR_NUM				1	1	1				1	1	1
	AUTHOR Total					3	3	6				3	3	6
	BOOK	BOOK_CODE				1	1	1				1	1	1
		PAPERBACK				1	1	6				1	1	6
		PRICE				1	1	5				1	1	5
		PUBLISHER_CODE				1	1	3				1	1	3
		TITLE				1	1	2				1	1	2
		TYPE				1	1	4				1	1	4
	BOOK Total					6	6	21				6	6	21
	branch	BRANCH_LOCATION				1	1	3				1	1	3
		BRANCH_NAME				1	1	2				1	1	2
		BRANCH_NUM				1	1	1				1	1	1
		NUM_EMPLOYEES				1	1	4				1	1	4
	branch Total					4	4	10				4	4	10
	DatabaseDictionary	ColumnName	1	1	3							1	1	3
		DataDefault	1	1	8							1	1	8
		DataType	1	1	4							1	1	4
		IsForeignKey	1	1	10							1	1	10
		IsNullable	1	1	7							1	1	7
		IsPrimaryKey	1	1	9							1	1	9
		Precision	1	1	5							1	1	5
		Scale	1	1	6							1	1	6
		SchemaName	1	1	1							1	1	1
		TableName	1	1	2							1	1	2
	DatabaseDictionary Total		10	10	55							10	10	55
	INVENTORY	BOOK_CODE				1	1	1				1	1	1
		BRANCH_NUM				1	1	2				1	1	2
		ON_HAND				1	1	3				1	1	3
	INVENTORY Total					3	3	6				3	3	6
	PUBLISHER	CITY				1	1	3				1	1	3
		PUBLISHER_CODE				1	1	1				1	1	1
		PUBLISHER_NAME				1	1	2				1	1	2
	PUBLISHER Total					3	3	6				3	3	6
	sysdiagrams	definition							1	1	5	1	1	5
		diagram_id							1	1	3	1	1	3
		name							1	1	1	1	1	1
		principal_id							1	1	2	1	1	2
		version							1	1	4	1	1	4
	sysdiagrams Total								5	5	15	5	5	15

3.3.3.3. DGDB

			CONSTRAINT_TYPE	Values										
			NULL			PK			PK,UQ					
DATABASE_NAME	TABLE_NAME	COLUMN_NAME	Count of TABLE_ID	Count of CONSTRAINT_NAME	Sum of attribute_id	Count of TABLE_ID	Count of CONSTRAINT_NAME	Sum of attribute_id	Count of TABLE_ID	Count of CONSTRAINT_NAME	Sum of attribute_id	Total Count of TABLE_ID	Total Count of CONSTRAINT_NAME	Total Sum of attribute_id
DGDB	DatabaseDictionary	ColumnName	1	1	3							1	1	3
		DataDefault	1	1	8							1	1	8
		DataType	1	1	4							1	1	4
		IsForeignKey	1	1	10							1	1	10
		IsNullable	1	1	7							1	1	7
		IsPrimaryKey	1	1	8							1	1	8
		Precision	1	1	5							1	1	5
		Scale	1	1	6							1	1	6
		SchemaName	1	1	1							1	1	1
		TableName	1	1	2							1	1	2
	DatabaseDictionary Total		10	10	55							10	10	55
	DG_Attributes	Attributes_ID				1	1	3				1	1	3
		Attributes_Name				1	1	4				1	1	4
		comment				1	1	5				1	1	5
		create_date				1	1	6				1	1	6
		modify_date				1	1	7				1	1	7
		Table_ID				1	1	1				1	1	1
		Table_Name				1	1	2				1	1	2
	DG_Attributes Total					7	7	28				7	7	28
	DG_Attributes_ConstCheck	Attributes_ID				1	1	4				1	1	4
		CC_ID				1	1	3				1	1	3
		comment				1	1	5				1	1	5
		Constraint_name				1	1	1				1	1	1
		Constraint_Type				1	1	2				1	1	2
		create_date				1	1	6				1	1	6
		modify_date				1	1	7				1	1	7
	DG_Attributes_ConstCheck Total					7	7	28				7	7	28
	DG_Attributes_FK	Attributes_ID				1	1	4				1	1	4
		comment				1	1	5				1	1	5
		Constraint_name				1	1	1				1	1	1
		Constraint_Type				1	1	2				1	1	2
		create_date				1	1	6				1	1	6
		FK_ID				1	1	3				1	1	3
		modify_date				1	1	7				1	1	7
	DG_Attributes_FK Total					7	7	28				7	7	28
	DG_Attributes_Index	Attributes_ID				1	1	4				1	1	4
		comment				1	1	5				1	1	5
		Constraint_name				1	1	1				1	1	1
		Constraint_Type				1	1	2				1	1	2
		create_date				1	1	6				1	1	6
		IDX_ID				1	1	3				1	1	3

3.3.4. For every schema, analyze PK, FK,and IDX

3.3.4.1. list tables without PK

3.3.4.1.1. Oracle

has_primary_key	NO	
DATABASE_NAME	**TABLE_NAME**	**Count of COLUMN_NAME**
book	TEMP_NN_COUNT	2
book Total		**2**
IT4	MISSING_GEN_ED	7
	MISSING_PREREQS	7
	RETAKE_GEN_ED	8
	RETAKE_PREREQS	8
IT4 Total		**30**
OES21	region_state	5
	return_category	2
	salary	3
	state_capital	4
	timedimension	2
OES21 Total		**16**
Prop2	lease	10
Prop2 Total		**10**
Grand Total		**58**

3.3.4.1.2. MSSQL

has_primary_key	NO	
DATABASE_NAME	TABLE_NAME	Count of COLUMN_NAME
book	DatabaseDictionary	10
book Total		10
Company	DatabaseDictionary	10
Company Total		10
oes2	DatabaseDictionary	10
	region_state	5
	return_category	2
	salary	3
	state_capital	4
	timedimension	4
oes2 Total		28
Prop2	DatabaseDictionary	10
	lease	10
Prop2 Total		20
SP	DatabaseDictionary	10
SP Total		10
ZeotaDB	DatabaseDictionary	10
ZeotaDB Total		10
Grand Total		88

3.3.4.1.3. DGDB

has_primary_key	NO	
DATABASE_NAME	TABLE_NAME	Count of COLUMN_NAME
DGDB	DatabaseDictionary	10
DGDB Total		10
Grand Total		10

3.3.4.2. list tables with PK but without FK

3.3.4.2.1. Oracle

has_primary_key	NO	
has_foreign_key	YES	
DATABASE_NAME	TABLE_NAME	Count of COLUMN_NAME
OES21	region_state	5
OES21 Total		5
Grand Total		5

3.3.4.2.2. MSSQL

has_primary_key	NO	
has_foreign_key	YES	
DATABASE_NAME	TABLE_NAME	Count of COLUMN_NAME
oes2	region_state	5
oes2 Total		5
Grand Total		5

3.3.4.2.3. DGDB

has_primary_key	NO	
has_foreign_key	YES	
DATABASE_NAME	TABLE_NAME	Count of COLUMN_NAME
Grand Total		

3.3.4.3. list tables with PK but without IDX

3.3.4.3.1. Oracle

has_primary_key	YES	
has_foreign_key	(All)	
IDX	NULL	
DATABASE_NAME	TABLE_NAME	Count of COLUMN_NAME
Grand Total		

3.3.4.3.2. MSSQL

has_primary_key	YES	
has_foreign_key	(All)	
IDX	NULL	
DATABASE_NAME	TABLE_NAME	Count of COLUMN_NAME
Grand Total		

3.3.4.3.3. DGDB

has_primary_key	YES	
has_foreign_key	(All)	
IDX	NULL	
DATABASE_NAME	TABLE_NAME	Count of COLUMN_NAME
Grand Total		

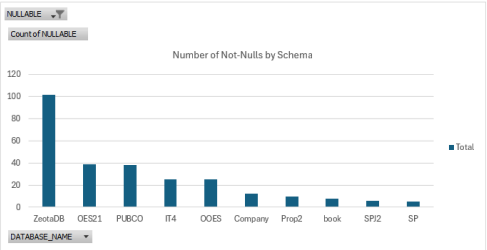
3.3.5. For every database/schema,

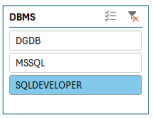
3.3.5.1. rank them according to number of NN (Not Null) have created.

3.3.5.1.1. Oracle

a. For every database/schema, rank them according to number of NN (Not Null) have created

NULLABLE	NOT NULL
Row Labels	Count of NULLABLE
ZeotaDB	102
OES21	39
PUBCO	38
IT4	25
OOES	25
Company	12
Prop2	10
book	8
SPJ2	6
SP	5
Grand Total	270



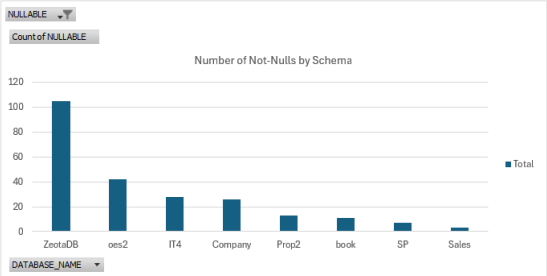


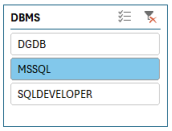
b. For every database/schema, every table list fields with NN

3.3.5.1.2. MSSQL

a. For every database/schema, rank them according to number of NN (Not Null) have created

NULLABLE	NOT NULL
Row Labels	Count of NULLABLE
ZeotaDB	105
oes2	42
IT4	28
Company	26
Prop2	13
book	11
SP	7
Sales	3
Grand Total	235





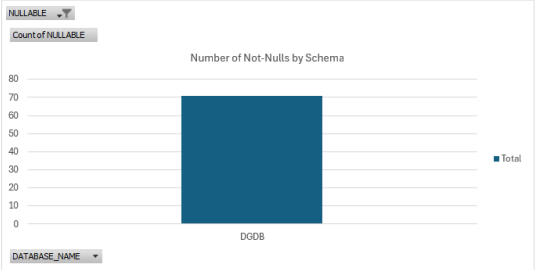
b. For every database/schema, every table list fields with NN

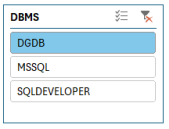
3.3.5.1.3. DGDB

a. For every database/schema, rank them according to number of NN (Not Null) have created

NULLABLE	NOT NULL

Row Labels	Count of NULLABLE
DGDB	71
Grand Total	**71**





b. For every database/schema, every table list fields with NN

3.3.5.2. For every table list fields with NN

3.3.5.2.1. Oracle

b. For every database/schema, every table list fields with NN

NULLABLE	NOT NULL

Row Labels
book
AUTHOR
AUTHOR_NUM
BOOK
BOOK_CODE
branch
BRANCH_NUM
INVENTORY
BOOK_CODE
BRANCH_NUM
PUBLISHER
PUBLISHER_CODE
WROTE
AUTHOR_NUM
BOOK_CODE
Company
DEPARTMENT
DNAME
DNUMBER
DEPENDENT
DEPENDENT_NAME
ESSN
DEPT_LOCATION

ZeotaDB
backorder
backorder_no
branch
BRANCH_NO
CITY
Branch_Size
BS_NO
Certificate
Cert_NO
COMPANY
Comp_NO
CUSTOMER
CUSTOMER_NO
DEPARTMENT
dept_no
Dependents
Dependent_NO
Dept_Branch
BRANCH_NO
dept_no
Dept_Emp
dept_no
Emp_NO
Dept_Mgr
dept_no
Emp_NO

3.3.5.2.2. MSSQL

b. For every database/schema, every table list fields with NN

NULLABLE	NOT NULL

Row Labels
book
AUTHOR
AUTHOR_NUM
BOOK
BOOK_CODE
branch
BRANCH_NUM
INVENTORY
BOOK_CODE
BRANCH_NUM
PUBLISHER
PUBLISHER_CODE
sysdiagrams
diagram_id
name
principal_id
WROTE
AUTHOR_NUM
BOOK_CODE
Company
DEPARTMENT
DNAME

ZeotaDB
backorder
backorder_no
branch
BRANCH_NO
CITY
Branch_Size
BS_NO
Certificate
Cert_NO
COMPANY
Comp_NO
CUSTOMER
CUSTOMER_NO
DEPARTMENT
dept_no
Dependents
Dependent_NO
Dept_Branch
BRANCH_NO
dept_no
Dept_Emp
dept_no
Emp_NO
Dept_Mgr
dept_no
Emp_NO
Dept_Position

3.3.5.2.3. DGDB

b. For every database/schema, every table list fields with NN

NULLABLE	NOT NULL

Row Labels
DGDB
DG_Attributes
Attributes_ID
DG_Attributes_ConstCheck
Attributes_ID
CC_ID
DG_Attributes_FK
Attributes_ID
FK_ID
DG_Attributes_Index
Attributes_ID
IDX_ID
DG_Attributes_NotNull
Attributes_ID
NN_ID
DG_Attributes_PK
Attributes_ID
PK_ID
DG_Attributes_Program
Attributes_ID
PGM_ID
DG_Attributes_Unique
Attributes_ID
UQI_ID
DG_Unit_Role_Table
DB_Operation
Role_ID
Table_ID
Unit_ID
URT_ID
DG_User_Database
Database_ID
User_ID
DG_User_Unit_Role
Role_ID
start_date
Unit_ID
User_ID
DG_Users
User_ID
DG_Vendor
VDR_ID
DG_View_Table
Table_ID
View_ID
DG_Views
View_ID
sysdiagrams
diagram_id
name
principal_id
Grand Total

3.3.6. For every database, list tables without PK

3.3.6.1. Oracle

has_primary_key	NO

Row Labels
book
TEMP_NN_COUNT
IT4
MISSING_GEN_ED
MISSING_PREREQS
RETAKE_GEN_ED
RETAKE_PREREQS
OES21
region_state
return_category
salary
state_capital
timedimension
Prop2
lease
Grand Total

DBMS

- DGDB
- MSSQL
- SQLDEVELOPER

3.3.6.2. MSSQL

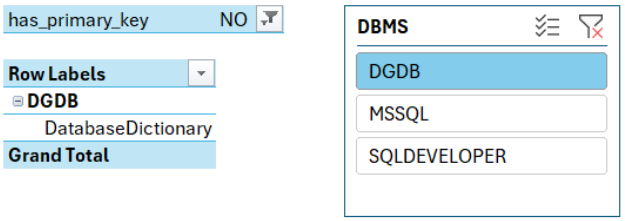
has_primary_key	NO

Row Labels
book
DatabaseDictionary
Company
DatabaseDictionary
oes2
DatabaseDictionary
region_state
return_category
salary
state_capital
timedimension
Prop2
DatabaseDictionary
lease
SP
DatabaseDictionary
ZeotaDB
DatabaseDictionary
Grand Total

DBMS

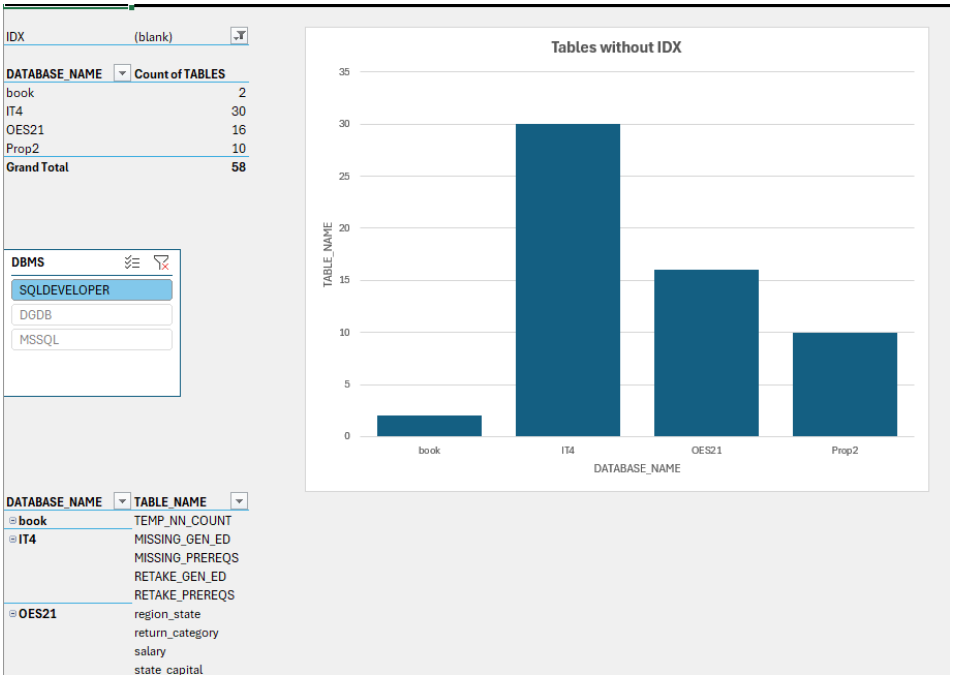
- DGDB
- MSSQL
- SQLDEVELOPER

3.3.6.3. DGDB

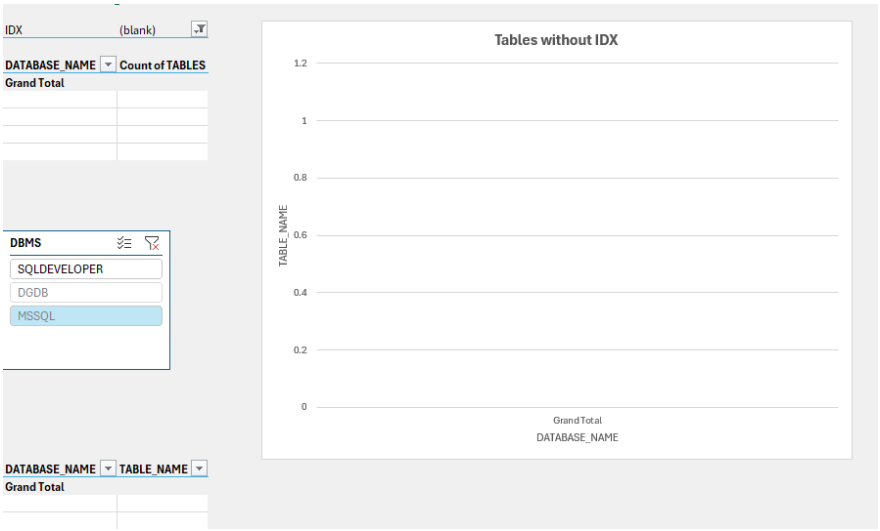


3.3.7. For every database, list tables without IDX

3.3.7.1. Oracle



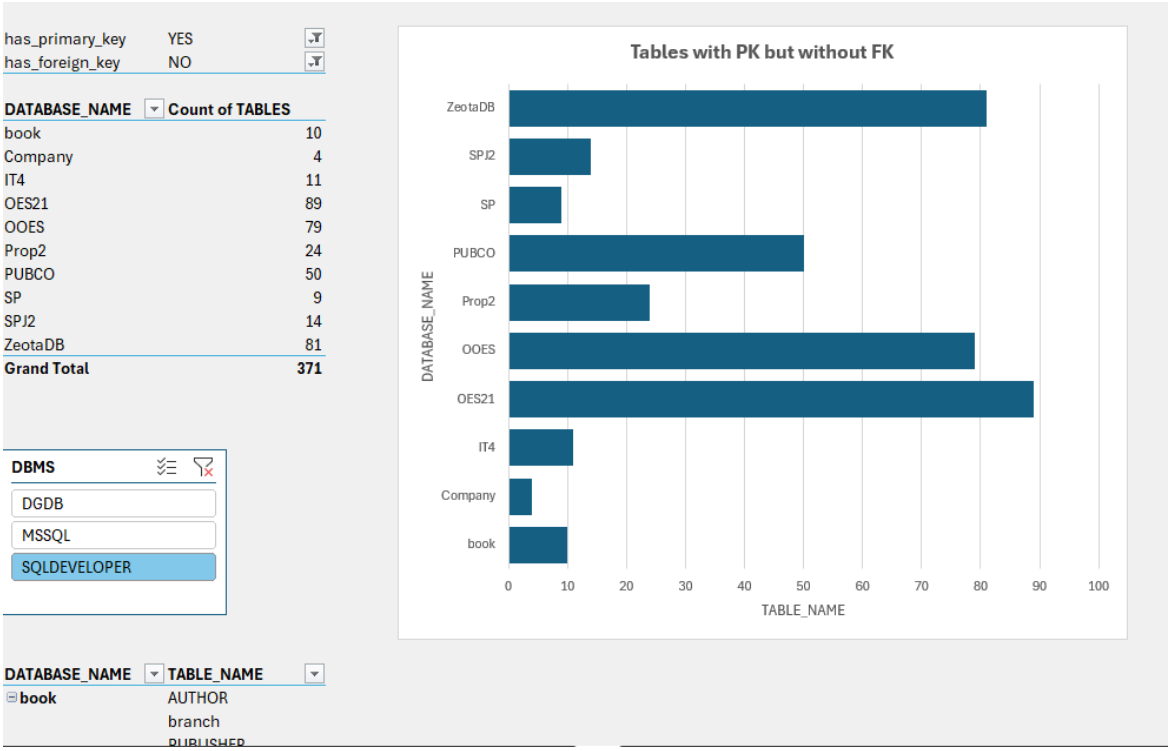
3.3.7.2. MSSQL



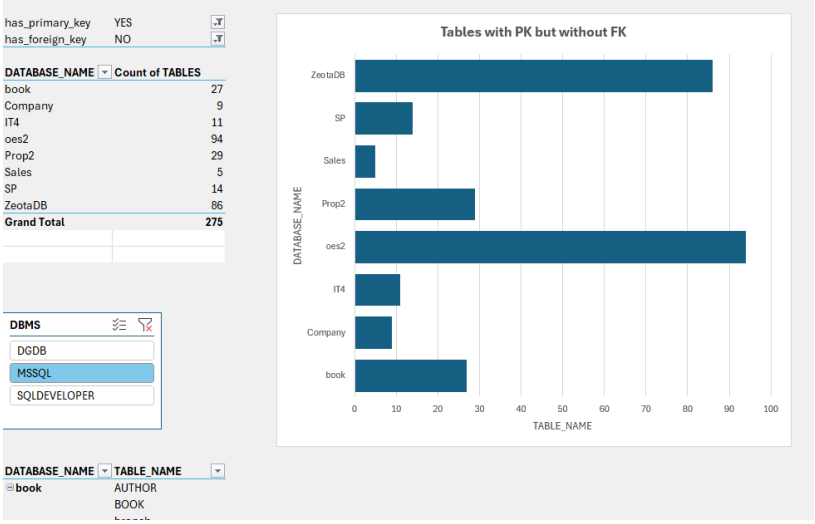
3.3.7.3. DGDB

3.3.8. For every database, list tables with PK but without FK (D)

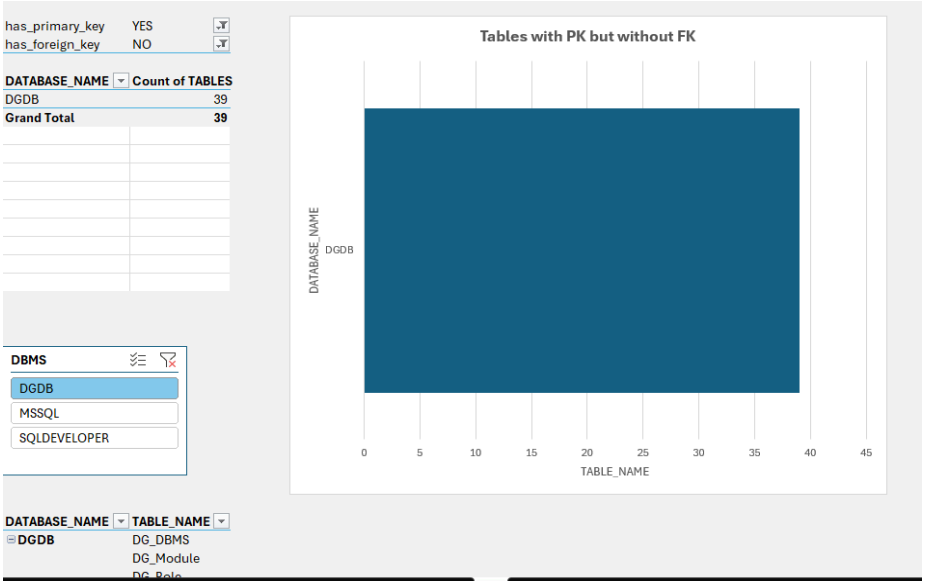
3.3.8.1. Oracle



3.3.8.2. MSSQL



3.3.8.3. DGDB



3.3.9. For every database/schema, list tables with IDX, index name (constraint name), fields used in IDX

3.3.9.1. Oracle

IDX	(Multiple Items)	
DATABASE_NAME	TABLE_NAME	CONSTRAINT_NAME
book	AUTHOR	SYS_C006987
		(blank)
	BOOK	BOOK_PUBLISHER_CODE_FK
		SYS_C006990
		(blank)
	branch	SYS_C006989
		(blank)
	INVENTORY	INVENTORY_BOOK_CODE_FK, SYS_C006991, SYS_C006993
		INVENTORY_BRANCH_NUM_FK, SYS_C006992, SYS_C006993
		(blank)
	PUBLISHER	SYS_C006988
		(blank)
	WROTE	SYS_C006994, SYS_C006995, WROTE_AUTHOR_NUM_FK
		SYS_C006995, WROTE_BOOK_CODE_FK
		(blank)
book Total		
Company	DEPARTMENT	DEPARTMENT_PK, SYS_C007650
		SYS_C007649
		(blank)
	DEPENDENT	DEPENDENT_FK, DEPENDENT_PK, SYS_C007669
		DEPENDENT_PK
		(blank)
	DEPT_LOCATION	DEPARTMENT_LOCATION_PK, DEPT_LOCATION_FK, SYS_C007657
		DEPARTMENT_LOCATION_PK, SYS_C007658
	EMPLOYEE	EMPLOYEE_FK
		EMPLOYEE_PK, SYS_C007654
		SYS_C007652
		SYS_C007653
		(blank)
	PROJECT	PROJECT_FK
		PROJECT_PK, SYS_C007661

DBMS: DGDB, MSSQL, SQLDEVELOPER

IDX included everything except NULL and Blank
(518 rows)

3.3.9.2. MSSQL

IDX	(Multiple Items)	
DATABASE_NAME	TABLE_NAME	CONSTRAINT_NAME
book	AUTHOR	PK__AUTHOR__5C4B4B5EB2F4C670
	BOOK	PK__BOOK__D8BE9AD379326CE8
	branch	PK__BRANCH__14C8CD2C6A85D990
	INVENTORY	PK__INVENTOR__09F2160112EFCFF3
	PUBLISHER	PK__PUBLISHE__79AFDF6696A0CFE3
	sysdiagrams	PK__sysdiagr__C2B05B61664F2E04,UK_principal_name
	WROTE	PK__WROTE__2D7A2E66420718E8
book Total		
Company	DEPARTMENT	DEPT_DNUM_PK
	DEPENDENT	DEP_ESSN_NAME_PK
	DEPT_LOCATIONS	DEPTLOC_DNUM_DLOC_PK
	EMPLOYEE	EMP_SSN_PK
	PROJECT	PROJ_PNUM_PK
	sysdiagrams	PK__sysdiagr__C2B05B6139E324BF,UK_principal_name
	WORKS_ON	WRK_ESSN_PNO_PK
Company Total		
IT4	CERT	PK__CERT__575FCEDBA778BBFA
	CERTIFICATION	PK__CERTIFIC__87EC34A276D15C81
	COMPANY	PK__COMPANY__6065E2B26665A045
	COURSE	PK__COURSE__C81B5B25E5BE4ADE
	COURSE_PROGRAM	PK__COURSE_P__6DDBE60F6A5A8B9E
	DEPARTMENT	PK__DEPARTME__DA3BF910A03DFDB7
	EMPLOYED	PK__EMPLOYED__B6E38D71DA7F866B
	ENROLLMENT	PK__ENROLLME__F60371824BAFF260
	INTERNSHIP	PK__INTERNSH__664F18F76CB23863
	JOB	PK__JOB__C4DECBEE0C9D57F4
	prerequisite	pre_c_o_p_pk
	PROGRAM	PK__PROGRAM__215A53BD847D387E
	STUD_PROG	STUD_PROG_PK
	STUDENT	PK__STUDENT__6F210D91FD52C722
	STUDENT_WORKED	PK__STUDENT__A52D15221EF0EEBE
	TEACHER	PK__TEACHER__E3D873762C919C18
IT4 Total		

DBMS: DGDB, MSSQL, SQLDEVELOPER

IDX included everything except NULL and Blank
(143 rows)

3.3.9.3. DGDB

IDX	(Multiple Items)

DATABASE_NAME	TABLE_NAME	CONSTRAINT_NAME
DGDB	DG_Attributes	DG_Attributes_Attributes_ID_PK
	DG_Attributes_Const	DG_Attributes_ConstCheck_CC_ID_Attributes_ID_PK
	DG_Attributes_FK	DG_Attributes_FK_FK_ID_Attributes_ID_PK
	DG_Attributes_Index	DG_Attributes_Index_IDX_ID_Attributes_ID_PK
	DG_Attributes_NotNu	DG_Attributes_NotNull_NN_ID_Attributes_ID_PK
	DG_Attributes_PK	DG_Attributes_PK_PK_ID_Attributes_ID_PK
	DG_Attributes_Progra	DG_Attributes_Program_PGM_ID_Attributes_ID_PK
	DG_Attributes_Uniqu	DG_Attributes_Unique_UQI_ID_Attributes_ID_PK
	DG_Audit_Table	DG_Audit_Table_Audit_ID_Table_ID_PK,UQ__DG_Audit__EDBEC758B2E86E32
	DG_Audit_Unit_Role_	DG_Audit_T_ID_Table_ID_Database_ID_PK
	DG_Audit_User_Unit_	DG_Audit_T_ID_PK
	DG_Audit_View_Table	DG_Audit_T_ID_ID_Table_ID_Database_ID_PK
	DG_Base_Trigger_Tab	DG_Base_Trigger_Table_Trigger_ID_Table_ID_PK
	DG_DBMS	DG_DBMS_DBMS_ID_PK
	DG_DBMS_Table	DG_DBMS_Table_DBMS_ID_Audit_T_ID_Table_ID_Database_ID_PK
	DG_DBMS_Tables	DG_DBMS_Tables_DBMS_ID_DBMS_V_ID_Table_ID_Database_ID_PK
	DG_DBMS_Version	DG_DBMS_ID_DBMS_V_ID_PK,UQ__DG_DBMS___34E622E5052F904F
	DG_External_Module	DG_External_Module_Table_EMT_ID_Table_id_Vendor_ID_PK
	DG_External_Module	DG_External_Module_TAudit_Table_Audit_T_ID_EMT_ID_Table_id_VDR_ID_Database_ID_PK
	DG_Instead_of_trigge	DG_Instead_of_triggers_Trigger_ID_PK
	DG_Module	DG_Module_Module_ID_PK
	DG_Role	DG_Role_Role_ID_PK
	DG_Schema	DG_Schema_schema_ID_PK
	DG_Tables	DG_Tables_Table_ID_PK
	DG_Unit	DG_Unit_Unit_ID_PK
	DG_Unit_Role_Table	DG_Unit_Role_Table_URT_ID_Table_id_Role_ID_Unit_ID_PK
	DG_User_Database	DG_User_Database_User_ID_Database_ID_PK
	DG_User_Unit_Role	DG_User_Unit_Role_User_ID_Role_ID_Unit_ID_start_date_PK
	DG_Users	DG_User_User_ID_Database_ID_PK
	DG_Vendor	DG_Vendor_VDR_ID_PK
	DG_View_Table	DG_View_Table_Table_ID_View_ID_PK
	DG_Views	DG_View_Table_ID_View_ID_PK

DBMS
DGDB
MSSQL
SQLDEVELOPE

IDX included everything except NULL and Blank
(34 rows)

3.3.10. For every database/schema, list tables with CC

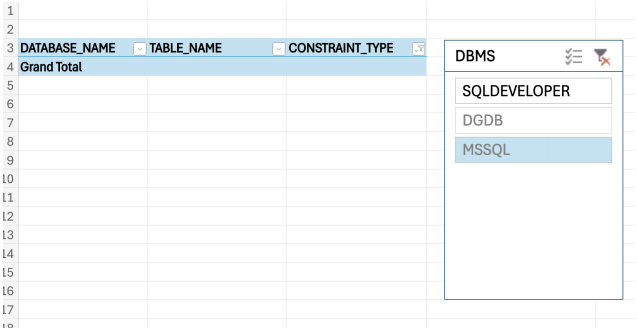
3.3.10.1. Oracle

DATABASE_NAME	TABLE_NAME	CONSTRAINT_TYPE
book	INVENTORY	CC, PK, FK
	INVENTORY Total	
	WROTE	CC, PK, FK
	WROTE Total	
Company	DEPARTMENT	CC
		CC, PK
	DEPARTMENT Total	
	DEPENDENT	CC, PK, FK
	DEPENDENT Total	
	DEPT_LOCATION	CC, PK
		CC, PK, FK
	DEPT_LOCATION Total	
	EMPLOYEE	CC
		CC, PK
	EMPLOYEE Total	
	PROJECT	CC, PK
	PROJECT Total	
	WORKS_ON	CC, PK, FK
	WORKS_ON Total	
IT4	CERT	CC, PK, FK
	CERT Total	
	CERTIFICATION	CC, PK
	CERTIFICATION Total	
	COMPANY	CC, PK
	COMPANY Total	
	COURSE	CC, PK
	COURSE Total	
	COURSE_PROGRAM	CC, PK, FK
	COURSE PROGRAM Total	

DBMS
SQLDEVELOPER
DGDB
MSSQL

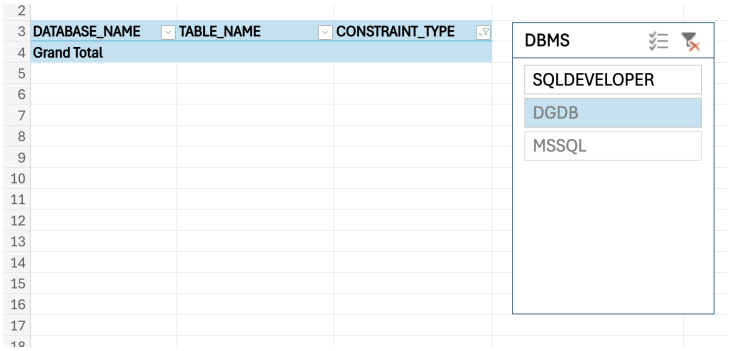
Constraint Type included ‘CC’, ‘CC, FK’, ‘CC, PK’, ‘CC, FK, PK’
(191 rows)

3.3.10.2. MSSQL



Constraint Type included ‘CC’, ‘CC, FK’, ‘CC, PK’, ‘CC, FK, PK’ (no result)

3.3.10.3. DGDB



Constraint Type included ‘CC’, ‘CC, FK’, ‘CC, PK’, ‘CC, FK, PK’ (no result)

3.4. Link all sheets to the PUI templates

Refer to Excel file attached to test linking.

Scrum Report

Scrum_Matrix – Sprint 3

Scrum		David Onyango			Eunsoo Park			Kiara Boyer			Pramudya Wicaksono			Rei Onishi		
1	Planned/Actual	Assign	Verify	Done	Assign	Verify	Done	Assign	Verify	Done	Assign	Verify	Done	Assign	Verify	Done
	Task1.1 Comparison for Oracle	A	EP	A	A	KB	A	A	PW	A	A	RO	A	A	DO	A
	Task1.2 Comparison for MSSQL	A	EP	A	A	KB	A	A	PW	A	A	RO	A	A	DO	A
	Task1.3 Comparison for DGDB	A	EP	A	A	KB	A	A	PW	A	A	RO	A	A	DO	A
2	Due Feb 16th															
	Task 2.1 Implement QList 1 in Oracle	A	EP	A	A	KB	A	A	PW	A	A	RO	A	A	DO	A
	Task2.2 Implement QList 1 in MSSQL	A	EP	A	A	KB	A	A	PW	A	A	RO	A	A	DO	A
	Task2.3 Implement QList 1 in MSSQL	A	EP	A	A	KB	A	A	PW	A	A	RO	A	A	DO	A
3	Due Feb 24th - changed to 28th															
	Task 3.1 Develop PUI	A	EP	A												
	Task 3.2 Implement PUI design using Excel Pivot	A	EP	A							A	RO	A			
	Task 3.3 Modify the pivot worksheet developed in Sprint2 to match the proposed solution for Qlist1	A	EP	A	A	KB	A	A	PW	A	A	RO	A	A	DO	A
	Task 3.4 Make sure the result sets from pivot matches what you have developed in SQL and PLSQL matches implementations	A	EP	A	A	KB	A	A	PW	A	A	RO	A	A	DO	A
	Task3.5 Link all sheets to the PUI templates	A	EP	A												
4	Due Mar 2nd															
	Task 4.1 Presentation file				A	KB	A									
	Task4.2 Report – Cover Page				A	KB	A									
	Task4.3 Report – Table of Contents				A	KB	A									
	Task4.4 Report – Introduction							A	PW	A						
	Task4.5 Report –Sprints	A	EP	A	A	KB	A	A	PW	A	A	RO	A	A	DO	A
	Task4.6 Report – Scrum Report				A	KB	A									
	Task4.7 Report – Final Analysis										A	RO	A			
	Task4.8 Report – Appendix													A	DO	A

Task4.9 Report – Next Sprint													A	DO	A
Task 4.10 Pert Chart							A	PW	A						
Task 4.11 Gantt Chart				A	KB	A									
Task 4.12 Burndown Chart				A	KB	A									
Task4.13 Script for video recording	A	EP	A	A	KB	A	A	PW	A	A	RO	A	A	DO	A
Task4.14 Presentation Recording	A	EP	A	A	KB	A	A	PW	A	A	RO	A	A	DO	A

Scrum Breakdown

Our goal in this sprint is to compare and verify DGDB, OSC, and MSC to produce discrepancy report. Our analysis will be done using SQL programming as well as Pivot programming.

By using SQL Programming and Pivot Tables, Implementing the following queries based on the following databases

- **DGDB database** (DGDB): This database contains information about meta data across all DBMSs in the organization. This database is implemented using MSSQL DBMS
- **Oracle System Catalog** (OSC): OSE provided meta data information about all databases implemented in Oracle DBMS
- **MSSQL System Catalog** (MSC): MSC provided meta data information about all databases implemented in MSSQL DBMS
- *** See sample useful programs using Oracle system catalog (click) in addition to lecture notes.
- *** See sample Oracle database programming (PLSQL) using system catalog (click) in addition to lecture notes.
- *** See data model representing the Oracle system catalog (click)

Query List 1 (Qlist1)

1. For every schema, list Schema, # tables, # records (K)
2. For every schema, list table name, # attributes, # PK_ attr, # FK_ attr, # NN_attr, # IDX_attr, # CC_attr, # UQ_attrs, # PGM_attr (K)
3. For every schema, list Table_ID, Table_Name, attribute_id, attribute_Name, PK_ID, FK_ID, NN_ID, constraint_type, and constraint_Name (E)
4. For every schema, analyze PK, FK,and IDX (E)
    a. list tables without PK
    b. list tables with PK but without FK
    c. list tables with PK but without IDX
5. For every database/schema, (P)
    a. rank them according to number of NN (Not Null) have created.
    b. For every table list fields with NN
6. For every database, list tables without PK (P)
7. For every database, list tables without IDX (D)
8. For every database, list tables with PK but without FK (D)
9. For every database/schema, list tables with IDX, index name (constraint name), fields used in IDX (R)
10. For every database/schema, list tables with CC (R)

Sprint Planning

1. Update product backlogs to include any additional tasks
2. From Product backlogs, drive sprint 2 backlog
3. Groom Sprint 2 backlogs to list all tasks to be completed in this sprint. Remember each task should be decomposed into smaller tasks that can be done in 2 to 4 hours.
4. Divide all tasks into 4 lists. Each list will be assigned to a scrum.
5. Identify each member capacity for each scrum.
6. For each scrum, each member SHOULD select several tasks to complete.
7. Every task should be verified by another member.
8. Every task should have the name of the member completing it and name of the member verified it. Use the form provided (scrum matrix)

Scrum1 (Meet Feb 16th, Complete by Feb 21st)

1. Make a comparison analysis the proposed solution with our query from Sprint 2

Scrum2 (Meet Feb 21st, Complete by Feb 24th)

1. Implement the Qlist1 against system catalog of Oracle System Catalog.
    - 1.1. Analyze OSC using SQL command
        - 1.1.1. For each query, list the query in English, program (SQL and DP Prog), and result to be included in Sprint report.
        - 1.1.2. All program to be included in the sprint report.
2. Implement the Qlist1 against system catalog of MSSQL.
    - 2.1. Analyze MSC using SQL commands
        - 2.1.1. For each query, list the query in English, program (SQL and DP Prog), and result to be included in Sprint report.
        - 2.1.2. All program to be included in the sprint report.
3. Implement the Qlist1 against DGDB.
    - 3.1. Analyze DGDB using SQL commands
        - 3.1.1. For each query, list the query in English, program (SQL and DP Prog), and result to be included in Sprint report.
        - 3.1.2. All program to be included in the sprint report.

Scrum3 (Meet Feb/ 24th, Complete by Feb 28th - Changed to Mar 2nd)

1. Implement the Qlist1 against system catalog of Oracle System Catalog.
    - 1.1. Analyze OSC Using Pivot Table
        - 1.1.1. For each query, list the query in English,
            - 1.1.1.1. screenshot of raw data in excel and paste it on a word doc page

1.1.1.2. screenshot of analysis containing report and chart; paste the screenshot on a word doc page
1.1.2. Attach the excel document to the Sprint report
2. Implement the Qlist1 against system catalog of MSSQL.
2.1. Analyze MSC using Pivot Table
2.1.1. For each query, list the query in English,
2.1.1.1. screenshot of raw data in excel and paste it on a word doc page
2.1.1.2. screenshot of analysis containing report and chart; paste the screenshot on a word doc page
2.1.2. Attach the excel document to the Sprint report
3. Implement the Qlist1 against DGDB.
3.1. Analyze DGDB using Pivot Table
3.1.1. For each query, list the query in English,
3.1.1.1. screenshot of raw data in excel and paste it on a word doc page
3.1.1.2. screenshot of analysis containing report and chart; paste the screenshot on a word doc page
3.1.2. Attach the excel document to the Sprint report
4. Report
4.1. Cover Page
4.2. Table of contents
4.3. Introduction
4.4. Sprints
4.5. Scrum report
4.6. Analysis (as it is said in video presentation)
4.7. Appendix
4.8. - Script for video recording
4.9. - Presentation Powerpoint

Scrum4 (Meet Mar 2nd, Complete by Mar 2nd)
- Presentation recording

Analysis

In this sprint, we replaced standard SQL queries with database programming techniques, examined metadata inconsistencies, and extended their Pivot implementation by creating an Excel Pivot User Interface (PUI). A key objective was to conduct a comprehensive comparison and verification of metadata across the database systems, identifying discrepancies and inconsistencies in schema, table attributes, constraints, and indexing patterns. The team also focused on improving the efficiency and accuracy of metadata analysis by implementing advanced database programming techniques and automating repetitive tasks. The ultimate goal was to compile our findings into a comprehensive sprint report and presentation, documenting their methodologies, results, and recommendations for future improvements.

We addressed several key deliverables, including comparing SQL implementations with proposed solutions for Qlist1 against system catalogs of Oracle and MSSQL, and the DGDB database in MSSQL. This involved tasks such as listing schemas, tables, and records; listing table names and attribute details; analyzing primary keys (PK), foreign keys (FK), and indexes (IDX); and listing tables with and without specific constraints. Additionally, the team repeated SQL implementations using database programming, replacing SQL queries. We also expanded the Pivot implementation by including a Pivot interface, developing a design for the Pivot User Interface (PUI), implementing the PUI design using an Excel Pivot worksheet, and ensuring the result sets from the pivot matched the SQL and PL/SQL implementations.

While this sprint is conducted, we have successfully and diligently completed this sprint within the time span. By implementing group meetings or scrums for updates, we were able to quickly adjust to our immediate needs that were unforeseen during the planning stage. Furthermore, implementation of Microsoft Teams as a means of direct and real-time communications was really helpful for us to assist each other in times of technical problems. However, during this process, we also encounter some hurdles, the biggest hurdle is timing. Taking into account the significant uptick of responsibilities with limited time, our team struggled with time management. We acknowledge that we weren't agile enough and adjust our usual plan to fit the increase in tasks that this project requires us to do. Therefore, we are planning for the next sprints to dedicate more time in resource and task management, increasing from 1 scrum meeting to 2. We believe that this step will allow us to effectively and strategically implement a more comprehensive plan to tackle the projects ahead.

Appendix

Script for presentation:

David

EunSoo

Kiara

Pram
Rei

1. Title

This is team number 1 and this is our sprint 3 presentation.

2. Contents

In this presentation, we will be going through the introduction, deliverables, scrum report, final analysis, and lastly our plan for the next sprint.

3. Introduction

This sprint started on February 17th and then lasted until March 2nd 2025. Our Scrum master for this sprint is Pram.

Our goal was to refine SQL implementations, compare Qlist1 across database catalogs, and develop a Pivot User Interface.
Some of the Key Tasks that we did were comparing Qlist1 queries across all databases. Implementing database programming to replace standard SQL queries. Also, expanding Pivot implementation and developing a structured PUI in Excel.

We developed PUI templates and ensured consistency between the databases and pivot results.
Then finally updated project management tools (Trello, Gantt charts) and compiled final reports.

4. Gantt Chart

In this Gantt Chart, you can see our updated timeline for sprint 3. We have regular meeting on Monday and Friday, so our small due dates are fixed with Monday and Fridays.
As we split the work during the Sprint Planning Time, we assigned the task and set the due dates. As the professor postponed the due date of submission, we had extra 3 days to work on the project, and had to manage the meeting time fixed to get the presentation recording.

5. Burndown Chart

In this Burndown chart, you can see the track of our project progress, and all of our teammates were able to finish the work in time.

6. Pert Chart

In this Pert chart, you will see that we started with gathering all of our requirements. Then we moved onto comparing our results from the last sprint to the queries given to us by the professor. Which leads to being able to perform database programming for the same 10 queries. The next step is working with the Pivot tables and making the PUI. Once we got that

all done, we updated the charts while working on the report. After that, we made a slideshow. Finally, we created a script which is the last step before submission.

7. Deliverables

7.1. Comparisons

For the first part of deliverables, we compared our SQL implementation in the Sprint2 with the proposed solution for Qlist1.

7.1.1. Query 1

Query 1 is to list Schema, # of tables, and # of records…

For the Oracle database, I noticed that on the document that was provided it did say that the query is incomplete. My SQL query provides a clear overview of the database by counting tables by schema and using ALL_TABLES to sum all records, making it more thorough than the professor's. The full picture is not provided by the professor's query, which simply provides a list of schema and table names without any aggregation. I would have to modify my method to use sys.schemas, sys.tables, and sys.partitions in order to make it function in a different structure. However, my query summarizes the contents of the schema more effectively overall.

For the MSSQL database, Both my SQL query and the professor’s aim is to gather schema statistics, but they take different approaches. My query is simpler—it uses a straightforward JOIN between sys.tables and sys.partitions to pull the schema name, table count, and record count for the database. It also aggregates record counts for tables where index_id <= 1. The professor’s query, on the other hand, is more advanced. It uses a cursor to loop through all online databases, automatically runs SQL for each one, and stores the results in a temporary table. While my query is focused on a single database and keeps things straightforward, the professor covers multiple databases, giving it a much broader scope.

For the DGDB database, Both queries aim to list schema-level tables and record counts but take different approaches. My query focuses on base tables, pulling schema names, table names, and record counts by joining system views like sys.schemas, sys.tables, and sys.partitions. The professor’s query, however, works with a different database structure, using custom tables (DG_Attributes, DG_Tables, and DG_Schema) to count attributes and records per table. While the professor’s query analyzes attributes within tables, mine is more focused on counting records in tables.

7.1.2. Query 2

Query 2 is to list table name, # of attributes, # of PK_ attr, # of FK_ attr, # of NN_attr, # of IDX_attr, # of CC_attr, # of UQ_attrs, # of PGM_attr"...

For the Oracle database, My SQL query and the professor's take different approaches to summarizing schema data. Mine focuses on aggregation by counting key attributes per table, like primary keys, foreign keys, non-nullable attributes, indexes, check constraints, and unique constraints. It then groups everything by schema and table for a quick overview, using LEFT JOIN to pull in constraints and indexes. The professor's query, on the other hand, dives into column-level details, listing every column along with its schema, table name, column ID, and constraints—without aggregation. So while my query gives a big-picture summary, the professor goes deep into individual columns for more detailed insights.

For the MSSQL database, The methods used by the professor and me for aggregating schema attributes differ. In a single database, my query collects information on important attributes such as primary keys (PK), foreign keys (FK), not null (NN), indexes (IDX), check constraints (CC), unique constraints (UQ), and PGM attributes using metadata from INFORMATION_SCHEMA and sys views. In one step, it counts these attributes for every table by combining several joins and conditionals.

However, the professor's question adopts a more structured strategy. It independently counts each type of constraint (PK, FK, attributes, and indexes) and executes distinct queries for the various schemas (ZEOTA, PROP, and SP). This clearly shows that while my approach compiles everything into a single query for each schema across several databases, the professor's method offers more accurate counts for various databases.

For the DGDB database, Although they use other structures and techniques, both queries seek to list metadata tables and attribute numbers. My query directly counts attributes like PK, FK, and NN from DG_Tables and DG_Attributes and joins other tables to collect constraint data. Similar work is done by the professor's query, which uses outer joins for each constraint table and groups by schema and table name. Both queries concentrate on the same essential characteristics, even though they employ different join techniques and table structures. The contrast lies in aggregation; my query counts Attributes_ID, whereas the professor uses different identifiers, such as pk_id and fk_id.

7.1.3. Query 3

Query 3 is to list Table_ID, Table_Name, attribute_id, attribute_Name, PK_ID, FK_ID, NN_ID, constraint_type, and constraint_Name for every schema.

For the Oracle database, The professor's query and my SQL query take different approaches to identifying tables that lack a primary key (PK). My query retrieves table names from all_tables and uses a LEFT JOIN on all_constraints to check if a table has a PK. If no PK is found (c.table_name IS NULL), the table is included in the results. The professor's query improves performance by using EXISTS instead of LEFT JOIN. Instead of joining tables and

checking for NULL values, it directly excludes tables that have a PK. Additionally, the professor's query supports multiple schemas, whereas mine requires manual schema modification.

For the MSSQL database, my query retrieves detailed column-level constraint information using sys.tables, sys.columns, sys.key_constraints, and sys.foreign_keys. It identifies primary keys and foreign keys using LEFT JOIN operations, ensuring that every column's constraint details are visible. Additionally, it labels constraint types explicitly using ISNULL to handle missing constraint names gracefully. The professor's query takes a more modular approach by leveraging Common Table Expressions (CTEs) to separate different constraint types into independent queries. These are then joined together to produce a structured final result. The use of CTEs enhances readability and improves maintainability, making it easier to modify or extend the query in the future.

For the DGDB database, my query follows the same approach as in MSSQL, retrieving constraint details from system tables. I use LEFT JOIN operations on sys.tables, sys.columns, and sys.key_constraints to ensure that PKs, FKs, and constraints are correctly associated with each column. The professor's query, on the other hand, is optimized for DGDB's unique schema. It directly queries DG_Schema, DG_Tables, and DG_Attributes_* tables, which store constraint relationships more explicitly. The professor's query benefits from DGDB's structured metadata storage, making it more efficient in retrieving constraints.

7.1.4. Query 4

Query 4 is to list tables without a primary key, with a primary key but without a foreign key, and with a primary key but without an index.

For the Oracle database, My approach to schema analysis in Oracle retrieves table constraints by performing LEFT JOIN operations on all_constraints and all_indexes. The logic ensures that tables missing PKs, tables with PKs but no FKs, and tables with PKs but no indexes are correctly identified. The professor's query simplifies constraint checking using EXISTS rather than LEFT JOIN, making it potentially more efficient. Additionally, while my query requires a specific schema name to be updated manually, the professor's query includes multiple schemas (BOOK, COMPANY, etc.) in a predefined list.

For the MSSQL database,my query retrieves schema analysis details by directly checking constraints in sys.tables, sys.foreign_keys, and sys.indexes. I use LEFT JOIN operations to determine missing constraints and indexes, ensuring that results are sorted at the table level. The professor's query, however, divides its logic across different schemas (SP, ZEOTA, PROP) and executes multiple schema-specific queries. Instead of using sys.objects or sys.indexes directly, it leverages INFORMATION_SCHEMA.TABLE_CONSTRAINTS, which provides an alternative metadata source.

For the DGDB database, my query retrieves tables without PKs, tables with PKs but without FKs, and tables with PKs but without indexes using LEFT JOIN operations on sys.tables, sys.key_constraints, and sys.indexes. The professor's query, on the other hand, uses DG_Schema, DG_Tables, and specialized attribute constraint tables (DG_attributes_PK, DG_attributes_FK, DG_attributes_Index). It directly extracts tables that meet specific constraint conditions, making it more tailored to DGDB's structure.

7.1.5. Query 5

7.1.5.1. A

Moving on to Query 5 which is ranking and listing not-null assigned databases, and tables.

For SQLDEVELOPER, Our query and the sample query differ in schema selection, structure, and naming. We focus on 10 schemas, while the sample query covers 17, excluding unavailable schemas like IT and FAOES to prevent errors.

Structurally, our query nests results before sorting, while the sample query sorts directly. Naming also differs—ours uses schema_name and nn_count, while the sample query keeps owner and num_nn_constraints.

Due to these differences, results vary—FAOES ranks highest in the sample query, while ZeotaDB ranks highest in ours, reflecting the schemas included in each query.

For MSSQL,Our query and the sample query differ in approach and efficiency. Our query uses INFORMATION_SCHEMA.COLUMNS, iterates with a cursor, and combines results with UNION ALL, making it simple but less efficient. The sample query uses sys.columns, sys.tables, and sys.schemas, improving performance by avoiding metadata abstraction.

Additionally, the sample query stores results in a temporary table and applies RANK() OVER for structured ranking, while our query sorts results in descending order. However, the sample query currently assigns all ranks as 1, indicating a ranking issue. If performance and ranking are priorities, the sample query is preferable; if simplicity and flexibility matter, our query is a better choice.

Both queries for DGDB count NOT NULL constraints but differ in approach. The sample query uses RANK() OVER for structured ranking and INNER JOINs, excluding schemas with no NN constraints. Our query uses LEFT JOINs, ensuring all schemas are included, making it more comprehensive. The sample query counts distinct NN constraints via a subquery, while ours directly counts Constraint_Type = 'NN' and sorts by NN count. If ranking is the priority, the sample query is better; for a broader view, our query is more informative

7.1.5.2. B

Both SQLDEVELOPER queries retrieve schema, table, and column information but differ in sorting, schema selection, and naming conventions. The sample query sorts by column_id,

reflecting internal structure, while our query sorts alphabetically by column_name. Our query also renames owner as schema_name for clarity, whereas the sample query uses default names. Additionally, the sample query filters 17 schemas, while ours focuses on 10, leading to variations in results.

MSSQL scripts retrieve non-nullable column information but differ in structure and flexibility. Our script uses a cursor to iterate through a fixed list of databases, constructing a single dynamic SQL statement executed with sp_executesql. This makes it efficient for predefined databases but less adaptable to changes.

The sample script, however, dynamically queries all online databases using sys.databases, storing results in a temporary table for easier aggregation. It uses system views like sys.columns and sys.tables, while ours relies on INFORMATION_SCHEMA.COLUMNS. Our script is ideal for controlled environments, while the sample script is better for scalability and dynamic databases.

For DGDB, Our query and the sample query both count NOT NULL constraints but differ in approach. The sample query ranks schemas using RANK() OVER and relies on INNER JOINs, excluding schemas with no constraints. Our query uses LEFT JOINs, ensuring all schemas are included. The sample query counts distinct constraints via a subquery, while ours directly counts Constraint_Type = 'NN' and sorts by NN count. If ranking is key, the sample query works best; for a broader view, our query is more comprehensive.

7.1.6. Query 6

In Query 6, for SQLDEVELOPER, Both queries aim to find tables without primary key constraints but differ in their methods. Our query uses a LEFT JOIN between ALL_TAB_COLUMNS and ALL_CONSTRAINTS, checking for NULL constraint names, and groups the results. The sample query uses a NOT EXISTS subquery, which generally offers better performance, especially with larger datasets, by stopping once a match is found. While both return the same result, the sample query is typically more efficient.

Both MSSQL queries identify tables without primary key constraints across multiple databases, but they differ in structure and execution. The sample query again uses a stored procedure with a cursor to iterate over all user databases, storing results in a temporary table for better control over data processing. This method is ideal for larger datasets but adds complexity.

Our query, on the other hand, iterates over a predefined list of databases, dynamically builds a single SQL statement using LEFT JOIN, and combines results with UNION ALL. This method is simpler and faster for smaller datasets but less flexible for larger or dynamic environments.

For DGDB, The sample query creates a stored procedure that uses a cursor to iterate over multiple databases, dynamically constructs SQL to find tables without primary keys, and stores results in a temporary table. This approach is flexible for querying across multiple databases but adds complexity due to dynamic SQL and temporary table management.

Our query, in contrast, operates within a single database (DGDB), using a straightforward LEFT JOIN to find tables without primary keys. It's simpler and more efficient for single-database scenarios but lacks the multi-database flexibility of the sample query.

7.1.7. Query 7

For Oracle In sprint2, the SQL query implemented to identify tables without indexes was implemented by creating two Common Table Expressions (CTEs). The first CTE, named TableList, retrieved the schema name, table name, and object name from the all_tables and all_tab_columns views, ensuring that the table and column information are matched based on the table name and owner. It filtered out system schemas and schemas starting with 'SYS' to focus on user-defined schemas. The second CTE, named IndexList, retrieved the table name as object_name and the index name from the all_tables and all_indexes views, ensuring that the table and index information are matched based on the table name and owner. The main query then selected the schema name and table name from the TableList CTE and performed a LEFT JOIN with the IndexList CTE on the object_name to find tables without indexes.
The proposed query selects the owner and table_name from the all_tables view. The WHERE clause includes a subquery that checks for the existence of indexes on each table by matching the owner and table_name in the all_indexes view. If no matching index is found, the table is included in the result set. After comparison, every record match between the two queries except for the three additional tables in FAOES in the proposed query.

For the SQL Server, A cursor is used to iterate through each database, excluding system databases and 'DGDB'. For each database, a dynamic SQL statement which uses CTEs to create a list of tables and their associated indexes is constructed and executed. The main query then inserts into a temporary table the names of tables that do not have any associated indexes. The results are then selected from the temporary table, and the temporary table is dropped.
The proposed query on the other hand declares variables and uses a cursor to iterate through a predefined list of databases. For each database, a dynamic SQL statement is constructed and executed. The statement selects the database name, schema name, and table name from the sys.tables and sys.schemas views, performing a left join with the sys.indexes view to find tables without indexes. The proposed query directly selects from the system views and limits the scope to a predefined list of databases. This makes it more efficient, simple and straightforward.

Lastly the DGDB performs a left join between these lists to find tables without indexes and inserts the results into a temporary table. It then selects and orders the results from the temporary table before dropping it. The proposed query is implemented in a more straightforward manner. It directly queries the DG_Tables, DG_Attributes, and DG_Attributes_Index tables to find tables without indexes. The query joins these tables and filters the results where IDX_ID is empty. This approach is simpler and more efficient as it avoids the use of cursors and temporary tables.

7.1.8. Query 8

For Oracle In sprint2, designed to list tables with primary key (PK) but no foreign key (FK) used three Common Table Expressions (CTEs) to do the implementation. The TableList CTE retrieved the schema name, table name, and object name from the all_tables and all_tab_columns views, filtering out system schemas. The PKList CTE retrieved the table name and primary key constraint name from the all_constraints view, filtering for primary key constraints. The FKList CTE retrieved the table name and foreign key constraint name from the all_constraints view, filtering for foreign key constraints. The main query then joins these CTEs to find tables that have a primary key but do not have a foreign key.
The proposed query on the other hand selects the owner and table name from the all_tables view, using subqueries to check for the existence of primary key constraints and the non-existence of foreign key constraints in the all_constraints view. It filters the results to include only those tables whose owners are in a specified list of schemas and limits the result set to the first 20 rows. The proposed query uses subqueries within the main query to achieve the same result as the sprint2 query. This can be more efficient in terms of execution but may be harder to read and maintain. Both queries however achieve the same results.

For the SQL server, A cursor is used to iterate through each database, excluding system databases and 'DGDB' and then integrating it with a combination of temporary tables. The proposed query implements a stored procedure named ListTablesWithPKWithoutFK. The procedure includes the logic to list tables with primary keys but without foreign keys. Inside the procedure, a temporary table is created to store the results. A cursor is used to iterate through a predefined list of databases, excluding system databases and 'DGDB'. The results are inserted into the temporary table, which is then selected and dropped at the end of the procedure.
The proposed query use of a stored procedure to encapsulate the logic makes it reusable and easier to manage. It limits the scope to a predefined list of databases, which can be more efficient and straightforward. Both queries effectively identify tables with primary keys but without foreign keys, but they differ in their approach and complexity.

The DGDB solution in sprint2 also uses a cursor to iterate through each database. The query uses CTEs to create a list of tables, a list of tables with primary keys (PKList), and a list of

tables with foreign keys (FKList). It then performs an inner join between TableList and PKList and a left join with FKList to find tables with primary keys but without foreign keys, and inserts the results into a temporary table. The proposed query directly queries the DG_Schema, dg_tables, DG_Attributes, DG_Attributes_PK, and DG_Attributes_FK tables to find tables with primary keys but without foreign keys. The query joins these tables and filters the results where PK_ID is not null and fk_id is null. This approach is simpler and more efficient as it avoids the use of cursors and temporary tables.

7.1.9. Query 9

Query 9 is to get a list of tables with index, constraint name, and fields used in index for every database/schema.

For the Oracle database, both queries list indexes, their associated tables, names, and indexed columns but differ in approach, filtering, and aggregation, resulting in 191 rows for the Sprint 2 query and 270 rows for the proposed query. The Sprint 2 query uses LISTAGG to concatenate indexed columns into a single row, groups results by schema, table, and index, and filters by a specific set of schemas, producing a condensed dataset. In contrast, the proposed query retrieves distinct rows for each indexed column without aggregation, includes additional schemas, and provides a more detailed, column-level breakdown of indexes. These differences highlight the trade-off between a compact, high-level view (Sprint 2) and a granular, detailed analysis (proposed), with the schema filtering and aggregation methods driving the variation in result counts.

For the MSSQL database, the sprint 2 query focused on gathering indexing details, such as table names, index types, and columns, from specific databases using dynamic SQL and a cursor, resulting in 168 records. In contrast, the proposed query expanded the scope by creating a stored procedure that collected both indexing and constraint data, such as primary keys and foreign keys, across a broader set of databases, yielding 568 records. While the Sprint 2 query was limited to indexing information, the proposed query provided a more comprehensive analysis by including constraints, leading to a significantly larger result set.

For the DGDB database, the Sprint 2 query focuses on a detailed, column-specific analysis of indexes by joining system tables like sys.tables, sys.indexes, sys.index_columns, and sys.columns. It retrieves table names, index names, index types, column names, and uniqueness, targeting predefined tables related to database schema, attributes, and auditing, with results ordered for clarity. In contrast, the proposed query adopts a broader, schema-based approach by joining user-defined tables like DG_Schema, dg_tables, and DG_Attributes_Index, providing a higher-level view of index constraints across schemas and tables without detailing individual columns. While Sprint 2's query is ideal for granular, table-specific index analysis, the proposed query's offers scalability for schema-level insights, making it more suitable for broader database management and analysis.

7.1.10. Query 10

Query 10 is to list tables with check constraints for every database/schema.

For the oracle database, the Sprint 2 query returns 191 results, while the proposed query returns 113. Sprint 2's query centers on indexing, joining `all_indexes` with `all_ind_columns` and indirectly including constraints via a `LEFT JOIN` with `all_constraints`. It does not explicitly filter for check constraints, resulting in a broader dataset that includes tables with and without them. In contrast, the proposed query directly targets tables with check constraints (`constraint_type = 'C'`) using `EXISTS` and filters schemas differently, yielding a more specific and refined output. This difference in scope—indexing versus constraint-specific filtering—explains the discrepancy in result counts.

For the MSSQL database, the Sprint 2 query uses dynamic SQL with a cursor to loop through a predefined list of databases (e.g., Book, Company, DGDB) and retrieves check constraints from system views like `sys.check_constraints` and `sys.tables`. It builds and executes dynamic SQL statements for each database, making it suitable for targeted database sets. In contrast, the proposed query introduces a stored procedure, `ListTablesWithCheckConstraint`, which is more flexible and scalable. It creates a temporary table to store results, loops through all user databases (excluding system databases), and uses `INFORMATION_SCHEMA` views to fetch check constraints, making it adaptable for broader environments. The proposed query approach is more structured, reusable, and standardized, while the Sprint 2 query is better suited for specific, fixed database sets.

For the DGDB database, the query from Sprint 2 retrieves distinct combinations of schema and table information while indirectly checking for check constraints. The Sprint 2 query joins the DG_Schema, dg_tables, and DG_Attributes tables and uses a LEFT JOIN with DG_Attributes_ConstCheck to filter results where CC_ID is not null, indicating the presence of check constraints. However, it lacks performance optimization and clarity in relationships between schema, table, and constraint data. In the proposed query, the query is improved by adopting a more precise and streamlined approach, ensuring all tables are included via LEFT JOIN but only returning those with valid check constraints through the WHERE clause. This refinement enhances clarity, accuracy, and performance, making the proposed query the more optimal solution for listing tables with check constraints.

7.2. QList 1 Database Programming

For the second part, we repeated SQL implementation done in Sprint2 but replace it with database programming.

7.2.1. Query 1

Query 1 is to list Schema, # of tables, and # of records…

For the Oracle database, This procedure works by looping through the schemas, running queries to get the table count and record count, and then printing the results. This makes it easy to get a quick overview of how much data is stored in each schema without running multiple separate queries.

For the MSSQL database, By figuring out how many tables and records there are in each schema, this SQL process seeks to provide metadata statistics across all databases. To obtain this data, it successively creates and runs SQL queries in each database, then saves the outcomes in a temporary table. A summary of the database name, schema name, table count, and record count is produced after all databases have been processed. The outcome shows that while some schemas, like db_accessadmin, display zero tables and records, others, like db_owner or dbo, have available table and record counts.

For the DGDB database, This procedure creates a temporary table to store the schema name, table count, and record count. After going through the table, it inserts values by joining the sys.schemas, sys.tables, and sys.partitions system views. For example, the dbo schema has 6 tables with a total of 4 records, while schemas like guest and sys have no tables or records.

7.2.2. Query 2

Query 2 is to list table name, # of attributes, # of PK, FK, NN, IDX, CC, unique, and PGM"...

For the Oracle database, this procedure breaks down each schema to show the structure of its tables, including attributes and constraints like primary keys, foreign keys, and not-null rules. It loops through schemas, pulling detailed constraint data to give a clear picture of how each table is structured. The goal is to provide a full view of database constraints, but execution issues—like connection resets—can happen due to the complexity of the query.

For the MSSQL database,This procedure gathers table and constraint details for all databases. It counts attributes, primary keys, foreign keys, not nulls, indexes, check constraints, and unique constraints for each table in each schema. The results show the database name, schema, table, and constraint counts, like AUTHOR in book with 3 attributes, 1 primary key, and 1 index, or VENDORPRICE in ZeotaDB with 5 attributes, 1 primary key, and 2 foreign keys.

For the DGDB database, The dbo.GetSchemaTableDetails procedure provides a summary of each schema's tables and a number of other attributes. For instance, the MSreplication_options table in the dbo schema has 36 attributes with 1 index and 6 non-nullable attributes, while sysdiagrams has 15 attributes with 1 primary key and 1 unique constraint. The results are organized by schema and table name.

7.2.3. Query 3

For query 3, the objective is to extract and list schema constraints, including the table ID, table name, attribute ID, attribute name, primary key (PK), foreign key (FK), not null (NN), constraint type, and constraint name.

For Oracle, we developed the GetColumnConstraints procedure. This procedure queries the ALL_TAB_COLUMNS and ALL_CONSTRAINTS tables to retrieve column constraints. It joins multiple subqueries to identify primary keys, foreign keys, and other constraints. The results are then printed using DBMS_OUTPUT.PUT_LINE.

For MSSQL, we developed the GetColumnConstraints stored procedure. The procedure joins sys.tables, sys.columns, sys.indexes, and sys.foreign_key_columns to extract constraint details. The results are structured to show whether a column is a primary key, foreign key, or has any constraints applied.

Lastly, the DGDB procedure closely mirrors the MSSQL procedure. The script retrieves table constraints using sys.tables, sys.columns, sys.indexes, and sys.foreign_key_columns. It ensures compatibility with DGDB's system views and constraint handling.

7.2.4. Query 4

a)

For query 4a, the objective is to list tables without a Prime key for every schema.

In Oracle, The procedure ListTablesWithoutPK accepts an input parameter p_owner, which specifies the schema. Also it selects table names from all_tables that do not have an associated primary key in all_constraints. Lastly, a cursor c_tables_no_pk is used to iterate through the result set, and the tables without primary keys are displayed using DBMS_OUTPUT.PUT_LINE.

In MSSQL, the query retrieves all user tables from sys.tables. It performs a LEFT JOIN with sys.key_constraints to check for primary keys (type = 'PK'). Tables where pk.parent_object_id IS NULL (meaning they lack a primary key) are included in the output.

Lastly, in DGDB, the stored procedure ListTablesWithoutPK follows the same logic as the MSSQL version.

It queries sys.tables and sys.key_constraints to find tables without primary keys. A LEFT JOIN ensures only tables with no primary key constraints are included in the result.

b)

For query 4b, the objective is to list tables with the Prime key but without the Foreign key for every schema.

In Oracle, we created a stored procedure called ListTablesWithPKNoFK. This procedure uses a cursor to fetch table names. It joins user_tables with user_constraints to identify tables that have a primary key (PK). It performs a LEFT JOIN with user_constraints again to check for foreign keys (FK). If a table does not have an FK (fk.table_name IS NULL), it is included in the output.

In MSSQL, we initially attempted to find tables with PK but no FK, but the query instead checked for tables with PK but no indexes (ListTablesWithPKNoIndex). The query joins sys.tables with sys.key_constraints to filter tables with primary keys. It then uses a LEFT JOIN with sys.indexes to check for missing indexes. If no index exists (i.object_id IS NULL), the table is selected.
In DGDB, it works similarly to the Oracle procedure, checking for tables with a primary key but no foreign key. The query joins sys.tables with sys.key_constraints (to identify PKs) and left joins sys.foreign_keys (to check for missing FKs). If fk.parent_object_id IS NULL, the table is selected.

c)
For query 4c, the objective is to list tables with the Prime key but without IDX for every schema.
In Oracle, the procedure uses a cursor to fetch table names that match our criteria. It joins user_tables with user_constraints to identify tables with a primary key (PK). Also, it performs a LEFT JOIN with user_indexes to check if an index exists. If no index exists (i.table_name IS NULL), the table is selected.
In MSSQL, the query selects tables from sys.tables that have a primary key (PK) using sys.key_constraints. It then performs a LEFT JOIN with sys.indexes to check if an index exists for each table. If the table has a PK but no associated index (i.object_id IS NULL), it is included in the results.
Lastly in DGDB, we used the same approach as in MSSQL by creating the ListTablesWithPKNoIndex procedure. It joins sys.tables with sys.key_constraints to find tables with PKs. and it performs a LEFT JOIN with sys.indexes to check for missing indexes. If a table does not have an index (i.object_id IS NULL), it is included in the results.

7.2.5. Query 5

7.2.5.1. A

For Query 5A, after converting these queries, the discrepancy of outputs shown is consistent with the outputs that were created for our previous sprint.

For SQLDeveloper, I'll walk you through a PL/SQL procedure called Get_Non_Nullable_Column_Count, which helps analyze database schemas by counting non-nullable columns. This procedure begins by inserting data into a temporary table, temp_nn_count, where it groups and counts non-nullable columns from the ALL_TAB_COLUMNS system view. It filters the results to focus only on specific schemas, such as 'BOOK', 'IT4', and 'ZEOTADB', and sorts them in descending order. After inserting the data, a message confirms the operation, and a reference cursor (p_cursor) is opened to retrieve and display the results. The script also includes an anonymous PL/SQL block that calls the procedure, fetches data from the cursor, and prints the schema name along with its corresponding count of non-nullable columns. Finally, the procedure ensures that

temp_nn_count is cleared for subsequent runs. This approach provides a structured way to assess database schemas and maintain efficient data retrieval in Oracle databases.

For MSSQL query however, This procedure dynamically constructs and executes a SQL query to analyze schemas from a predefined set of databases, such as 'BOOK', 'COMPANY', and 'ZEOTADB'. It uses a cursor to iterate through these databases, building a SELECT statement that retrieves the schema name and the count of non-nullable columns from INFORMATION_SCHEMA.COLUMNS. The queries for each database are combined using UNION ALL, ensuring that results from multiple databases are merged efficiently. The final query is sorted by the count of non-nullable columns in descending order. Once the SQL query is fully assembled, it is executed using sp_executesql. This approach provides a dynamic and scalable way to assess database structures, helping developers and administrators quickly identify schemas with the highest number of required fields across multiple databases.

In terms of DGDB, the stored procedure executes a query that joins four key tables: DG_Attributes, which contains attribute details; DG_Attributes_NotNull, which identifies attributes with 'NOT NULL' constraints; DG_Tables, which links attributes to their respective tables; and DG_Schema, which provides schema-level organization. The query filters attributes with the constraint type 'NN' and groups results by schema, ordering them in descending order based on the count of non-nullable attributes. This procedure helps database administrators quickly assess the distribution of mandatory fields across schemas, aiding in data quality analysis and schema optimization.

7.2.5.2. B

For SQLDEVELOPER, this stored procedure helps identify non-nullable columns within specific database schemas. This procedure works by opening a reference cursor (p_cursor) that retrieves schema names, table names, and column names from the ALL_TAB_COLUMNS system view. It filters the results to include only non-nullable columns (NULLABLE = 'N') from selected schemas such as 'BOOK', 'IT4', and 'ZEOTADB', and orders the output by schema, table, and column name. The procedure is then called in an anonymous PL/SQL block, which iterates through the cursor, fetching and printing each schema, table, and column combination. This structured approach enables efficient analysis of database constraints, helping developers and database administrators understand which columns must always have values across multiple schemas.

Now looking into MSSQL query, This procedure dynamically generates and executes a SQL query to retrieve schema names, table names, and column names from a predefined list of databases, such as 'BOOK', 'COMPANY', and 'ZEOTADB'. It uses a cursor to iterate through each database, constructing a SELECT statement that queries INFORMATION_SCHEMA.COLUMNS for columns where IS_NULLABLE is set to 'NO'. The

results from each database are combined using UNION ALL, and once the loop completes, the final query is executed using sp_executesql. By dynamically handling multiple databases, this procedure provides database administrators and developers with a comprehensive view of required fields across different schemas, aiding in data integrity analysis and schema auditing.

For DGDB, This procedure starts by setting NOCOUNT ON to improve performance by suppressing row count messages. It then executes a query that joins four key tables: DG_Attributes, which contains attribute details; DG_Attributes_NotNull, which identifies attributes with 'NOT NULL' constraints; DG_Tables, which maps attributes to their respective tables; and DG_Schema, which provides schema-level context. The procedure filters results to include only attributes from the schemas 'SPJ2', 'ZEOTA', and 'BOOK'. By executing this stored procedure, users can quickly identify required fields across multiple schemas, ensuring data consistency and integrity.

7.2.6. Query 6

For Query 6 again the output for this query across all DBMS remains consistent after DB Programming conversion.

This PL/SQL query procedure opens a reference cursor (p_cursor) to execute a query that retrieves schema names and table names from the ALL_TAB_COLUMNS system view. It uses a LEFT JOIN with ALL_CONSTRAINTS to check for primary key constraints (CONSTRAINT_TYPE = 'P'). If no primary key is found (ac.CONSTRAINT_NAME IS NULL), the table is included in the results. The procedure filters for specific schemas, such as 'BOOK', 'IT4', and 'ZEOTADB', and organizes the output in ascending order by schema and table name. The procedure is then called in an anonymous PL/SQL block, which iterates through the cursor to display each schema and its corresponding table without a primary key. This method provides valuable insight for database administrators and developers, helping to identify potential issues with data integrity and normalization.

For MSSQL query, Using a cursor, it iterates through each database, constructing a query that retrieves schema names and table names from INFORMATION_SCHEMA.TABLES. It performs a LEFT JOIN with INFORMATION_SCHEMA.TABLE_CONSTRAINTS to check for primary key constraints. If no primary key is found (tc.TABLE_NAME IS NULL), the table is included in the results. The queries for each database are combined using UNION ALL, ensuring a unified output. Before execution, the procedure ensures that only online databases are processed. Finally, the dynamically generated SQL is executed using sp_executesql.

Moving on to DGDB, This procedure queries multiple joined tables within DGDB to identify attributes with a 'NOT NULL' constraint. It joins DG_Attributes, which contains attribute details, with DG_Attributes_NotNull, which holds constraint information, and DG_Tables,

which maps attributes to tables. Additionally, it joins DG_Schema to associate attributes with specific schemas. The query filters results to include only schemas of interest, such as 'SPJ2', 'ZEOTA', and 'BOOK'. By executing this procedure, database administrators can efficiently retrieve a list of mandatory fields across multiple schemas, ensuring data completeness and integrity.

7.2.7. Query 7

For oracle databases, the PL/SQL stored procedure lists all tables without indexes for specific schemas. The procedure iterates through each schema specified in the owner list using a cursor. For each schema, it declares a variable v_table_name to store the table name. Another cursor is used to iterate through all tables in the current schema that do not have any associated indexes. This is determined by checking if there is no matching entry in the all_indexes view for the table. For each table without an index, the procedure outputs the schema name and table name.

For the SQL server, The stored procedure also iterates through a list of specified databases and lists tables that do not have any indexes. It uses a cursor to loop through each database, constructs a dynamic SQL query to find tables without indexes, and executes the query for each database. This approach allows for flexible and reusable code to handle similar tasks across multiple databases. The stored procedure in this database returns zero records.

For the DGDB, The stored procedure has been specifically made to process only DGDB. The query is similar to the previous procedure on MSSQL apart from only pointing at the DGDB instead of all the other databases.

7.2.8. Query 8

For oracle databases, the stored procedure to get tables with PK but without FK, the procedure is implemented by iterating through specified database schemas and lists tables that have primary keys but do not have foreign keys. It prints the schema and table names to the console. The procedure is structured with nested loops to handle each schema and its corresponding tables, making it reusable and flexible for similar tasks.

For the implementation of the same in SQL server, the stored procedure uses a cursor to loop through each database and constructs a dynamic SQL query to find the relevant tables. The results are stored on a temporary table and then selected and displayed. The procedure returns 35 records.

And lastly For the DGDB, The stored procedure has been specifically made to process only DGDB. The query is similar to the previous procedure on MSSQL apart from only pointing at the DGDB instead of all the other databases.

7.2.9. Query 9

Query 9 is to get a list of tables with index, constraint name, and fields used in index for every database/schema.

For SQL developer, the script defines a custom type, `indexed_column_rec`, to store indexed column details, and a collection type, `indexed_column_table`, to hold multiple records of this type. It then creates a pipelined function, `get_indexed_columns`, which retrieves distinct indexed column information from `all_indexes` and `all_ind_columns` for specific schemas, and streams the results row by row using `PIPE ROW`. Finally, a procedure, `fetch_top_indexes`, is created to fetch a limited number of rows from the pipelined function and return them via an output cursor. This approach provides a structured and efficient way to retrieve and process indexed column data.

For the MSSQL database, the script creates a stored procedure, `dbo.GetConstraintColumnUsageAllDatabases`, in SQL Server to retrieve constraint column usage information across all user databases. It uses a cursor to iterate through online databases, dynamically constructs SQL queries to fetch data from the `INFORMATION_SCHEMA.CONSTRAINT_COLUMN_USAGE` view, and stores the results in a temporary table. Errors during database access are caught and logged without interrupting the process. Finally, the procedure returns the collected data, ordered by database name, and cleans up the temporary table.

For the DGDB database, the script creates a table-valued function, `dbo.GetSchemaTableInfo`, in the `DGDB` database, which returns schema and table information by joining the `DG_Schema` and `dg_tables` tables. It then creates a stored procedure, `dbo.GetIndexedAttributes`, that uses this function to retrieve schema and table details, joins them with indexed attribute information from `DG_Attributes` and `DG_Attributes_Index`, and returns the results ordered by table name. Both the function and procedure are designed to provide a clear and organized view of indexed attributes within the database. If the function or procedure already exists, they are dropped and recreated to ensure the latest logic is applied.

7.2.10. Query 10

Query 10 is to list tables with check constraints for every database/schema.

For SQL Developer, the script drops existing objects (if they exist) and defines a custom type, `constrained_table_rec`, and a collection type, `constrained_table_list`, to store table information. It creates a pipelined function, `get_constrained_tables`, to retrieve tables with check constraints from specific schemas and streams the results row by row. A procedure, `fetch_top_constrained_tables`, is then created to fetch a limited number of rows from the

function and return them via a cursor. Finally, an anonymous block executes the procedure, fetches the results, and prints them, providing a streamlined way to retrieve and display tables with check constraints.

For the MSSQL database, the script creates a stored procedure, `dbo.ListTablesWithCheckConstraint`, in SQL Server to list tables with CHECK constraints across all user databases, excluding a specific database named. It uses a cursor to iterate through online databases, dynamically constructs SQL queries to fetch CHECK constraint details from `INFORMATION_SCHEMA.TABLE_CONSTRAINTS` and `INFORMATION_SCHEMA.TABLES`, and stores the results in a temporary table. Errors during database access are caught and logged without stopping execution. Finally, the procedure returns the collected data, ordered by database, schema, and table names, and cleans up the temporary table.

For the DGDB database, The script creates a stored procedure, `dbo.GetTablesWithConstraints`, in the `DGDB` database, which retrieves distinct schema and table information along with associated constraints. It joins the `DG_Schema`, `dg_tables`, and `DG_Attributes` tables, and uses a `LEFT JOIN` with `DG_Attributes_ConstCheck` to include tables with or without check constraints. The procedure is designed to provide a clear view of tables and their relationships to constraints within the database. If the procedure already exists, it is dropped and recreated to ensure the latest logic is applied.

7.3. Pivot

7.3.1. We worked to modify the pivot worksheet developed in Sprint2 to match the proposed solution for Qlist1

7.3.1.1. Query 1

Query 1 is to list Schema, # of tables, and # of records. In order to count all of the tables in each database, we applied the distinct count on the Table_names so duplicates were not counted. One downside to using static raw data is that we are not able to count the number of records without loading each table into the excel sheet.

7.3.1.2. Query 2

Query 2 is to list table name, # of attributes, # of primary keys, # of foreign keys, # of not null, # of IDX, # of check constraint, # of UQ, # of PGM_attr". To do this I had to put some of the attributes into their own tables so it was simpler to see. We put the schema name and table names in the rows section and the rest of the calculated attributes into different columns. We also added a slicer to be able to choose which database that you wanted to look at.

7.3.1.3. Query 3

To check the list of schema constraints, including the table ID, table name, attribute ID, attribute name, primary key (PK), foreign key (FK), not null (NN), constraint type, and constraint name,
we build a pivot table by putting database name, table name, and column name to rows section, constraint type to columns section, and lastly count of table id, constraint name, and attribute id to values section. We also used a slicer to list the tables by Oracle, MSSQL, and DGDB.
These are the result of pivot tables.

7.3.1.4. Query 4

To check the list of tables without PK, with PK but without FK, and with PK but without IDX for every schema, we build a pivot table using has_primary_key, has_foreign_key, and IDX column.
We put database name and table name in the rows section, and put count of column name to values section. And then we put a filter with has_primary_key, has_foreign_key, and IDX, so we can change the setting to see if the tables have PK, FK, or IDX.

7.3.1.5. Query 5

Query 5 ranks and lists all tables with Not-nulls, for query 5A we set our pivot table field settings to host the Nullable field as both values and filter, while assigning Database_name row. The result of this pivot concludes that for all DBMS, the output is still consistent with the stored procedure and our previous sprint results.

For 5B, we managed to list all tables and columns that are assigned with not-nulls across three DBMS. We completed this task by assigning Nullable into both values and filter then assigning that filter to NOT NULL setting, while assigning database_name, table_name, and column_name into rows. This creates a dropdown feature where we can see where each column that was assigned not null belongs to which table and databases.

7.3.1.6. Query 6

For Query 6 this pivot table tries to query all databases and tables that have no Primary Keys assigned across all DBMS. We were able to complete this task by assigning has_primary_keys as a filter then assigning that filter to NO setting, and database_name and table_name as its rows. The output that we see here is consistent to our previous sprint and stored procedures.

7.3.1.7. Query 7

Query 7 is to get a list of tables without indexes for every database/schema. To do this, in the pivot table fields, we put the index in filters, database name and table name in rows, with the database management system in the slicer. The index filter is set to NULL and Blank.

The Oracle database resulted in 58 records, MSSQL resulted in zero records and the DGDB database resulted in zero records too.

7.3.1.8. Query 8

Query 8 is to get a list of tables without foreign keys but has primary keys for every database/schema. To do this, in the pivot table fields, we put the has_primary_key and has_foreign_key in filters, database name and table name in rows, with the database management system in the slicer. The has_primary_key filter is set to YES and has_foreign_key set to NO. The Oracle database resulted in 371 records, MSSQL resulted in 275 records and the DGDB database resulted in 39 records.

7.3.1.9. Query 9

Query 9 is to get a list of tables with index, constraint name, and fields used in index for every database/schema. To do this, in the pivot table fields, we put the index in filters, database name, table name in rows, with the database management system in the slicer. The index filter included everything except NULL and Blank. The Oracle database resulted in 518 rows, MSSQL resulted in 143 rows, and the DGDB database resulted in 34 rows.

7.3.1.10. Query 10

Query 10 is to list tables with check constraints for every database/schema. To do this, in the pivot table fields, we put database name, table name and constraint type in the rows and the database management system in the slicer. The Constraint Type included 'CC', 'CC, FK', 'CC, PK', 'CC, FK, PK'. The oracle database resulted in 191 rows, and MSSQL and DGDB resulted in no results.

8. Scrum Report

Here is a snippet of our Scrum Matrix. Each row represents different Scrums and their associated tasks, while each column corresponds to a team member. We divided 10 queries into 2 queries each for one team member, so we can all work on a similar amount of project. That way, we all had the common query-making work and some individual assigned work. For this time, we changed the teammates who were going to verify each other, but we also checked each team member's status at every scrum meeting to make sure everyone was on the right track.

9. Final Analysis

Now moving on to Final Analysis.

In this sprint, we replaced standard SQL queries with database programming techniques, examined metadata inconsistencies, and extended their Pivot implementation by creating an

Excel Pivot User Interface (PUI). A key objective was to conduct a comprehensive comparison and verification of metadata across the database systems, identifying discrepancies and inconsistencies in schema, table attributes, constraints, and indexing patterns. The team also focused on improving the efficiency and accuracy of metadata analysis by implementing advanced database programming techniques and automating repetitive tasks. The ultimate goal was to compile our findings into a comprehensive sprint report and presentation, documenting their methodologies, results, and recommendations for future improvements.

While this sprint is conducted, we have successfully and diligently completed this sprint within the timespan. By implementing group meetings or scrums for updates, we were able to quickly adjust to our immediate needs that were unforeseen during the planning stage. Furthermore, implementation of Microsoft Teams as a means of direct and real-time communications was really helpful for us to assist each other in times of technical problems. However, during this process, we also encounter some hurdles, the biggest hurdle is timing. Taking into account the significant uptick of responsibilities with limited time, our team struggled with time management. We acknowledge that we weren't agile enough and adjust our usual plan to fit the increase in tasks that this project requires us to do. Therefore, we are planning for the next sprints to dedicate more time in resource and task management, increasing from 1 scrum meeting to 2. We believe that this step will allow us to effectively and strategically implement a more comprehensive plan to tackle the projects ahead.

10. Next Sprint

The next sprint is from March 1st to March 19th. The goal of this sprint is to implement query lists using Pivot by connecting Pivot tables to databases as part of the Data Warehouse. This involves creating dynamic links between Pivot tables and the underlying database systems to enable real-time data analysis and reporting.

Based on the deliverables, we have split them up into 4 scrums with the following tentative internal deadlines:

- Scrum 1: combines Develop Organizational Reporting and Data Warehouse Implementations of QList 1 and QList 3 by 7th of March
- Scrum 2: Machine Learning Implementation of Fuzzy Logistics QList 2 by 10th of March
- Scrum 3: Expand Pivot User Interface by connecting pivot tables to databases QList1, 2, 3 by 14th of March
- Scrum 4: Report, presentation recording, and submission of materials by 19th of March.

Thank you!

Next Sprint

1. Sprint 4 Planning

Sprint Duration: March 1 - March 19, 11:59 pm

2. Sprint Goal

Data Warehouse implementation of query lists using Pivot (connecting Pivot tables to databases)

3. Deliverables

Scrum 1	Develop Organizational Reporting, Data Warehouse Implementations of QList 1 and QList 3	Deadline: 7th Mar
Scrum 2	Machine Learning Implementation of Fuzzy Logistics QList 2	Deadline: 10th Mar
Scrum 3	Expand Pivot User Interface by connecting pivot tables to databases QList1, 2, 3.	Deadline: 14th Mar
Scrum 4	Report, Presentation and Script	Deadline: 19th Mar